

TOPICS IN

CRYPTOGRAPHY

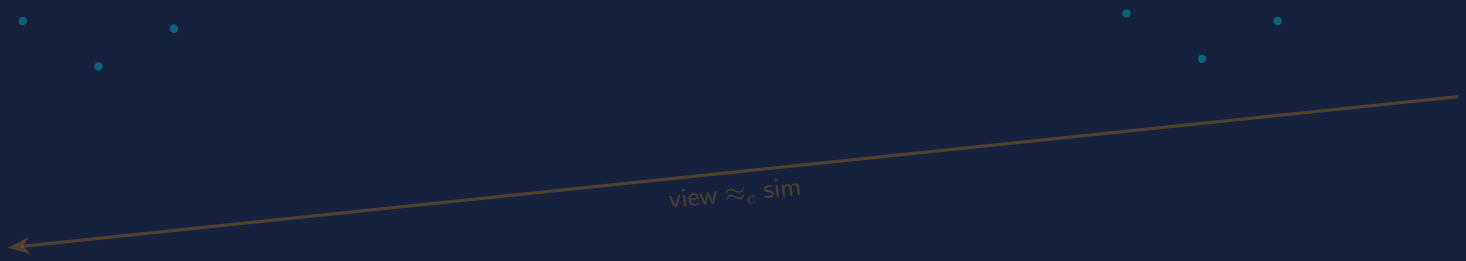
Course Notes

INTRODUCTION • SECRET SHARING • INFORMATION RETRIEVAL
RANDOMIZED ENCODING • SECURE COMPUTATION • PROOF SYSTEMS

Tel-Aviv University

Spring Semester, 2026

written by Ron Goldman



Contents

I Introduction 1

1 Introduction to Information-Theoretic Cryptography 2

1.1 Perfect Encryption [Shannon 48] . . . 2

1.1.1 Perfect Secrecy . . . 2

1.1.2 Statistical Secrecy . . . 3

1.1.3 Computational Secrecy [GM'82] . . . 3

1.1.4 Perfect Indistinguishability and the Turing Game . . . 4

1.2 One-Time Pad Is Perfectly Secure . . . 5

1.3 Efficiency Measures and Shannon's Lower Bound . . . 6

1.4 Broadcast Encryption [Fiat–Naor 94] . . . 6

1.5 Error Correction and Detection . . . 7

1.6 Distributed Storage Model . . . 7

II Secret Sharing 10

2 Threshold Secret Sharing 11

2.1 t-out-of-n Secret Sharing: Syntax . . . 11

2.1.1 Dealing Function . . . 11

2.1.2 Reconstruction Function . . . 11

2.2 Linear Secret Sharing and Hiding Properties of Linear Maps . . . 11

2.3 n-out-of-n Secret Sharing over a Finite Group . . . 12

2.4 The Case $t < n$ . . . 13

2.5 $\mathbb{Z}_p$ as a Field . . . 13

2.6 Shamir's Secret Sharing . . . 14

2.6.1 The Case $t = 2$ . . . 14

2.6.2 General t . . . 14
2.7 Recovering the Secret via Lagrange Interpolation . . . 14
2.7.1 Polynomial Interpolation . . . 15
2.8 Security of Shamir's Scheme . . . 15
2.9 Lower Bound on Share Size . . . 16
2.10 Field Size and Efficiency . . . 16
2.10.1 Other Access Structures . . . 17
2.11 Threshold-Based Key Recovery . . . 17

III Information Retrieval 18

3 Private Information Retrieval (IT-PIR) 19

3.1 Problem Formulation . . . 19
3.2 Definition of 1-Private k-Server PIR . . . 19
3.2.1 Impossibility of Non-Trivial Single-Server PIR . . . 20
3.2.2 PIR over Longer Records . . . 20
3.3 Two-Server PIR from the Hadamard Code . . . 21
3.3.1 Two Servers with $\sqrt{N}$-Bit Records . . . 21
3.3.2 Linear Secret-Sharing Formulation . . . 22
3.4 Four-Server Hypercube PIR . . . 22
3.4.1 Three-Dimensional Hypercube PIR . . . 23
3.5 Three-Server Reed–Muller PIR . . . 24
3.5.1 Interpolation Equation . . . 25
3.5.2 Reducing a Server Using Partial Derivatives . . . 25
3.5.3 Four-Server RM-PIR and Reduction to Two Servers . . . 26
3.6 Matching Vectors . . . 26
3.6.1 Hadamard as an Inner-Product Construction . . . 26

3.6.2 A Conditional Two-Server Cancellation Construction . . . 27
3.6.3 Standard Matching-Vector Families . . . 28
3.6.4 Four-Server PIR from a Standard Matching-Vector Family . . . 29
3.6.5 Further Optimizations . . . 30
3.7 From IT-PIR to Locally Decodable Codes . . . 30

4 Computational PIR (cPIR) 32

4.1 Computational Privacy and Indistinguishability . . . 32
4.2 cPIR from Fully Homomorphic Encryption . . . 33
4.3 cPIR from Additively Homomorphic Encryption . . . 34
4.3.1 Basic Construction [Kushilevitz–Ostrovsky 97] . . . 35
4.3.2 Recursive Optimization . . . 35
4.4 Two-Server cPIR from Pseudorandom Generators . . . 36
4.4.1 Additive Correction [Gilboa–Ishai 14] . . . 37
4.5 Function Secret Sharing . . . 38
4.6 Distributed Point Functions: The Boyle–Gilboa–Ishai Tree . . . 39
4.6.1 Conditional Correction Gadget . . . 40
4.6.2 Concrete Efficiency . . . 41
4.7 Applications of FSS and DPF . . . 41
4.7.1 Keyword Search . . . 41
4.7.2 Secure Aggregation . . . 42
4.7.3 Anonymous Messaging [CBM15] . . . 42
4.8 Security Analysis of a PIR-Based MPC Candidate . . . 42

IV Randomized Encoding 45

5 Conditional Disclosure of Secrets (CDS) 46

5.1 Definition and Syntax . . . 46

5.1.1 Example: XOR . . . 46
5.2 Deterministic CDS Protocols . . . 47
5.3 Conditional Enforcement of PIR Query Validity . . . 48
5.4 Simple Closure Properties . . . 48
5.4.1 Closure under AND . . . 48
5.4.2 Closure under OR . . . 49
5.5 Formula-Based CDS . . . 49
5.6 CDS for General Predicates . . . 50
5.6.1 Completeness of the Index Function . . . 50
5.6.2 Linear CDS for Index . . . 50
5.6.3 Balancing . . . 51
5.7 Subexponential CDS . . . 51
5.7.1 Construction from a Linear PIR Identity . . . 51
5.8 Amortized CDS for General Functions . . . 52

6 Private Simultaneous Messages and Randomized Encodings 53

6.1 Private Simultaneous Message Protocols . . . 53
6.1.1 Example: XOR . . . 53
6.2 PSM versus CDS . . . 54
6.3 PSM for Every Function [FKN94] . . . 54
6.3.1 Three-Party PSM from a Three-Dimensional Truth Table . . . 55
6.4 An AND PSM from Basic Principles . . . 56
6.5 Randomized Encoding of Functions . . . 56
6.5.1 Useful Properties . . . 57
6.5.2 Concatenation with Independent Randomness . . . 57
6.6 PSM Complexity Bounds . . . 58
6.7 Yao's Garbled-Circuit Construction . . . 58

6.8 Encoding a Function Family . . . 59
6.9 Perfect DARE for Formulas . . . 59
6.9.1 DARE for Simple Functions . . . 59
6.9.2 Encoding the Circuit Layer by Layer . . . 59
6.10 Computational DARE for General Circuits . . . 60
6.10.1 Key-Shrinking Gadget . . . 60

V Secure Computation 61

7 Secure Two-Party Computation over an OT Channel 62

7.1 Two-Party Interactive Protocols . . . 62
7.1.1 Example: XOR . . . 62
7.2 The Impossibility of AND in the Plain Model . . . 63
7.3 The OT-Hybrid Model . . . 64
7.3.1 Computing AND with OT . . . 65
7.4 Inner Product via OT . . . 66
7.4.1 Every Two-Party Function Reduces to Inner Product . . . 66
7.5 The Two-Party GMW Protocol . . . 67
7.5.1 Input Gates . . . 67
7.5.2 Addition and Subtraction Gates . . . 68
7.5.3 Multiplication Gates . . . 68
7.5.4 Output Phase . . . 68
7.6 From Decomposable Randomized Encodings to Protocols . . . 69
7.7 Constructing OT from Encryption with a Fake Mode . . . 70
7.7.1 Efficiency for String OT . . . 70
7.8 Precomputing OT and GMW . . . 71

8 Passive Multi-Party Computation in the Plain and OT-Hybrid Models 72

8.1 The Multi-Party Protocol Model . . . 72
8.1.1 The REAL/IDEAL Paradigm . . . 72
8.2 Privately Computing the Sum . . . 72
8.2.1 The Complete-Graph Protocol . . . 73
8.2.2 The Sum Protocol on an Arbitrary Graph . . . 74
8.2.3 Rushing Adversaries . . . 75
8.3 Completeness of Functionalities . . . 76
8.4 The Multi-Party GMW Protocol in the OT-Hybrid Model . . . 77
8.4.1 Explicit GMW Protocol for $f(w, x, y, z) = wy + xz$ . . . 77
8.5 The Multi-Party BGW Protocol in the Plain Model . . . 79
8.5.1 Input and Linear Gates . . . 79
8.5.2 Multiplication and Degree Reduction . . . 79
8.5.3 Hybrid View and Composition . . . 80

9 Active Multi-Party Computation and Maurer's CNF Construction 81

9.1 Active Security . . . 81
9.1.1 Simple Examples . . . 81
9.2 Information-Theoretic Active MPC in the Plain Model . . . 82
9.2.1 Additional Requirements for Active Security . . . 82
9.3 The $(4, 1)$-CNF Secret-Sharing Scheme . . . 82
9.3.1 DNF and CNF Share-Size Optimization . . . 83
9.4 Robust Secret Sharing and Reconstruction . . . 84
9.5 Verifiable Secret Sharing for $(4, 1)$-CNF . . . 85
9.6 Active $(4, 1)$-MPC . . . 86
9.6.1 Secure Multiplication . . . 86
9.6.2 Subprotocol for Sharing a Monomial . . . 86

VI Proof Systems **88**

10 Probabilistic Proof Systems **89**

10.1 Classical Proof Systems . . . 89
10.2 NP Proofs . . . 89
10.3 Interactive Proofs . . . 89
10.4 Zero Knowledge . . . 90
10.5 Graph Isomorphism Examples . . . 90
10.5.1 An Interactive Proof beyond NP . . . 90
10.5.2 A Zero-Knowledge Interactive Proof . . . 90
10.6 The Class IP . . . 91
10.7 Two-Prover Interactive Proofs and PCPs . . . 91
10.8 Doubly Efficient Interactive Proofs . . . 92
10.9 Interactive Proofs and Cryptography . . . 92
10.9.1 Succinct Arguments for NP . . . 92
10.9.2 Zero Knowledge for NP . . . 93

11 IP = PSPACE and the Sum-Check Protocol **94**

11.1 Flawed Interactive Proofs for #SAT . . . 94
11.2 Arithmetization . . . 94
11.3 The Sum-Check Protocol . . . 95
11.3.1 Completeness . . . 96
11.4 The Schwartz–Zippel Lemma . . . 96
11.5 Soundness of Sum Check . . . 98
11.6 Proof of IP = PSPACE . . . 98
11.7 IP $\subseteq$ PSPACE . . . 99
11.8 TQBF Is PSPACE-Complete . . . 99
11.9 Interactive Proof for TQBF . . . 100

11.9.1 The Degree Problem . . . 100
11.9.2 Eliminating Degree-Reduction Operators . . . 101
11.9.3 Efficiency . . . 101

12 Probabilistically Checkable Proofs and Succinct Arguments 102

12.1 Probabilistically Checkable Proofs . . . 102
12.2 From Interactive Proofs to PCPs . . . 102
12.3 PCP Existence Theorems . . . 103
12.4 3CNF Formulas and Cook–Levin . . . 104
12.5 A Succinct Representation of ϕ_M . . . 104
12.6 Low-Degree Extensions . . . 105
12.7 Arithmetizing Satisfiability . . . 105
12.8 The PCP Prover and Verifier . . . 106
12.9 Zero Sub-Cube Proofs . . . 107
12.9.1 The Prover . . . 107
12.9.2 The Verifier . . . 107
12.10 Low-Degree Tests . . . 108
12.11 Local Correction . . . 109
12.12 Succinct Arguments . . . 109
12.12.1 Compiling a PCP . . . 110
12.12.2 Merkle Hash Trees . . . 110
12.12.3 Kilian's Succinct Argument . . . 110

13 Zero Knowledge, Composition, and the Fiat–Shamir Heuristic 112

13.1 Freivalds' Test for Matrix Multiplication . . . 112
13.2 Zero-Error Interactive Proofs . . . 113
13.3 Sequential Repetition . . . 114
13.4 Quadratic Non-Residuosity . . . 115

13.4.1 Interactive Protocol 116
13.5 Zero-Knowledge Proofs 117
13.6 Commitments 118
13.7 The GMW Protocol for Graph 3-Coloring 118
13.8 Succinctness and Zero Knowledge by Composition 119
13.8.1 Soundness and Proofs of Knowledge 119
13.9 Removing Interaction 120
13.10 The Fiat–Shamir Heuristic 120

PART I

Introduction

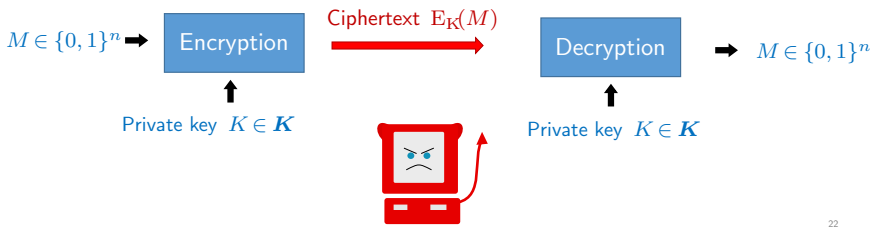
1 Introduction to Information-Theoretic Cryptography

1.1 Perfect Encryption [Shannon 48]

Definition 1.1 [Encryption scheme]. An encryption scheme is a pair of algorithms (E, D) with message space $\mathcal{M}$, key space $\mathcal{K}$, and ciphertext space $\mathcal{C}$. For every $K \in \mathcal{K}$ and $M \in \mathcal{M}$, encryption outputs $E_K(M) \in \mathcal{C}$, and correctness requires

$$D_K(E_K(M)) = M.$$

Secrecy: For every $X, Y \in \{0,1\}^n$ where $K \in_R \boldsymbol{K}$ $\quad \mathrm{E_K}(X) \equiv \mathrm{E_K}(Y)$



1.1.1 Perfect Secrecy

Definition 1.2 [Perfect secrecy]. For every $X, Y \in \{0,1\}^n$,

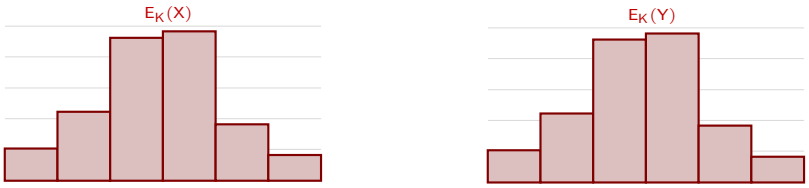
$$E_K(X) \equiv E_K(Y), \qquad K \xleftarrow{R} \mathcal{K}.$$

Equivalently, for every $C \in \mathcal{C}$,

$$\Pr_K[E_K(X) = C] = \Pr_K[E_K(Y) = C].$$

Secrecy: For every $X, Y \in \{0,1\}^n$ where $K \in_R \boldsymbol{K}$ $\quad \mathrm{E_K}(X) \equiv \mathrm{E_K}(Y)$

$$\forall\ C, \Pr_K[E_K(X) = C] = \Pr_K[E_K(Y) = C]$$



1.1.2 Statistical Secrecy

Definition 1.3 [Statistical distance]. For random variables U, V over a common finite domain, their statistical distance is

$$\Delta(U,V) = \max_S |\Pr[U \in S] - \Pr[V \in S]| .$$

Equivalently,

$$\Delta(U,V) = \max_A |\Pr[A(U) = 1] - \Pr[A(V) = 1]| ,$$

where A ranges over all computationally unbounded distinguishers.

Definition 1.4 [Statistical secrecy]. For every $X, Y \in \{0,1\}^n$,

$$\Delta(E_K(X), E_K(Y)) \le \delta, \qquad K \xleftarrow{R} \mathcal{K}.$$

That is, for every set of ciphertexts S,

$$\left| \Pr_K[E_K(X) \in S] - \Pr_K[E_K(Y) \in S] \right| \le \delta,$$

or, equivalently, for every unbounded A,

$$\left| \Pr_K[A(E_K(X)) = 1] - \Pr_K[A(E_K(Y)) = 1] \right| \le \delta.$$

Secrecy: For every $X, Y \in \{0,1\}^n$ where $K \in_R \boldsymbol{K}$

$\mathrm{E_K}(X) \approx \mathrm{E_K}(Y)$

$\forall$ set of ciphertexts $S, \left| \Pr_K[E_K(X) \in S] - \Pr_K[E_K(Y) \in S] \right| \le \delta$



1.1.3 Computational Secrecy [GM'82]

Definition 1.5 [(t,δ)-computational secrecy]. An encryption scheme has (t,δ)-computational secrecy if, for every $X, Y \in \{0,1\}^n$ and every probabilistic distinguisher A of running time at most t,

$$\left| \Pr_K[A(E_K(X)) = 1] - \Pr_K[A(E_K(Y)) = 1] \right| \le \delta, \qquad K \xleftarrow{R} \mathcal{K}.$$

Secrecy: For every $X, Y \in \{0,1\}^n$ where $K \in_R \boldsymbol{K}$

$$\mathrm{E}_K(X) \approx \mathrm{E}_K(Y)$$

$$\forall \text{ "efficient" } Adv, \left| \Pr_K\left[Adv\left(E_K(X)\right) = 1\right] - \Pr_K\left[Adv\left(E_K(Y)\right) = 1\right] \right| \leq \delta$$



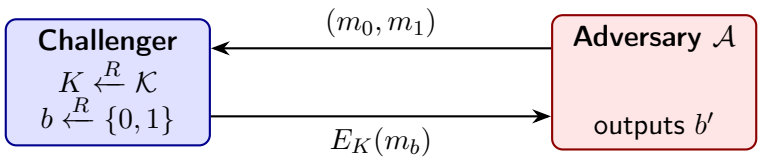
1.1.4 Perfect Indistinguishability and the Turing Game

Definition 1.6 [Perfect indistinguishability]. The encryption scheme (E, D) is perfectly indistinguishable (or perfectly secure) if for every $m \neq m' \in \mathcal{M}$ and every $c \in \mathcal{C}$,

$$\Pr_{K \xleftarrow{R} \mathcal{K}}[E_K(m) = c] = \Pr_{K \xleftarrow{R} \mathcal{K}}[E_K(m') = c].$$

Protocol (Perfect-indistinguishability experiment)

1. *Message selection.* The adversary $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}$.
2. *Challenge.* Sample $K \xleftarrow{R} \mathcal{K}$ and $b \xleftarrow{R} \{0,1\}$, and give $c = E_K(m_b)$ to $\mathcal{A}$.
3. *Guess.* The adversary outputs $b' \in \{0,1\}$.
4. *Output.* The experiment outputs 1 if $b' = b$ and 0 otherwise.



$\mathcal{A}$ wins exactly when $b' = b$.

Figure 1: The perfect-indistinguishability experiment.

Definition 1.7 [Adversarial indistinguishability]. The cipher is adversarial indistinguishable if, for every adversary $\mathcal{A}$,

$$\Pr[b = b'] = \frac{1}{2}.$$

Theorem 1.8 [Equivalence of the two definitions]. A cipher is adversarial indistinguishable if and

only if it is perfectly indistinguishable. Namely,

$$\forall m_0, m_1 \in \mathcal{M},\ \forall c \in \mathcal{C}: \quad \Pr_K[E_K(m_0) = c] = \Pr_K[E_K(m_1) = c]$$
$$\iff \quad \Pr[b = b'] = \frac{1}{2} \quad \text{for every adversary } \mathcal{A}.$$

Proof

Suppose first that the cipher is perfectly indistinguishable. For the two messages selected by $\mathcal{A}$, the distribution of $E_K(m_0)$ is identical to the distribution of $E_K(m_1)$. Hence the challenge ciphertext is independent of the uniform bit b, and every adversary satisfies $\Pr[b = b'] = 1/2$.
Conversely, suppose perfect indistinguishability fails. Then there exist $m_0, m_1 \in \mathcal{M}$ and $c \in \mathcal{C}$ such that

$$p_0 := \Pr_K[E_K(m_0) = c] \neq p_1 := \Pr_K[E_K(m_1) = c].$$

After exchanging the messages if needed, assume $p_0 > p_1$. Let $\mathcal{A}$ output $b' = 0$ when the challenge ciphertext is c, and output $b' = 1$ otherwise. Then

$$\Pr[b = b'] = \frac{1}{2}p_0 + \frac{1}{2}(1 - p_1) = \frac{1}{2} + \frac{p_0 - p_1}{2} > \frac{1}{2}.$$

Thus the cipher is not adversarial indistinguishable. □

1.2 One-Time Pad Is Perfectly Secure

For $M \in \{0,1\}^n$, sample $K \xleftarrow{R} \{0,1\}^n$ and define

$$E_K(M) = K \oplus M, \qquad D_K(C) = C \oplus K.$$

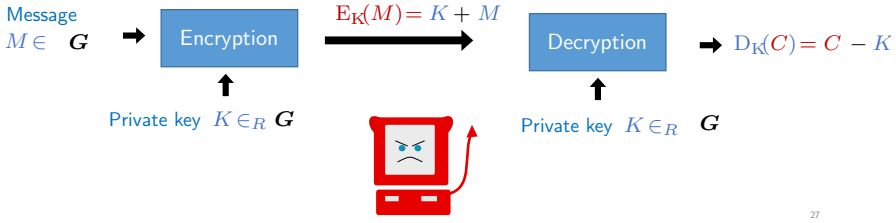
Theorem 1.9 [One-time pad security]. For every X, Y, $E_K(X) \equiv E_K(Y)$. Thus the one-time pad is perfectly secure.

Proof

More generally, let G be a finite group and let K be uniform in G. For every $X, C \in G$,

$$\Pr_K[E_K(X) = C] = \Pr_K[K + X = C] = \Pr_K[K = C - X] = \frac{1}{|G|}.$$

Put differently: for every X, the mapping $K \mapsto E_K(X)$ is a bijection from randomness space to ciphertext space. In fact, it is a non-degenerate linear mapping. □



1.3 Efficiency Measures and Shannon's Lower Bound

The efficiency measures are communication, randomness/key length, and round complexity.

	Communication	**Randomness/key length**	**Round complexity**
OTP of n-bit message	n bits	n bits	1
Lower-bound (any perfect scheme)	n bits	n bits	1

Theorem 1.10 [Shannon's lower bound]. For n-bit messages, any perfectly secure encryption scheme requires at least n bits of key length and at least n bits of communication.

Proof

Fix a ciphertext c that can occur. Perfect secrecy implies that c can occur for every message. For each fixed key, correctness permits at most one message to decrypt from c. Therefore $|\mathcal{K}| \geq |\mathcal{M}| = 2^n$, so the key length is at least n bits.
For every fixed key K, correctness makes $m \mapsto E_K(m)$ injective. Thus $|\mathcal{C}| \geq |\mathcal{M}| = 2^n$, so the ciphertext communication is at least n bits. □

By Theorem 1.9 and Theorem 1.10, the one-time pad is optimal in communication, randomness/key length, and round complexity.

1.4 Broadcast Encryption [Fiat–Naor 94]

Definition 1.11 [Broadcast encryption]. A broadcast-encryption scheme for N recipients consists of a setup algorithm producing keys $(K_1, \ldots, K_N)$, an encryption map

$$\mathrm{Enc} : \mathcal{M} \times 2^{[N]} \times \mathcal{K}^N \longrightarrow \mathcal{C},$$

and decryption maps Dec_i. Correctness requires that for every $M \in \mathcal{M}$, every $S \subseteq [N]$, and every $i \in S$,

$$\mathrm{Dec}_i(K_i, \mathrm{Enc}(M, S, K_1, \ldots, K_N)) = M.$$

Privacy requires that every coalition contained in $[N] \setminus S$ obtain no information about M from the ciphertext and its keys.



1.5 Error Correction and Detection

[Hamming 47, Shannon 48]

Definition 1.12 [Error-correcting encoding]. Let Σ be an alphabet. An error-correcting encoding is a pair of maps

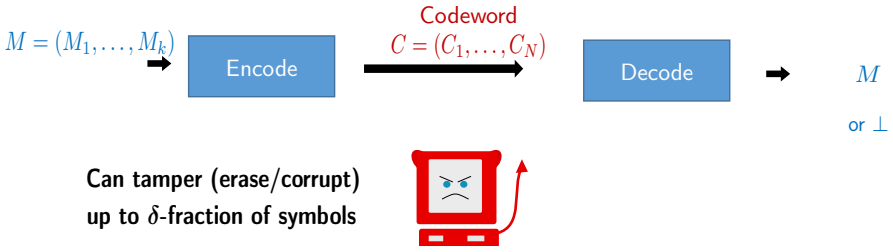
$$\mathrm{Enc} : \Sigma^k \longrightarrow \Sigma^N, \qquad \mathrm{Dec} : (\Sigma \cup \{\perp\})^N \longrightarrow \Sigma^k.$$

For every message $M = (M_1, \ldots, M_k) \in \Sigma^k$, let $C = \mathrm{Enc}(M)$. Correctness against a δ-fraction of corruptions or erasures requires that, for every $Y \in (\Sigma \cup \{\perp\})^N$ satisfying

$$|\{i \in [N] : Y_i \neq C_i\}| \leq \delta N,$$

the decoder recovers

$$\mathrm{Dec}(Y) = M.$$



1.6 Distributed Storage Model

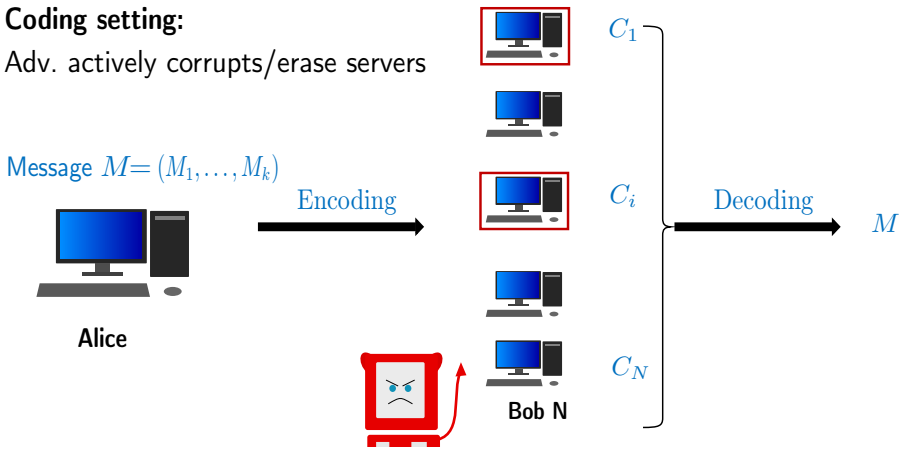
Definition 1.13 [Distributed encoding]. A distributed encoding consists of maps

$$\mathrm{Enc} : \mathcal{M} \times \mathcal{R} \longrightarrow \mathcal{C}_1 \times \cdots \times \mathcal{C}_N$$

and a decoding map defined on an admissible family of received share vectors. For $(C_1, \ldots, C_N) = \mathrm{Enc}(M; r)$, component C_i is stored by server i.

Coding Setting

The adversary may replace or erase stored components. Correctness requires recovery of M for every admissible corruption pattern.



Secrecy Setting

The adversary reads a prescribed collection of stored components. Privacy requires the resulting view to be independent of M.

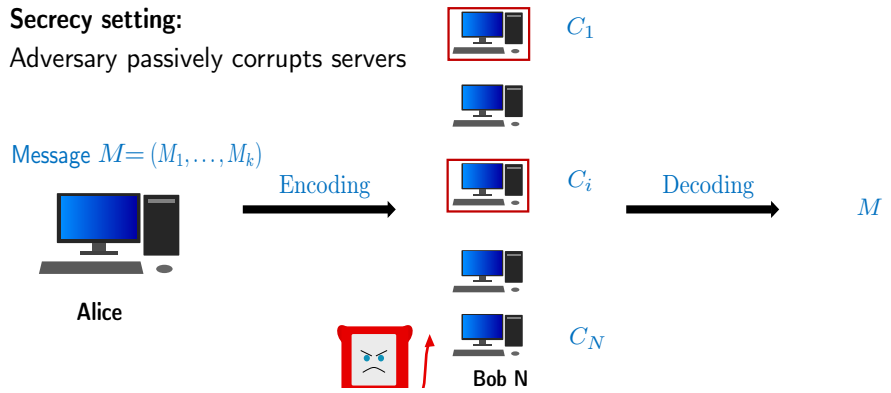
For the one-time pad, encode

$$E_K(M) = K + M$$

using K and $M + K$.

For N servers, sample independent uniform $K_1, \ldots, K_{N-1}$ and store

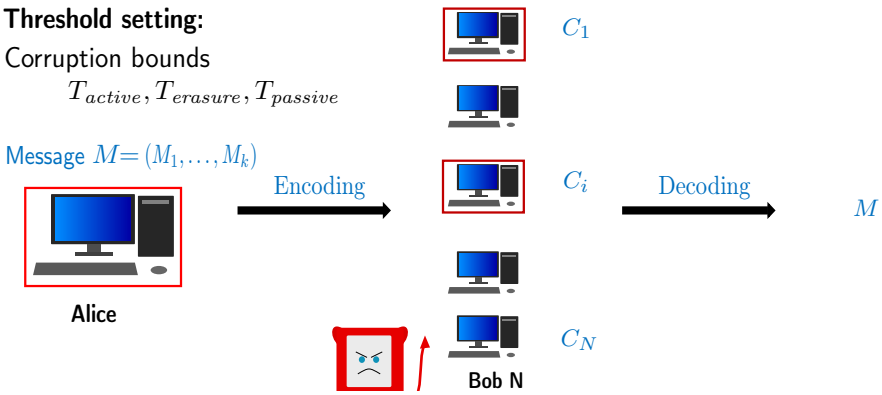
$$K_1, \ldots, K_{N-1}, \qquad M + K_1 + \cdots + K_{N-1}.$$



Secret-Sharing

A secret-sharing or robust-storage scheme specifies admissible bounds

$$T_{\text{active}}, \qquad T_{\text{erasure}}, \qquad T_{\text{passive}}.$$


Threshold setting:
Corruption bounds
$T_{active}, T_{erasure}, T_{passive}$
Message $M = (M_1, \ldots, M_k)$
Encoding
Alice
C_1
C_i
C_N
Bob N
Decoding
M

PART II

Secret Sharing

2 Threshold Secret Sharing

2.1 t-out-of-n Secret Sharing: Syntax

Definition 2.1 [Threshold secret sharing]. Let $\mathcal{S}$ be a finite secret space, let $\mathcal{R}$ be a finite randomness space, and let $\mathcal{S}_1, \ldots, \mathcal{S}_n$ be share spaces. A t-out-of-n secret-sharing scheme consists of a randomized dealing map

$$\mathcal{E} : \mathcal{S} \times \mathcal{R} \longrightarrow \mathcal{S}_1 \times \cdots \times \mathcal{S}_n$$

and, for every $T \subseteq [n]$ with $|T| = t$, a deterministic reconstruction map

$$\mathcal{D}_T : \prod_{i \in T} \mathcal{S}_i \longrightarrow \mathcal{S}.$$

2.1.1 Dealing Function

Definition 2.2 [Share notation]. For $(s_1, \ldots, s_n) = \mathcal{E}(s; r)$, write

$$s_T = (s_i)_{i \in T}.$$

Definition 2.3 [Perfect privacy]. The distribution of any $t-1$ shares $s_{i_1}, \ldots, s_{i_{t-1}}$ is independent of the secret s. Formally, for every pair $s, s' \in \mathcal{S}$ and every coalition $T \subseteq [n]$ with $|T| < t$,

$$\big(\mathcal{E}_i(s; r)\big)_{i \in T} \equiv \big(\mathcal{E}_i(s'; r)\big)_{i \in T}.$$

2.1.2 Reconstruction Function

Definition 2.4 [Reconstruction and correctness]. For every coalition $T = \{i_1, \ldots, i_t\} \subseteq [n]$ of t parties, the reconstruction function is

$$\mathcal{D}_T : (s_{i_1}, \ldots, s_{i_t}) \longmapsto s.$$

Correctness requires that for every $s \in \mathcal{S}$ and $r \in \mathcal{R}$,

$$\mathcal{D}_T\big((\mathcal{E}_i(s; r))_{i \in T}\big) = s.$$

2.2 Linear Secret Sharing and Hiding Properties of Linear Maps

Let $\mathbb{F}$ be a finite field, let $A \in \mathbb{F}^{n \times k}$ be a public matrix, and let $v \in \mathbb{F}^n$ be a public vector. For a scalar message $s \in \mathbb{F}$ and a random column vector $r \xleftarrow{R} \mathbb{F}^k$, define

$$E_r(s) = A \cdot r + s \cdot v.$$

Lemma 2.5 [Linear separation]. If $v \notin \mathrm{Col}(A)$, then there exists $w \in \mathbb{F}^n$ such that

$$w^T A = 0 \qquad \text{and} \qquad w^T v \neq 0.$$

Proof

Since v does not belong to the subspace $\mathrm{Col}(A)$, there is a linear functional that vanishes on $\mathrm{Col}(A)$ but does not vanish on v. Writing this functional as $x \mapsto w^T x$ gives the required vector w. □

Claim 2.6 [Recoverability]. If $v \notin \mathrm{Col}(A)$, then s is recoverable from $E_r(s)$ by a linear decoder. Let

$$u = \frac{w}{w^T v}.$$

Then, for a ciphertext c,

$$u^T c = \frac{w^T c}{w^T v},$$

where $w^T A = 0$ and $w^T v \neq 0$.

Proof

For $c = E_r(s) = Ar + sv$,

$$u^T c = \frac{w^T(Ar + sv)}{w^T v} = \frac{w^T Ar + s(w^T v)}{w^T v} = s.$$

The map $c \mapsto u^T c$ is linear over $\mathbb{F}$. □

Claim 2.7 [Perfect hiding]. If $v \in \mathrm{Col}(A)$, then $E_r(s)$ perfectly hides s. Formally, for every $s, s' \in \mathbb{F}$ and uniform $r \xleftarrow{R} \mathbb{F}^k$, the random variables $E_r(s)$ and $E_r(s')$ are identically distributed.

Proof

Since $v \in \mathrm{Col}(A)$, there exists $z \in \mathbb{F}^k$ such that $v = Az$. Therefore

$$E_r(s) = Ar + sv = A(r + sz).$$

For every fixed s, the map $r \mapsto r + sz$ is a bijection on $\mathbb{F}^k$. Thus $r + sz$ is uniform whenever r is uniform, so the distribution of $E_r(s)$ is independent of s. □

Corollary 2.8. The one-time pad proof is an instance of the hiding property of a non-degenerate linear mapping.

2.3 n-out-of-n Secret Sharing over a Finite Group

Let $(\mathcal{S}, +)$ be a finite abelian group.

Protocol (n-out-of-n additive sharing)

1. *Share.* On input $s \in \mathcal{S}$, sample $r_1, \ldots, r_{n-1} \xleftarrow{R} \mathcal{S}$ independently and define

$$s_i = \begin{cases} r_i, & 1 \leq i \leq n-1, \\ -\left(\sum_{i=1}^{n-1} r_i\right) + s, & i = n. \end{cases}$$

2. *Reconstruct.* On input $(s_1, \ldots, s_n)$, output $s_1 + \cdots + s_n$.

Theorem 2.9 [Correctness and perfect security over a generic group]. The additive protocol is an n-out-of-n secret-sharing scheme. Moreover, for every $T \in \binom{[n]}{n-1}$ and every secret $s \in \mathcal{S}$, the share vector $(s_i)_{i \in T}$ is distributed according to $\mathcal{U}(\mathcal{S}^{n-1})$.

Proof

For correctness,

$$s_1 + \cdots + s_n = \left(\sum_{i=1}^{n-1} r_i\right) - \left(\sum_{i=1}^{n-1} r_i\right) + s = s.$$

For perfect security, fix a secret s and a coalition $T \in \binom{[n]}{n-1}$.
If $n \notin T$, then $T = \{1, \ldots, n-1\}$ and

$$(s_i)_{i \in T} = (r_1, \ldots, r_{n-1})$$

is uniform over $\mathcal{S}^{n-1}$.
Otherwise, there is a unique $j \in \{1, \ldots, n-1\}$ such that $j \notin T$. Fix arbitrary target values $(a_i)_{i \in T} \in \mathcal{S}^{n-1}$. For all $i \in T \setminus \{n\}$, the equality $s_i = a_i$ uniquely fixes $r_i = a_i$. The equality $s_n = a_n$ then uniquely fixes r_j, since the map

$$r_j \longmapsto -\left(\sum_{i=1}^{n-1} r_i\right) + s$$

is a bijection of $\mathcal{S}$ when all other r_i are fixed. Hence exactly one randomness vector $(r_1, \ldots, r_{n-1})$ produces the target vector $(a_i)_{i \in T}$. Since the randomness vector is uniform over $\mathcal{S}^{n-1}$,

$$\Pr[(s_i)_{i \in T} = (a_i)_{i \in T}] = \frac{1}{|\mathcal{S}|^{n-1}}.$$

Thus $(s_i)_{i \in T} \sim \mathcal{U}(\mathcal{S}^{n-1})$, independently of s. □

2.4 The Case $t < n$

The following protocol realizes the threshold condition by evaluating a random low-degree polynomial over a finite field.

2.5 $\mathbb{Z}_p$ as a Field

Definition 2.10 [$\mathbb{Z}_p$].

$$\mathbb{Z}_p = \{0, \ldots, p-1\}$$

is a field when p is a prime. Namely, $(\mathbb{Z}_p, +_p)$ is a commutative additive group, and

$$(\mathbb{Z}_p \setminus \{0\} = \mathbb{Z}_p^*, \cdot_p)$$

is a commutative multiplicative group.

Theorem 2.11 [Root bound]. Let $P(x)$ be a non-zero polynomial of degree d in one variable x over a field F. Namely,

$$P(x) = a_d x^d + a_{d-1} x^{d-1} + a_{d-2} x^{d-2} + \cdots + a_1 x + a_0,$$

where $a_d, a_{d-1}, \ldots, a_1, a_0 \in F$. Then the equation $P(x) = 0$ has at most d solutions in F. Such a solution is called a root of $P(x)$.

2.6 Shamir's Secret Sharing

2.6.1 The Case $t = 2$

Assume $n < p$ and $\mathcal{S} = \mathbb{Z}_p$. For a secret $s \in \mathbb{Z}_p$, sample $a \xleftarrow{R} \mathbb{Z}_p$ and define

$$f[x] = ax + s$$

and shares $s_i = f(i)$.

Lemma 2.12 [The case $t = 2$]. Every individual share is uniform over $\mathbb{Z}_p$ and independent of s, while every pair of distinct shares uniquely determines s.

Proof

For fixed $i \neq 0$, the map $a \mapsto ai + s$ is a bijection of $\mathbb{Z}_p$, so $f(i)$ is uniform. For $i \neq j$, the equations

$$f(i) = ai + s, \qquad f(j) = aj + s$$

have a unique solution because $i - j \neq 0$ in $\mathbb{Z}_p$. □

2.6.2 General t

Definition 2.13 [Shamir's t-out-of-n scheme]. Assume $n < p$. On input $s \in \mathbb{Z}_p$, sample $r_1, \ldots, r_{t-1} \xleftarrow{R} \mathbb{Z}_p$ and define

$$f[x] = r_{t-1} x^{t-1} + \cdots + r_1 x + s,$$

The share of participant $i \in [n]$ is $s_i = f(i)$.

Lemma 2.14 [Reconstruction]. Every set of t participants can recover the secret.

Lemma 2.15 [Privacy]. The scheme is secure: the view of $t - 1$ parties is uniformly distributed over $\mathbb{Z}_p^{t-1}$.

2.7 Recovering the Secret via Lagrange Interpolation

For the scheme of Definition 2.13, t parties can recover all coefficients of the polynomial.

2.7.1 Polynomial Interpolation

Let $(x_1, y_1), \ldots, (x_t, y_t)$ be pairs over a field with distinct $x_1, \ldots, x_t$. There is a unique polynomial g of degree at most $t-1$ satisfying $g(x_i) = y_i$ for every i.

Let

$$f_1[x] = y_1 \cdot \frac{x - x_2}{x_1 - x_2} \cdot \frac{x - x_3}{x_1 - x_3} \cdots \frac{x - x_t}{x_1 - x_t}.$$

The polynomial $f_1[x]$ has degree $t-1$, satisfies $f_1(x_1) = y_1$, and satisfies $f_1(x_i) = 0$ for every $i \neq 1$.

Definition 2.16 [Lagrange interpolation polynomials]. For every $i \in \{1, \ldots, t\}$, define

$$f_i[x] = y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{x - x_j}{x_i - x_j}.$$

Then

$$f[x] = f_1[x] + f_2[x] + \cdots + f_t[x] = \sum_{i=1}^{t} y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{x - x_j}{x_i - x_j}.$$

The polynomial f has degree at most $t-1$ and is consistent with all t pairs. If g is another such polynomial, then $f - g$ has degree at most $t-1$ and has the t distinct roots $x_1, \ldots, x_t$. The root bound implies $f - g = 0$, proving uniqueness.

For a coalition T whose shares are $(i, f(i))_{i \in T}$, the reconstruction function recovers

$$\mathcal{D}_T\big((f(i))_{i \in T}\big) = f(0) = \sum_{i \in T} f(i) \prod_{\substack{j \in T \\ j \neq i}} \frac{0 - j}{i - j} = s.$$

2.8 Security of Shamir's Scheme

For the scheme of Definition 2.13, let

$$f[x] = r_{t-1} x^{t-1} + \cdots + r_1 x + s, \qquad r_{t-1}, \ldots, r_1 \xleftarrow{R} \mathbb{Z}_p,$$

be the sharing polynomial.

Theorem 2.17 [Every set of $t-1$ shares is uniformly distributed]. Fix a coalition T of size $t-1$, a secret a, and a vector $\vec{c} \in \mathbb{Z}_p^{t-1}$. Then

$$\Pr[s_T = \vec{c}] = \frac{1}{p^{t-1}}, \qquad s_T = (s_i)_{i \in T}.$$

Proof

It suffices to show that there is exactly one choice of $r = (r_1, \ldots, r_{t-1})$ for which $s_T = \vec{c}$. The $t-1$ pairs $(i, f(i))_{i \in T}$, together with the pair $(0, a)$, uniquely determine a degree-$t-1$ polynomial

$$f[x] = r_{t-1}x^{t-1} + \cdots + r_1 x + a.$$

Therefore exactly one of the p^{t-1} equally likely randomness vectors produces $\vec{c}$, and the claimed probability is $1/p^{t-1}$. □

2.9 Lower Bound on Share Size

Theorem 2.18 [HW 1, Exercise 3]. Let $t < n$ be positive integers and let $\mathcal{S}$ be a finite secret space of size at least 2. Consider an arbitrary t-out-of-n secret-sharing scheme $(\mathcal{E}, \mathcal{D})$ where

$$\mathcal{E} : \mathcal{S} \times \mathcal{R} \longrightarrow \mathcal{S}_1 \times \cdots \times \mathcal{S}_n.$$

Then, for every $i \in \{1, \ldots, n\}$,

$$\log |\mathcal{S}_i| \geq \log |\mathcal{S}|.$$

That is, the i-th share size is at least as large as the length of the secret. Logarithms are taken to base 2.

Proof

Fix $i \in [n]$. Since $t < n$, choose a set $T \subseteq [n] \setminus \{i\}$ of size $t-1$. Choose a secret $s_0 \in \mathcal{S}$ and a value $a_T \in \prod_{j \in T} \mathcal{S}_j$ that occurs with positive probability as the shares of T when the secret is s_0. Perfect privacy implies that the distribution of the shares of T is identical for every secret. Hence, for every $s \in \mathcal{S}$, there is some randomness $r_s \in \mathcal{R}$ such that

$$\big(\mathcal{E}_j(s; r_s)\big)_{j \in T} = a_T.$$

Define

$$\phi(s) = \mathcal{E}_i(s; r_s) \in \mathcal{S}_i.$$

We claim that ϕ is injective. If $s \neq s'$ but $\phi(s) = \phi(s')$, then the authorized coalition $T \cup \{i\}$ receives exactly the same tuple of t shares in the two executions. Correctness would therefore require the deterministic reconstruction function $\mathcal{D}_{T \cup \{i\}}$ to output both s and s', a contradiction. Thus ϕ is injective, so

$$|\mathcal{S}_i| \geq |\mathcal{S}|.$$

Taking base-2 logarithms proves the theorem. □

2.10 Field Size and Efficiency

Shamir sharing is defined over any finite field $\mathbb{F}$ containing at least $n+1$ distinct elements: one evaluation point for the secret and n nonzero evaluation points for the shares. Lagrange interpolation applies because every nonzero field element is invertible.

Definition 2.19 [Ideal secret-sharing scheme]. A secret-sharing scheme is ideal if every share space has the same cardinality as the secret space.

If the field contains the $n+1$ required evaluation points, Shamir's scheme is ideal. Multiple secrets may be packed into distinct prescribed evaluations of one polynomial, subject to the resulting degree and reconstruction threshold.

2.10.1 Other Access Structures

Definition 2.20 [Access structure]. For a secret-sharing scheme on participant set $[n]$, its access structure is the monotone family

$$\Gamma = \{T \subseteq [n] : T \text{ can reconstruct the secret}\}.$$

A threshold scheme has $\Gamma = \{T \subseteq [n] : |T| \geq t\}$.

2.11 Threshold-Based Key Recovery

Let k be a cryptographic key, let $(s_1, \ldots, s_5)$ be a 3-out-of-5 sharing of k, and let $a_1, \ldots, a_5$ be answers to recovery questions.

Protocol (Question-based key recovery)

1. *Enrollment.* Store
$$c_i = H(a_i) \oplus s_i, \qquad i \in [5].$$
2. *Share recovery.* Given a candidate answer a'_i, compute
$$s'_i = c_i \oplus H(a'_i).$$
If $a'_i = a_i$, then $s'_i = s_i$.
3. *Key reconstruction.* Apply the reconstruction map of the 3-out-of-5 scheme to any three correctly recovered shares.

PART III

Information Retrieval

3 Private Information Retrieval (IT-PIR)

3.1 Problem Formulation

Let $x = (x[1], \dots, x[N]) \in \{0,1\}^N$ be replicated among k non-communicating servers. A private information retrieval (PIR) protocol allows a user holding an index $i \in [N]$ to recover $x[i]$. Information-theoretic 1-privacy requires the marginal distribution of the query received by each server to be independent of i.

The communication complexity is the sum of the lengths of all queries and answers. The full-download protocol communicates N bits. A PIR protocol has sublinear communication if its communication complexity is $o(N)$.

[Chor–Goldreich–Kushilevitz–Sudan 95]

3.2 Definition of 1-Private k-Server PIR

Definition 3.1 [1-private k-server PIR]. A 1-private k-server PIR protocol for N-bit databases consists of a finite randomness space $\mathcal{R}$, query spaces $\mathcal{Q}_1, \dots, \mathcal{Q}_k$, answer spaces $\mathcal{A}_1, \dots, \mathcal{A}_k$, and maps

$$\begin{aligned} Q &: [N] \times \mathcal{R} \longrightarrow \mathcal{Q}_1 \times \cdots \times \mathcal{Q}_k, \\ A_s &: \{0,1\}^N \times \mathcal{Q}_s \longrightarrow \mathcal{A}_s, \qquad s \in [k], \\ D &: [N] \times \mathcal{R} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_k \longrightarrow \{0,1\}. \end{aligned}$$

For $i \in [N]$, $r \in \mathcal{R}$, and $x \in \{0,1\}^N$, write

$$(q_1, \dots, q_k) = Q(i, r), \qquad a_s = A_s(x, q_s).$$

The protocol satisfies the following conditions.

1. *Perfect correctness:* for every $x \in \{0,1\}^N$, $i \in [N]$, and $r \in \mathcal{R}$,
$$D(i, r, a_1, \dots, a_k) = x[i].$$
2. *Perfect 1-privacy:* for every $s \in [k]$ and every $i, i' \in [N]$,
$$Q_s(i, R) \equiv Q_s(i', R), \qquad R \xleftarrow{R} \mathcal{R},$$
where Q_s is the s-th component of Q.

The communication complexity of an execution is

$$\sum_{s=1}^{k} \big(|q_s| + |a_s|\big).$$

Unless stated otherwise, each answer has length $O(1)$ bits.

3.2.1 Impossibility of Non-Trivial Single-Server PIR

Theorem 3.2 [HW 2, Exercise 1]. In every perfectly correct, perfectly private single-server PIR protocol for N-bit databases, the answer length is at least N bits.

Proof

By perfect privacy, the random variables $Q(i, R)$ have the same distribution for all $i \in [N]$. Fix a query q in their common support. For every $i \in [N]$, choose $r_i \in \mathcal{R}$ such that $Q(i, r_i) = q$. Suppose that the answer length is t, and define

$$F_q : \{0,1\}^N \longrightarrow \{0,1\}^t, \qquad F_q(x) = A(x, q),$$

where A is the answer map. The map F_q is injective. Indeed, suppose $F_q(x) = F_q(x')$ for distinct $x, x' \in \{0,1\}^N$, and choose $i \in [N]$ such that $x[i] \neq x'[i]$. Perfect correctness with randomness r_i requires

$$D(i, r_i, F_q(x)) = x[i] \quad \text{and} \quad D(i, r_i, F_q(x')) = x'[i],$$

which contradicts $F_q(x) = F_q(x')$. Hence

$$2^N \leq 2^t,$$

and therefore $t \geq N$. □

3.2.2 PIR over Longer Records

Theorem 3.3 [HW 2, Exercise 2]. Let $(Q, A_1, \ldots, A_k, D)$ be a perfectly correct, perfectly 1-private PIR for databases in $\{0,1\}^N$. Then there is a perfectly correct, perfectly 1-private PIR for databases in $(\{0,1\}^b)^N$ with the same query algorithm and with each answer length multiplied by b.

Proof

For

$$x = (x[1], \ldots, x[N]) \in (\{0,1\}^b)^N$$

define $x^{(1)}, \ldots, x^{(b)} \in \{0,1\}^N$ by

$$x^{(\ell)}[i] = x[i]_\ell, \qquad i \in [N],\ \ell \in [b].$$

Use the original query algorithm. For every $s \in [k]$, define

$$\widetilde{A}_s(x, q_s) = \big(A_s(x^{(1)}, q_s), \ldots, A_s(x^{(b)}, q_s)\big).$$

If $\widetilde{a}_s = (a_s^{(1)}, \ldots, a_s^{(b)})$, define

$$\widetilde{D}(i, r, \widetilde{a}_1, \ldots, \widetilde{a}_k) = \big(D(i, r, a_1^{(\ell)}, \ldots, a_k^{(\ell)})\big)_{\ell=1}^b.$$

Perfect correctness of the original protocol implies

$$\widetilde{D}(i, r, \widetilde{a}_1, \ldots, \widetilde{a}_k) = x[i].$$

The query distribution is unchanged, so perfect 1-privacy is preserved. Each new answer concatenates b original answers. □

Applied to the Hadamard protocol, this transformation gives a two-server PIR for b-bit records.

3.3 Two-Server PIR from the Hadamard Code

Theorem 3.4 [Hadamard PIR]. There is a perfectly correct, perfectly 1-private two-server PIR for $x \in \mathbb{F}_2^N$ with two N-bit queries and two one-bit answers.

Proof

For $i \in [N]$, let $e_i \in \mathbb{F}_2^N$ denote the i-th unit vector.

Protocol (Hadamard PIR)

The query algorithm samples $r \xleftarrow{R} \mathbb{F}_2^N$ and outputs

$$Q(i, r) = (q_1, q_2) = (r, r + e_i).$$

For $s \in \{1, 2\}$, the answer and decoding maps are

$$A_s(x, q_s) = \langle x, q_s \rangle, \qquad D(i, r, a_1, a_2) = a_1 + a_2.$$

Correctness follows from

$$a_1 + a_2 = \langle x, q_1 + q_2 \rangle = \langle x, e_i \rangle = x[i].$$

For privacy, $Q_1(i, R) = R$ is uniform over $\mathbb{F}_2^N$, and $Q_2(i, R) = R + e_i$ is uniform over the same space. Thus the distribution of each individual query is independent of i. □

3.3.1 Two Servers with $\sqrt{N}$-Bit Records

Assume $N = m^2$ and identify the database with a matrix $X \in \mathbb{F}_2^{m \times m}$ whose records are its columns.

Theorem 3.5 [Hadamard PIR for length-m records]. There is a perfectly correct, perfectly 1-private two-server PIR that retrieves a column of X using two m-bit queries and two m-bit answers.

Proof

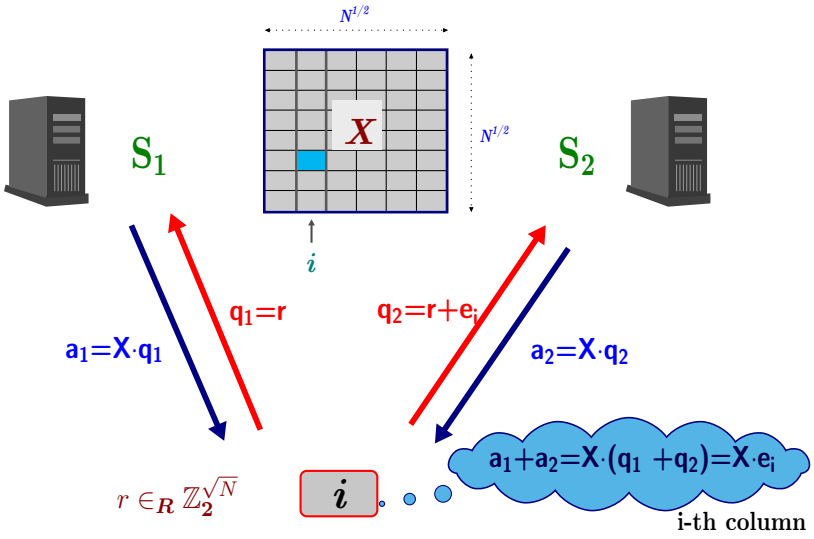
For a requested column $i \in [m]$, sample $r \xleftarrow{R} \mathbb{F}_2^m$ and set

$$q_1 = r, \qquad q_2 = r + e_i, \qquad a_s = X q_s.$$

The decoder outputs $a_1 + a_2$. Its output is

$$a_1 + a_2 = X(q_1 + q_2) = X e_i,$$

which is the i-th column of X. Each query is uniform over $\mathbb{F}_2^m$, so perfect 1-privacy holds. The total communication is $4m = O(N^{1/2})$ bits. □



3.3.2 Linear Secret-Sharing Formulation

Let $f_x : \mathbb{F}^m \to \mathbb{F}$ be linear and suppose that designated vectors $v_1, \ldots, v_N \in \mathbb{F}^m$ satisfy $f_x(v_i) = x[i]$. If $v_i = q_1 + q_2$ is additively shared so that each q_s is uniform, then linearity gives

$$f_x(q_1) + f_x(q_2) = f_x(v_i) = x[i].$$

Consequently, additive secret sharing of the designated vector yields a two-server PIR whenever the shares have index-independent marginals.

3.4 Four-Server Hypercube PIR

Assume $N = m^2$, identify x with $X \in \mathbb{F}_2^{m \times m}$, and define

$$f_x(a, b) = a^T X b.$$

Protocol (Four-server hypercube PIR)

For a requested position $(i, j) \in [m]^2$, sample $a, b \xleftarrow{R} \mathbb{F}_2^m$ and define

$$q_1 = (a, b), \qquad q_2 = (a + e_i, b),$$
$$q_3 = (a, b + e_j), \qquad q_4 = (a + e_i, b + e_j).$$

The answer map of server ℓ is $A_\ell(X, q_\ell) = f_X(q_\ell)$.

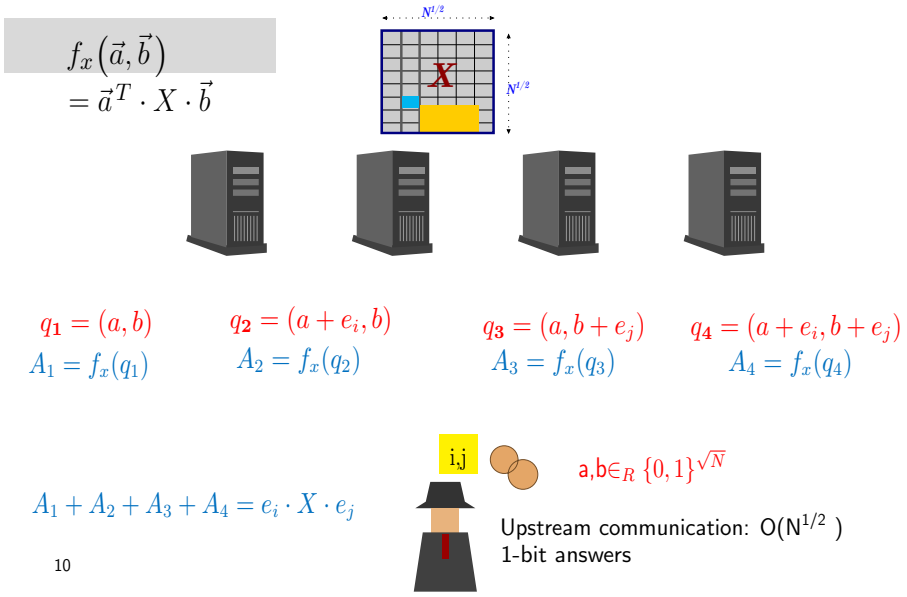
Theorem 3.6 [Four-server hypercube PIR]. The protocol is perfectly correct and perfectly 1-private. It has four queries of length $2m$ and four one-bit answers.

Proof

Bilinearity over $\mathbb{F}_2$ gives

$$A_1 + A_2 + A_3 + A_4 = e_i^T X e_j = X[i,j].$$

For each ℓ, the two components of q_ℓ are independent uniform vectors translated by fixed vectors. Hence q_ℓ is uniform over $\mathbb{F}_2^m \times \mathbb{F}_2^m$ and its distribution is independent of (i,j). The total communication is $8m + 4 = O(N^{1/2})$ bits. □



3.4.1 Three-Dimensional Hypercube PIR

Assume $N = m^3$, index the database by triples $(a,b,c) \in [m]^3$, and define

$$f_x(u,v,w) = \sum_{a,b,c \in [m]} x[a,b,c] u_a v_b w_c.$$

The function is multilinear and $f_x(e_a, e_b, e_c) = x[a,b,c]$.

Theorem 3.7 [HW 2, Exercise 3: eight-server hypercube PIR]. The following protocol is a perfectly correct, perfectly 1-private eight-server PIR with communication $O(N^{1/3})$.

Proof

Protocol (Eight-server hypercube PIR)

For a requested position (i, j, k), choose independent uniform $r, s, t \in \mathbb{F}_2^m$. The server indexed by $(\alpha, \beta, \gamma) \in \{0,1\}^3$ receives

$$q_{\alpha,\beta,\gamma} = (r + \alpha e_i, s + \beta e_j, t + \gamma e_k),$$

and returns $a_{\alpha,\beta,\gamma} = f_x(q_{\alpha,\beta,\gamma})$.

Multilinearity and cancellation over $\mathbb{F}_2$ give

$$\sum_{\alpha,\beta,\gamma \in \{0,1\}} a_{\alpha,\beta,\gamma} = f_x(e_i, e_j, e_k) = x[i, j, k].$$

Each individual query is uniform and independent of (i, j, k). There are eight queries of length $3m$ and eight one-bit answers, for total communication $O(N^{1/3})$. □

Theorem 3.8 [HW 2, Exercise 3, bonus: derivative reduction]. Over a field of characteristic greater than three, the degree-three hypercube protocol can be implemented with two servers and $O(N^{1/3})$ field elements of communication.

Proof

Replace $\mathbb{F}_2$ in the preceding multilinear construction by a field $\mathbb{F}$ of characteristic greater than three, and sample $r, s, t \xleftarrow{R} \mathbb{F}^m$. Use the two queries

$$q_0 = (r, s, t), \qquad q_1 = (r + e_i, s + e_j, t + e_k).$$

Define

$$P(\lambda) = f_x(r + \lambda e_i, s + \lambda e_j, t + \lambda e_k).$$

This polynomial has degree at most three, and its coefficient of λ^3 is $f_x(e_i, e_j, e_k) = x[i, j, k]$. Server $b \in \{0, 1\}$ returns $P(b)$ and all $3m$ first partial derivatives of f_x at q_b. By the chain rule, these values determine $P'(b)$. The Hermite data

$$P(0), P'(0), P(1), P'(1)$$

uniquely determine P, because the field has characteristic different from 2 and 3. The coefficient of λ^3 is $x[i, j, k]$. Both queries are uniform, and each answer contains $3m + 1$ field elements. □

3.5 Three-Server Reed–Muller PIR

Let $\mathbb{F}$ contain distinct nonzero elements $1, 2, 3$. Fix designated points $v_1, \ldots, v_N \in \mathbb{F}^m$ and, for every database $x \in \mathbb{F}^N$, a polynomial $f_x : \mathbb{F}^m \to \mathbb{F}$ of total degree at most two satisfying

$$f_x(v_i) = x[i], \qquad i \in [N].$$

Theorem 3.9 [Three-server Reed–Muller PIR]. If $m = O(N^{1/2})$, the following protocol is a perfectly

correct, perfectly 1-private three-server PIR with query length $O(N^{1/2})$ field elements and one field-element answer from each server.

Proof

For a requested index i, sample $r \xleftarrow{R} \mathbb{F}^m$ and define

$$q_\ell = v_i + \ell r, \qquad a_\ell = f_x(q_\ell), \qquad \ell \in \{1, 2, 3\}.$$

The univariate polynomial

$$P(T) = f_x(v_i + Tr)$$

has degree at most two. The decoder interpolates P from $P(1), P(2), P(3)$ and outputs

$$P(0) = f_x(v_i) = x[i].$$

For every nonzero ℓ, multiplication by ℓ is a permutation of $\mathbb{F}^m$; hence q_ℓ is uniform and independent of i. □



Shamir's SS

$q_1 = r \cdot 1 + e_{i,j}$ $a_1 = f_x(q_1)$

$q_2 = r \cdot 2 + e_{i,j}$ $a_2 = f_x(q_2)$

$q_3 = r \cdot 3 + e_{i,j}$ $a_3 = f_x(q_3)$

3.5.1 Interpolation Equation

Let $P(t) = p_2 t^2 + p_1 t + p_0$. Given $a_1 = P(1)$, $a_2 = P(2)$, and $a_3 = P(3)$, recover the coefficients from

$$\begin{cases} p_2 1^2 + p_1 1 + p_0 = a_1, \\ p_2 2^2 + p_1 2 + p_0 = a_2, \\ p_2 3^2 + p_1 3 + p_0 = a_3. \end{cases}$$

The coefficient matrix is a Vandermonde matrix and is invertible because $1, 2, 3$ are distinct in $\mathbb{F}$.

3.5.2 Reducing a Server Using Partial Derivatives

Retain the queries $q_1 = v_i + r$ and $q_2 = v_i + 2r$. In addition to $P(1)$ and $P(2)$, server 1 returns all first partial derivatives of f_x at q_1. The chain rule gives

$$P'(t) = \sum_{j=1}^{m} r_j \frac{\partial f_x}{\partial z_j}(v_i + tr).$$

Thus the decoder obtains $P'(1)$. The data $P(1), P'(1), P(2)$ uniquely determine a polynomial of degree at most two, and therefore determine $P(0) = x[i]$. The two query distributions remain uniform.

3.5.3 Four-Server RM-PIR and Reduction to Two Servers

Theorem 3.10 [HW 2, Exercise 4]. Over a field of characteristic greater than three, a degree-three generalization of RM-PIR gives a perfectly 1-private four-server PIR with communication $O(N^{1/3})$, and partial derivatives reduce it to two servers with the same asymptotic communication.

Proof

Let $m = O(N^{1/3})$, let $f_x : \mathbb{F}^m \to \mathbb{F}$ have total degree at most three, and let designated points v_i satisfy $f_x(v_i) = x[i]$. For a requested index i, choose uniform $r \in \mathbb{F}^m$ and define

$$q_\ell = v_i + \ell r, \qquad \ell \in \{1, 2, 3, 4\}.$$

For $P(t) = f_x(v_i + tr)$, the four answers are $P(1), P(2), P(3), P(4)$. Interpolation recovers $P(0) = x[i]$. Every query is uniform.
To retain only servers 1 and 2, each returns $P(\ell)$ and all first partial derivatives at q_ℓ. The client computes

$$P'(\ell) = \sum_{j=1}^{m} r_j \frac{\partial f_x}{\partial z_j}(q_\ell).$$

The Hermite data $P(1), P'(1), P(2), P'(2)$ uniquely determine a polynomial of degree at most three. Each retained server sends $m + 1$ field elements, so the communication remains $O(N^{1/3})$. □

3.6 Matching Vectors

3.6.1 Hadamard as an Inner-Product Construction

In the Hadamard protocol,

$$a_s = \left\langle \sum_{j=1}^{N} x[j] e_j, q_s \right\rangle$$

and $q_2 - q_1 = e_i$. Therefore

$$a_2 - a_1 = \sum_{j=1}^{N} x[j] \langle e_j, e_i \rangle = x[i].$$

Matching-vector constructions replace the standard basis vectors by vectors of dimension $h \ll N$.

3.6.2 A Conditional Two-Server Cancellation Construction

Definition 3.11 [Cancellation matching-vector family]. A cancellation matching-vector family of size N and dimension h is a pair of indexed families

$$U = (u_1, \ldots, u_N), \qquad V = (v_1, \ldots, v_N)$$

in $\mathbb{Z}_6^h$ satisfying

$$\langle u_i, v_j \rangle_6 = \begin{cases} 1, & i = j, \\ 2, 3, \text{ or } 4, & i \neq j. \end{cases}$$

Thus every off-diagonal inner product vanishes modulo 2 or modulo 3, whereas every diagonal inner product equals one in both fields.

For $w \in \mathbb{Z}_6^h$, write $w^{(2)}$ and $w^{(3)}$ for its coordinate-wise reductions modulo 2 and modulo 3, respectively. The query algorithm samples $r \xleftarrow{R} \mathbb{Z}_2^h$ and defines

$$q_1 = r, \qquad q_2 = r + u_i^{(2)}.$$

For $s \in \{1, 2\}$, define the answer map by

$$a_s = \sum_{j=1}^{N} x[j] \langle v_j^{(2)}, q_s \rangle_2 v_j^{(3)} \in \mathbb{Z}_3^h.$$

The decoder computes

$$z = \langle u_i^{(3)}, a_2 - a_1 \rangle_3.$$

Its output bit is 1 if $z \neq 0$ and 0 otherwise.

Lemma 3.12 [Cancellation identity]. For $i, j \in [N]$ and $r \in \mathbb{Z}_2^h$, define

$$\eta_{i,j}(r) = \langle u_i, v_j \rangle_3 \, d_{i,j}(r),$$
$$d_{i,j}(r) = \langle v_j^{(2)}, r + u_i^{(2)} \rangle_2 - \langle v_j^{(2)}, r \rangle_2,$$

where the two values in $\mathbb{Z}_2$ are represented by 0 and 1 before subtraction and the result is reduced modulo 3. Then

$$\eta_{i,j}(r) \in \begin{cases} \{1, -1\}, & i = j, \\ \{0\}, & i \neq j \end{cases}.$$

Proof

For $i = j$, adding $u_i^{(2)}$ toggles the bit $\langle v_i^{(2)}, r \rangle_2$, while $\langle u_i, v_i \rangle_3 = 1$. The difference of the two bit representatives is therefore 1 or -1. For $i \neq j$, the inner product lies in $\{2, 3, 4\}$. If it is 2 or 4, then $\langle u_i^{(2)}, v_j^{(2)} \rangle_2 = 0$, so the two bit values are equal. If it is 3, then $\langle u_i, v_j \rangle_3 = 0$. In either case the product is zero modulo 3. □

Theorem 3.13 [Two-server cancellation protocol]. Given a cancellation matching-vector family as in Definition 3.11, the maps defined above form a perfectly correct, perfectly 1-private two-server PIR with query and answer lengths $O(h)$.

Proof

Expanding the decoder value and applying Lemma 3.12 gives

$$z = \sum_{j=1}^{N} x[j]\eta_{i,j}(r) = x[i]\eta_{i,i}(r).$$

Because $\eta_{i,i}(r)$ is nonzero in $\mathbb{Z}_3$, one has $z \neq 0$ if and only if $x[i] = 1$. Thus the decoder outputs $x[i]$. The query q_1 is uniform in $\mathbb{Z}_2^h$, and q_2 is its translation by the fixed vector $u_i^{(2)}$; hence each individual query is uniform and independent of i. □

3.6.3 Standard Matching-Vector Families

Definition 3.14 [Matching-vector family over $\mathbb{Z}_6$]. A matching-vector family of size N and dimension h is a pair

$$U = (u_1, \ldots, u_N), \qquad V = (v_1, \ldots, v_N)$$

in $\mathbb{Z}_6^h$ such that

$$\langle u_i, v_j \rangle_6 = \begin{cases} 0, & i = j, \\ 1, 3, \text{ or } 4, & i \neq j. \end{cases}$$

Theorem 3.15 [HW 2, Exercise 5]. Assume that a polynomial $P(x_1, \ldots, x_n)$ over $\mathbb{Z}_6$ has total degree $d = O(\sqrt{n})$ and satisfies

$$P(0, \ldots, 0) = 0, \qquad P(x) \in \{1, 3, 4\} \quad \text{for every } x \in \{0,1\}^n \setminus \{0^n\}.$$

Then there is a matching-vector family of size $N = 2^n$ and dimension

$$h = \binom{n+d}{d} = n^{O(\sqrt{n})} = \exp\big(O(\sqrt{\log N \log\log N})\big).$$

Proof

For $b_i, x_i \in \{0,1\}$, define the affine polynomial

$$\delta_{b_i}(x_i) = \begin{cases} x_i, & b_i = 0, \\ 1 - x_i, & b_i = 1. \end{cases}$$

For every $b \in \{0,1\}^n$, define

$$g_b(x_1, \ldots, x_n) = P\big(\delta_{b_1}(x_1), \ldots, \delta_{b_n}(x_n)\big).$$

For $a, b \in \{0,1\}^n$, the input to P is 0^n exactly when $a = b$. Moreover, g_b has degree at most d because every δ_{b_i} is affine. Consequently,

$$g_b(a) = \begin{cases} 0, & a = b, \\ 1, 3, \text{ or } 4, & a \neq b \end{cases} \pmod 6.$$

List all monomials of total degree at most d as $M_1, \dots, M_h$, and write

$$g_b(x) = \sum_{\ell=1}^{h} c_{b,\ell} M_\ell(x).$$

Define

$$u_a = (M_1(a), \dots, M_h(a)), \qquad v_b = (c_{b,1}, \dots, c_{b,h}).$$

Then $\langle u_a, v_b \rangle_6 = g_b(a)$, so Definition 3.14 is satisfied. The dimension is the number of monomials of total degree at most d. □

3.6.4 Four-Server PIR from a Standard Matching-Vector Family

Lemma 3.16 [Embedding into $\mathbb{F}_7^*$]. The map

$$\varphi : \mathbb{Z}_6 \longrightarrow \mathbb{F}_7^*, \qquad \varphi(y) = 3^y \pmod 7,$$

is a group isomorphism.

Proof

The element $3 \in \mathbb{F}_7^*$ has order six, since

$$3^0, 3^1, 3^2, 3^3, 3^4, 3^5 \equiv 1, 3, 2, 6, 4, 5 \pmod 7.$$

Thus φ is a bijective homomorphism between cyclic groups of order six. □

Fix a matching-vector family as in Definition 3.14 and define

$$f_x(z) = \sum_{j=1}^{N} x[j] 3^{\langle v_j, z \rangle_6} \in \mathbb{F}_7.$$

For a requested index i, the query algorithm samples $r \xleftarrow{R} \mathbb{Z}_6^h$ and, for $t \in \{0, 1, 2, 3\}$, defines

$$q_t = r + t u_i$$

The answer map is $A_t(x, q_t) = f_x(q_t)$. Define

$$P(t) = f_x(r + t u_i).$$

For $\ell \in \{0, 1, 3, 4\}$, let

$$c_\ell = \sum_{\substack{j \in [N] \\ \langle v_j, u_i \rangle_6 = \ell}} x[j] 3^{\langle v_j, r \rangle_6}.$$

Then

$$P(t) = c_0 + 3^t c_1 + 3^{3t} c_3 + 3^{4t} c_4,$$

and the matching property implies

$$c_0 = x[i] 3^{\langle v_i, r \rangle_6}.$$

Theorem 3.17 [Four-server matching-vector PIR]. The maps defined above form a perfectly correct, perfectly 1-private four-server PIR with query length $O(h)$ and $O(1)$-bit answers.

Proof

The four answers $P(0), P(1), P(2), P(3)$ form a linear system in (c_0, c_1, c_3, c_4) with coefficient matrix

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 3 & 6 & 4 \\ 1 & 2 & 1 & 2 \\ 1 & 6 & 6 & 1 \end{pmatrix} \in \mathbb{F}_7^{4\times 4}.$$

Its determinant is $2 \pmod 7$, so the system is invertible. After recovering c_0, divide by the known nonzero value $3^{\langle v_i, r\rangle_6}$ to obtain $x[i]$.
For every fixed t, the query q_t is uniform in $\mathbb{Z}_6^h$ because it is a translation of the uniform vector r. Hence each server's query is independent of i. □

3.6.5 Further Optimizations

The three-server refinement has query length

$$O(h) = N^{O(\log\log N/\log N)}$$

and $O(1)$-bit answers. Balancing the query and answer lengths gives total communication

$$N^{O\left((\log\log N/\log N)^{2/3}\right)}.$$

With $O(1)$-bit answers, the analogous two-server improvement is impossible.

3.7 From IT-PIR to Locally Decodable Codes

Definition 3.18 [Locally decodable code]. Let Σ be a finite alphabet. A (k, δ, p)-locally decodable code is a map

$$C : \{0,1\}^N \longrightarrow \Sigma^M$$

with a randomized decoder $\mathcal{D}$ that makes at most k queries to its oracle word and satisfies the following condition: for every $x \in \{0,1\}^N$, every $i \in [N]$, and every $y \in \Sigma^M$ such that

$$\left|\{j \in [M] : y[j] \neq C(x)[j]\}\right| \leq \delta M,$$

one has

$$\Pr[\mathcal{D}^y(i) = x[i]] \geq p,$$

where the probability is over the randomness of $\mathcal{D}$.

Theorem 3.19 [PIR-to-LDC correspondence, Katz–Trevisan 00]. Let Π be a perfectly correct k-server PIR in which every query belongs to $\{0,1\}^\alpha$, every answer belongs to $\Sigma = \{0,1\}^\beta$, and $Q_s(i, R)$ is uniform over $\{0,1\}^\alpha$ for every $s \in [k]$ and $i \in [N]$. Then Π defines a $(k, \delta, 1 - k\delta)$-locally decodable code of length

$$M = k2^\alpha$$

over Σ.

Proof

Index the codeword coordinates by pairs $(s, q) \in [k] \times \{0,1\}^\alpha$ and define

$$C(x)[s, q] = A_s(x, q).$$

The local decoder samples $r \xleftarrow{R} \mathcal{R}$ and computes

$$(q_1, \ldots, q_k) = Q(i, r).$$

It reads the coordinates (s, q_s) for $s \in [k]$ and applies the PIR decoding map to the resulting k symbols.
Let $B_s \subseteq \{0,1\}^\alpha$ be the set of corrupted coordinates in the block belonging to server s. Uniformity gives

$$\Pr[q_s \in B_s] = \frac{|B_s|}{2^\alpha}.$$

If at most a δ fraction of the M coordinates are corrupted, then

$$\sum_{s=1}^{k} |B_s| \leq \delta k 2^\alpha.$$

The union bound therefore gives

$$\Pr[\exists s : q_s \in B_s] \leq \sum_{s=1}^{k} \frac{|B_s|}{2^\alpha} \leq k\delta.$$

Conditioned on querying no corrupted coordinate, perfect PIR correctness implies that the decoder outputs $x[i]$. □

The reverse implication requires the local decoder's query distribution to satisfy the corresponding smoothness condition.

4 Computational PIR (cPIR)

4.1 Computational Privacy and Indistinguishability

Definition 4.1 [Computationally 1-private k-server PIR]. A computationally 1-private k-server PIR protocol is a family of probabilistic polynomial-time algorithms indexed by a security parameter λ:

$$\begin{aligned} Q &: (1^\lambda, N, i; r) \longmapsto (q_1, \ldots, q_k), \\ A_s &: (1^\lambda, x, q_s) \longmapsto a_s, \qquad s \in [k], \\ D &: (1^\lambda, N, i, r, a_1, \ldots, a_k) \longmapsto z, \end{aligned}$$

where $x \in \{0,1\}^N$ and $i \in [N]$. The protocol satisfies:

1. *Correctness.* For every λ, N, x, i,

$$\Pr[D(1^\lambda, N, i, R, a_1, \ldots, a_k) = x[i]] \geq 1 - \mathrm{negl}(\lambda),$$

   where $(q_1, \ldots, q_k) = Q(1^\lambda, N, i; R)$ and $a_s = A_s(1^\lambda, x, q_s)$.

2. *Computational 1-privacy.* For every polynomially bounded $N = N(\lambda)$, every $s \in [k]$, and every $i_0, i_1 \in [N]$, the ensembles

$$\{Q_s(1^\lambda, N, i_0; R)\}_{\lambda \in \mathbb{N}} \quad \text{and} \quad \{Q_s(1^\lambda, N, i_1; R)\}_{\lambda \in \mathbb{N}}$$

   are computationally indistinguishable.

Definition 4.2 [Distinguishing gap]. Let X, Y be distributions over a common finite set Ω, and let $A : \Omega \to \{0,1\}$. Define

$$\Delta_A(X, Y) = \left| \Pr[A(X) = 1] - \Pr[A(Y) = 1] \right|.$$

The distributions are perfectly indistinguishable if $\Delta_A(X,Y) = 0$ for every A; they are ϵ-statistically indistinguishable if $\Delta_A(X,Y) \leq \epsilon$ for every A; and they are (t, ϵ)-computationally indistinguishable if this inequality holds for every probabilistic algorithm A of running time at most t.

Definition 4.3 [Negligible function]. A function $\mu : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is negligible if for every constant $c > 0$ there exists λ_c such that

$$\mu(\lambda) < \lambda^{-c}$$

for every $\lambda \geq \lambda_c$.

Definition 4.4 [Asymptotic indistinguishability]. Distribution ensembles $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ are statistically indistinguishable, written $X \approx_s Y$, if

$$\max_A \Delta_A(X_\lambda, Y_\lambda) = \mathrm{negl}(\lambda).$$

They are computationally indistinguishable, written $X \approx_c Y$, if for every probabilistic polynomial-time A,

$$\Delta_A(X_\lambda, Y_\lambda) = \mathrm{negl}(\lambda).$$

Subexponential and exponential variants replace polynomial-time distinguishers by distinguishers of running time $\exp(\lambda^\delta)$ and $\exp(\delta\lambda)$, respectively, for a fixed constant $\delta > 0$.

Lemma 4.5 [Closure properties]. Let all ensembles below be efficiently samplable. Computational indistinguishability satisfies:

1. Transitivity: if $X \approx_c Y$ and $Y \approx_c Z$, then $X \approx_c Z$.
2. Closure under efficient computation: if $X \approx_c Y$ and F is PPT, then $F(X) \approx_c F(Y)$.
3. Closure under independent products: if $X \approx_c X'$ and $Y \approx_c Y'$, and the two components are sampled independently, then $(X, Y) \approx_c (X', Y')$.

4.2 cPIR from Fully Homomorphic Encryption

This section replaces the information-theoretic privacy condition of Definition 3.1 by the computational condition of Definition 4.1.

Definition 4.6 [Fully homomorphic encryption]. An FHE scheme consists of the following four probabilistic algorithms, each running in polynomial time:

$$\begin{aligned}(pk, sk) &\leftarrow \mathrm{KeyGen}(1^\lambda),\\ c &\leftarrow \mathrm{Enc}_{pk}(m),\\ c' &\leftarrow \mathrm{Eval}_{pk}(C, c_1, \dots, c_t),\\ m' &= \mathrm{Dec}_{sk}(c').\end{aligned}$$

Correctness requires that for every circuit C and messages $m_1, \dots, m_t$,

$$\mathrm{Dec}_{sk}\big(\mathrm{Eval}_{pk}(C, \mathrm{Enc}_{pk}(m_1), \dots, \mathrm{Enc}_{pk}(m_t))\big) = C(m_1, \dots, m_t)$$

except with negligible probability. Compactness requires the evaluated ciphertext length to be bounded by a polynomial in λ and the output length of C, independently of $|C|$. Security is semantic security of the underlying encryption scheme.

Theorem 4.7 [Single-server cPIR from FHE]. Assume an FHE scheme whose message space contains $[N]$. Then there is a computationally private single-server PIR for databases in $\{0,1\}^N$.

Proof

Define the database-selection circuit

$$f_x(j) = x[j].$$

Protocol (Single-server FHE-PIR)

The query algorithm generates (pk, sk) and outputs

$$q = (pk, \mathrm{Enc}_{pk}(i)).$$

The answer map outputs

$$a = \mathrm{Eval}_{pk}(f_x, \mathrm{Enc}_{pk}(i)),$$

and the decoder outputs

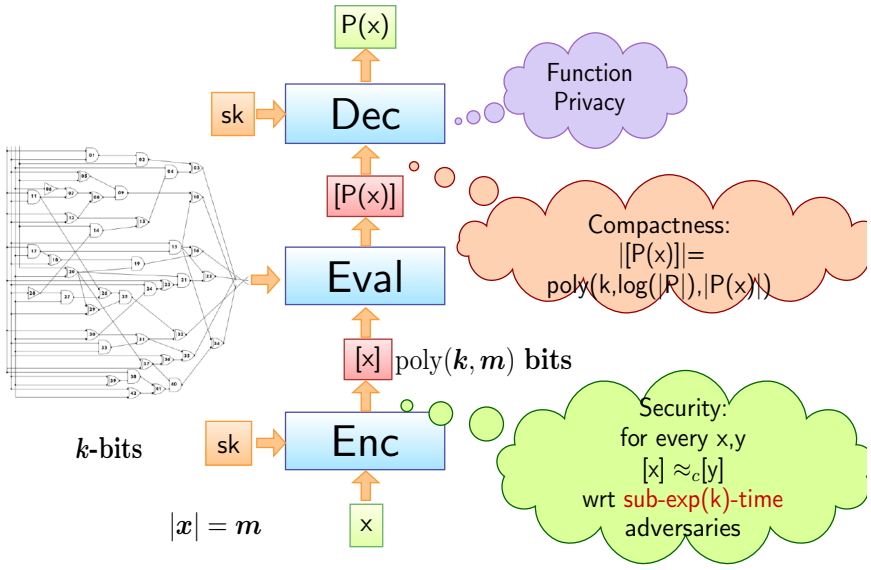
$$\mathrm{Dec}_{sk}(a) = x[i].$$

FHE correctness proves PIR correctness. For every $i_0, i_1 \in [N]$, semantic security implies

$$(pk, \mathrm{Enc}_{pk}(i_0)) \approx_c (pk, \mathrm{Enc}_{pk}(i_1)),$$

which proves computational privacy. □

By compactness, the answer length is polynomial in λ and the output length. The query length is polynomial in λ and $\log N$.



4.3 cPIR from Additively Homomorphic Encryption

An additively homomorphic encryption scheme supports evaluation of every linear function

$$L : \mathbb{F}^m \longrightarrow \mathbb{F}.$$

on encrypted inputs, with decryption recovering $L(v)$.

4.3.1 Basic Construction [Kushilevitz–Ostrovsky 97]

Assume $N = m^2$ and identify the database with $X \in \mathbb{F}^{m \times m}$. Let one ciphertext have length κ field elements.

Theorem 4.8 [Single-server PIR from additive homomorphism]. The following protocol is correct and computationally private. Its query and answer lengths are both κm field elements.

Proof

Protocol (Additively homomorphic PIR)

For a requested position $(i, j) \in [m]^2$, the query algorithm outputs the componentwise encryption

$$q = \big(\mathrm{Enc}_{sk}(e_j[1]), \ldots, \mathrm{Enc}_{sk}(e_j[m])\big).$$

For every $r \in [m]$, define

$$f_r(v) = \langle X_r, v \rangle.$$

The answer map returns

$$a_r = \mathrm{Eval}(f_r, q), \qquad r \in [m].$$

The decoder outputs

$$\mathrm{Dec}_{sk}(a_i) = \langle X_i, e_j \rangle = X[i, j].$$

Correctness follows from homomorphic correctness. Computational privacy follows from semantic security, since encryptions of e_j and $e_{j'}$ are computationally indistinguishable for every $j, j' \in [m]$. □

4.3.2 Recursive Optimization

Arrange the database as $N^{1-\alpha}$ rows of length N^{α}. The first evaluation layer produces

$$a_1, \ldots, a_{N^{1-\alpha}}, \qquad a_r = \mathrm{Eval}(f_r, q),$$

The same construction is applied recursively to the encrypted answer vector.

Theorem 4.9 [Recursive AHE communication bound]. Let a field element be encrypted by κ field elements, and assume $d = 1/\alpha \in \mathbb{N}$. The recursive communication satisfies the formal recurrence

$$T(N) = \kappa N^{\alpha} + T\big(\kappa N^{1-\alpha}\big).$$

With recursion depth d, the query and answer lengths are

$$\text{user-to-server communication} = d\kappa N^{\alpha},$$

and

$$\text{server-to-user communication} = \kappa^{d}.$$

If $\kappa = N^{\beta}$, these are respectively

$$O(N^{\alpha+\beta}) \quad \text{and} \quad O(N^{\beta/\alpha}).$$

Consequently, communication $O(N^\epsilon)$ is achievable for every $\epsilon > 0$ by taking

$$\alpha = \frac{\epsilon}{2}, \qquad \beta = \frac{\epsilon^2}{2}.$$

Proof

At each of the d levels, the query contains N^α ciphertexts, for $d\kappa N^\alpha$ field elements. The recursive process leaves a single ciphertext whose expansion through d encryption layers has size κ^d. Substituting $\kappa = N^\beta$ yields the displayed exponents. With the chosen α, β, both exponents are at most ϵ. □

4.4 Two-Server cPIR from Pseudorandom Generators

Definition 4.10 [Pseudorandom generator]. Let $M = M(\lambda) > \lambda$ be polynomially bounded. A polynomial-time function

$$G : \{0,1\}^\lambda \longrightarrow \mathbb{F}_2^M$$

is a pseudorandom generator if

$$\{G(U_\lambda)\}_{\lambda\in\mathbb{N}} \approx_c \{U_M\}_{\lambda\in\mathbb{N}},$$

where U_d denotes the uniform distribution on $\{0,1\}^d$.

Definition 4.11 [Compressed sharing of a unit vector]. Let Gen and Expand be probabilistic polynomial-time algorithms such that

$$(K_0, K_1) \leftarrow \mathrm{Gen}(1^\lambda, N, i), \qquad Q_p = \mathrm{Expand}(K_p) \in \mathbb{F}_2^N.$$

They form a compressed sharing scheme for unit vectors if:

1. for every $i \in [N]$, $Q_0 + Q_1 = e_i$ with probability one;
2. for every $p \in \{0,1\}$ and $i_0, i_1 \in [N]$,

$$\{K_p \mid i = i_0\} \approx_c \{K_p \mid i = i_1\}.$$

Theorem 4.12 [cPIR from compressed unit-vector sharing]. A compressed sharing scheme with key length $\ell(\lambda, N)$ yields a computationally private two-server PIR with query length $2\ell(\lambda, N)$ and two one-bit answers.

Proof

Protocol (PIR from compressed unit-vector sharing)

The query algorithm runs $\mathrm{Gen}(1^\lambda, N, i)$ and sends K_p to server p. Server p computes

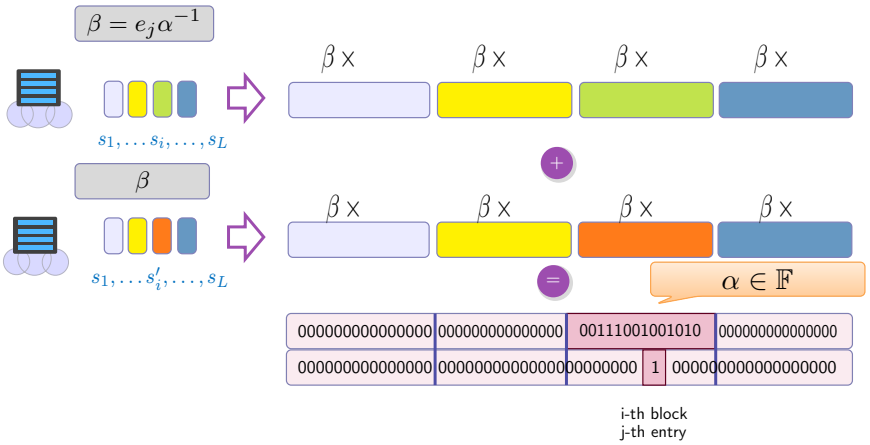
$$a_p = \langle x, \mathrm{Expand}(K_p)\rangle.$$

The decoder outputs $a_0 + a_1$.

Correctness follows from

$$a_0 + a_1 = \langle x, Q_0 + Q_1 \rangle = \langle x, e_i \rangle = x[i].$$

Computational 1-privacy is exactly the second condition in the definition. □



4.4.1 Additive Correction [Gilboa–Ishai 14]

Let $N = LM$ and let

$$G : \{0,1\}^\lambda \longrightarrow \mathbb{F}_2^M$$

be a pseudorandom generator. Identify a requested coordinate with a pair $(i, j) \in [L] \times [M]$.

Theorem 4.13 [Additive-correction sharing]. There is a compressed sharing of $e_{i,j} \in \mathbb{F}_2^{LM}$ whose key length is

$$L\lambda + L + M.$$

Proof

Protocol (Additive-correction key generation)

Sample seeds $s_1^0, \ldots, s_L^0$ independently. Set $s_\ell^1 = s_\ell^0$ for $\ell \neq i$, and sample s_i^1 independently. Sample $b^0 \xleftarrow{R} \mathbb{F}_2^L$ and set

$$b^1 = b^0 + e_i.$$

Define

$$\Delta = e_j + G(s_i^0) + G(s_i^1) \in \mathbb{F}_2^M$$

and give server p the key

$$K_p = (s_1^p, \ldots, s_L^p, b^p, \Delta).$$

The ℓ-th block of $\mathrm{Expand}(K_p)$ is

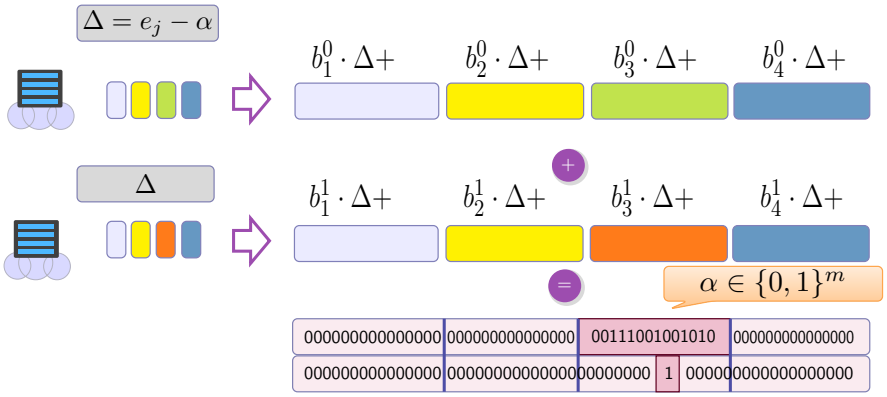
$$Q_{p,\ell} = G(s_\ell^p) + b_\ell^p \Delta.$$

If $\ell \neq i$, then $s_\ell^0 = s_\ell^1$ and $b_\ell^0 = b_\ell^1$, so $Q_{0,\ell} + Q_{1,\ell} = 0$. For $\ell = i$,

$$Q_{0,i} + Q_{1,i} = G(s_i^0) + G(s_i^1) + \Delta = e_j.$$

Thus $Q_0 + Q_1 = e_{i,j}$.
For the privacy of K_0, replace the hidden value $G(s_i^1)$ by a uniform vector in $\mathbb{F}_2^M$. PRG security implies that this change is computationally indistinguishable, and in the resulting hybrid Δ is uniform and independent of (i, j). All seeds in K_0 and the vector b^0 are independently uniform. The argument for K_1 is symmetric, using the hidden seed s_i^0 and the fact that b^1 is uniform. Hence each key distribution is computationally independent of (i, j). □

Balancing $L\lambda$ and $M = N/L$ gives key length $O(\sqrt{N\lambda})$.



4.5 Function Secret Sharing

Definition 4.14 [Function secret sharing]. Let $\mathcal{F}$ be a family of functions from a domain D to a finite abelian group G. A two-party function secret-sharing scheme consists of probabilistic polynomial-time algorithms

$$(K_0, K_1) \leftarrow \mathrm{Gen}(1^\lambda, f),$$
$$y_p = \mathrm{Eval}(p, K_p, x), \qquad p \in \{0, 1\}.$$

Correctness requires that for every $f \in \mathcal{F}$ and $x \in D$,

$$\mathrm{Eval}(0, K_0, x) + \mathrm{Eval}(1, K_1, x) = f(x)$$

with probability one. Privacy requires that for each $p \in \{0, 1\}$ and every $f_0, f_1 \in \mathcal{F}$ of the same public size,

$$\{K_p : (K_0, K_1) \leftarrow \mathrm{Gen}(1^\lambda, f_0)\} \approx_c \{K_p : (K_0, K_1) \leftarrow \mathrm{Gen}(1^\lambda, f_1)\}.$$

Definition 4.15 [Distributed point function]. For $\alpha \in \{0, 1\}^n$ and $\beta \in G$, define the point function

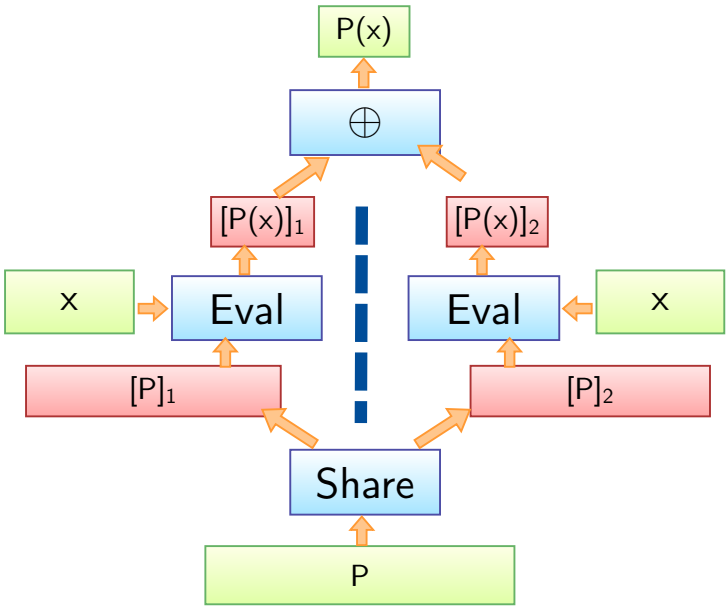
$$f_{\alpha,\beta} : \{0, 1\}^n \longrightarrow G$$

by

$$f_{\alpha,\beta}(\alpha) = \beta, \qquad f_{\alpha,\beta}(x) = 0 \quad \text{for } x \neq \alpha.$$

A distributed point function is an FSS scheme for the family

$$\{f_{\alpha,\beta} : \alpha \in \{0,1\}^n,\ \beta \in G\}.$$



4.6 Distributed Point Functions: The Boyle–Gilboa–Ishai Tree

Assume $N = 2^d$ and write the target location as

$$(\alpha_1, \ldots, \alpha_d), \qquad d = \log_2 N.$$

At level ℓ, each evaluator holds a seed-control pair. The expansion algorithm produces the two child pairs and then applies the public correction word for level ℓ. The maintained invariant is:

- the two states associated with the prefix $(\alpha_1, \ldots, \alpha_\ell)$ are unequal;
- the two states associated with every other prefix of length ℓ are equal.

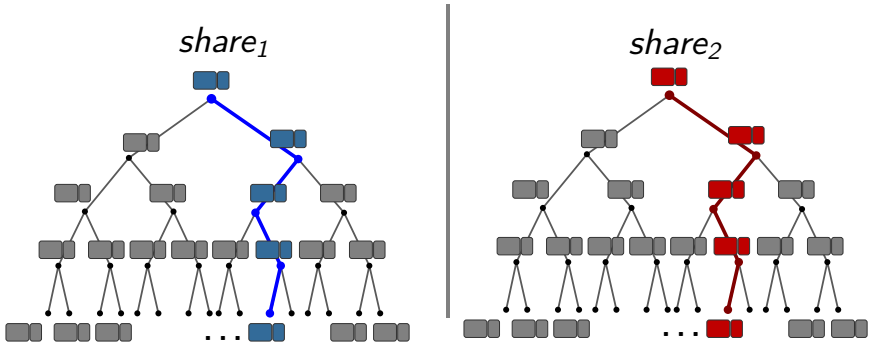
Theorem 4.16 [DPF tree key length]. If a seed has length λ, the unoptimized tree construction has key length

$$\lambda + 1 + 2(\lambda + 1)\log_2 N$$

bits per evaluator.

Proof

The root state contains one λ-bit seed and one control bit. Each of the d levels contributes a correction word containing two seed-control pairs, hence $2(\lambda+1)$ bits. Summing these contributions gives the displayed expression. □

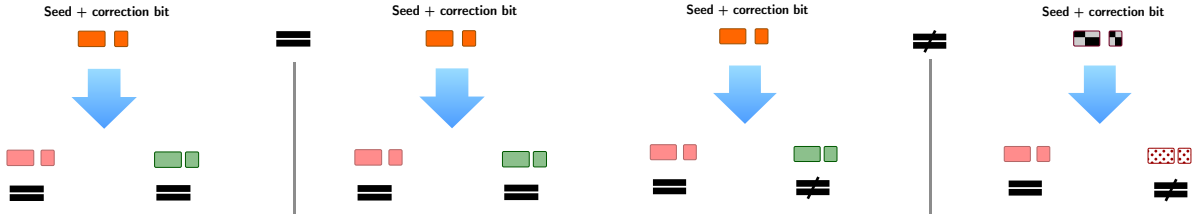


4.6.1 Conditional Correction Gadget

Let V be a vector space over a field $\mathbb{F}$. At a node, an evaluator holds a state $(S, b) \in V \times \mathbb{F}$. Write the two components of the expansion map as $G_L, G_R : V \to V$. For a correction word $(L, R) \in V^2$, define

$$\begin{aligned} \mathrm{Left}(S, b) &= G_L(S) + bL, \\ \mathrm{Right}(S, b) &= G_R(S) + bR. \end{aligned}$$

For unequal input states, the correction is chosen so that one designated branch has equal output states and the other has unequal output states. For equal input states, both output pairs remain equal.



Claim 4.17 [Correction word for the chosen branch]. Suppose the two states are (S, b) and (S', b') with $b \neq b'$. The left children are equal if and only if

$$L = \frac{G_L(S) - G_L(S')}{b' - b}.$$

The right children are unequal if and only if

$$R \neq \frac{G_R(S) - G_R(S')}{b' - b}.$$

Proof

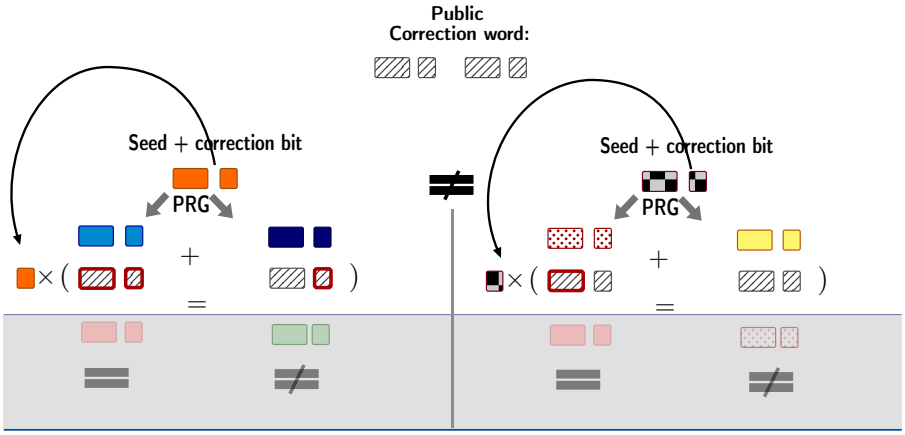
For the left children, equality is equivalent to

$$G_L(S) + bL - \big(G_L(S') + b'L\big) = 0,$$

which rearranges to the displayed value of L because $b' - b \neq 0$. The same calculation characterizes

equality on the right branch. If the two input states are equal, both evaluators apply identical expansions and identical corrections; hence both pairs of children remain equal. □

The DPF key-generation algorithm samples the two root states and computes one correction word at each level so that the invariant follows the path $(\alpha_1, \ldots, \alpha_d)$.



4.6.2 Concrete Efficiency

For the expansion map

$$G : \{0,1\}^{\lambda} \longrightarrow \{0,1\}^{2(\lambda+1)},$$

the key length is $O(\lambda \log N)$. Key generation performs $O(\log N)$ expansions, evaluation at one point performs $\log N$ expansions, and evaluation on the full domain performs $O(N)$ expansions.

4.7 Applications of FSS and DPF

4.7.1 Keyword Search

Let $X = \{x_1, \ldots, x_N\} \subseteq \{0,1\}^n$ and let x be a query keyword. Generate DPF keys (K_0, K_1) for $f_{x,1}$ over $\mathbb{Z}_2$. Server p returns

$$y_p = \bigoplus_{i=1}^{N} \mathrm{Eval}(p, K_p, x_i).$$

Correctness of the DPF gives

$$y_0 \oplus y_1 = \bigoplus_{i=1}^{N} f_{x,1}(x_i) = \mathbf{1}_{\{x \in X\}},$$

where the last equality uses that X is a set.

For records (x_i, p_i) with payloads in an abelian group, define

$$y_p = \sum_{i=1}^{N} p_i \,\mathrm{Eval}(p, K_p, x_i).$$

Then $y_0 + y_1$ equals the sum of the payloads whose keyword is x.

More generally, let (K_0, K_1) be FSS keys for $f : \{0,1\}^n \to \mathbb{Z}_u$ and define

$$y_p = \sum_{i=1}^{N} \mathrm{Eval}(p, K_p, x_i).$$

The reconstruction identity is

$$y_0 + y_1 = \sum_{i=1}^{N} f(x_i).$$

4.7.2 Secure Aggregation

For an update to location $\alpha \in \{0,1\}^n$, generate DPF keys (K_0, K_1) for $f_{\alpha,1} : \{0,1\}^n \to \mathbb{Z}_u$. Server p updates each aggregate coordinate z by

$$X_p[z] \longleftarrow X_p[z] + \mathrm{Eval}(p, K_p, z).$$

After reconstruction, the aggregate changes by one at $z = \alpha$ and is unchanged at every $z \neq \alpha$.

4.7.3 Anonymous Messaging [CBM15]

To write $m \in G$ at location α, generate DPF keys for $f_{\alpha,m}$. Server p adds $\mathrm{Eval}(p, K_p, z)$ to its database share at every location z. The reconstructed database changes by

$$f_{\alpha,m}(z) = \begin{cases} m, & z = \alpha, \\ 0, & z \neq \alpha. \end{cases}$$

The privacy condition of Definition 4.14 hides (α, m) from each individual server.

4.8 Security Analysis of a PIR-Based MPC Candidate

Consider the three-party functionality

$$F(x, i, \perp) = (\perp, x[i], \perp),$$

where Alice holds $x \in \{0,1\}^n$, Bob holds $i \in [n]$, and Carol has no input or output.

Protocol (PIR-based MPC candidate)

Using a two-server PIR (Q, A_1, A_2, D):

1. Alice samples $r \xleftarrow{R} \{0,1\}^n$, sends $y = r \oplus x$ to Carol, and sends r to Bob.
2. Bob samples $(q_1, q_2) = Q(i; r')$, sends q_1 to Alice, and sends q_2 to Carol.
3. Alice and Carol return
$$a_1 = A_1(y, q_1), \qquad a_2 = A_2(y, q_2).$$
4. Bob computes $y[i] = D(a_1, a_2)$ and outputs $y[i] \oplus r[i] = x[i]$.

Theorem 4.18 [Exam A 2025, Question 3]. With the Hadamard PIR, the protocol is correct and protects the index from each of the two parties answering PIR queries. It is not passively private against the querying party, is not passively 2-private, and is not secure against one active corruption.

Proof

Correctness follows from PIR correctness and $y[i] \oplus r[i] = (r[i] \oplus x[i]) \oplus r[i] = x[i]$.
The view of Alice contains a uniformly distributed Hadamard query, and the view of Carol contains the uniformly distributed mask y and a uniformly distributed query. These distributions do not reveal i.
Privacy against Bob fails even when Bob follows the protocol. For the Hadamard answer map,

$$a_1 = \langle y, q_1 \rangle = \langle r, q_1 \rangle + \langle x, q_1 \rangle.$$

Since Bob knows r and q_1, he recovers $\langle x, q_1 \rangle$. To obtain an explicit distinguishing pair, fix $j \neq i$ and compare $x = 0^n$ with $x' = e_j$. Both inputs have prescribed output zero at location i. For x, the recovered inner product is always zero; for x', it equals the uniform bit $q_1[j]$. Hence no simulator given only i and $x[i]$ can reproduce Bob's view.
The protocol is not 2-private. Alice and Carol together see both Hadamard queries and compute

$$q_1 \oplus q_2 = e_i,$$

revealing i. Bob and Carol together know both r and y, and therefore recover

$$x = r \oplus y.$$

The protocol is not actively 1-secure. A malicious Bob need not generate valid queries for a unit vector. He can choose Hadamard queries satisfying

$$q_1 \oplus q_2 = e_1 \oplus e_2,$$

and thereby recover $y[1] \oplus y[2]$. Since he knows r, he obtains the unauthorized linear combination

$$x[1] \oplus x[2] = (y[1] \oplus y[2]) \oplus (r[1] \oplus r[2]).$$

Moreover, a malicious Carol can inject errors in a way that is unavailable in the ideal functionality. □

Remark. PIR privacy protects an honestly generated index query from an individual server. It does not

by itself enforce valid client queries, protect against server collusion, or provide active MPC security.

PART IV

Randomized Encoding

5 Conditional Disclosure of Secrets (CDS)

5.1 Definition and Syntax

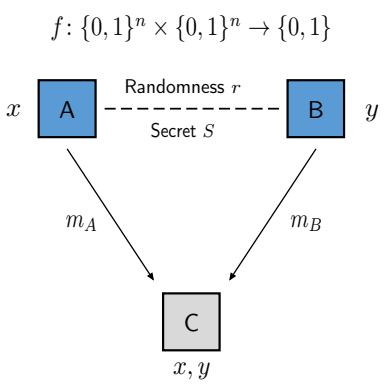
Definition 5.1 [Conditional Disclosure of Secrets]. A CDS protocol for a predicate $f : X \times Y \to \{0,1\}$ and secret space $\mathcal{S}$ consists of a finite randomness space $\mathcal{R}$, message spaces $\mathcal{M}_A, \mathcal{M}_B$, and maps

$$A : X \times \mathcal{S} \times \mathcal{R} \to \mathcal{M}_A,$$
$$B : Y \times \mathcal{S} \times \mathcal{R} \to \mathcal{M}_B,$$
$$C : X \times Y \times \mathcal{M}_A \times \mathcal{M}_B \to \mathcal{S} \cup \{\perp\}.$$

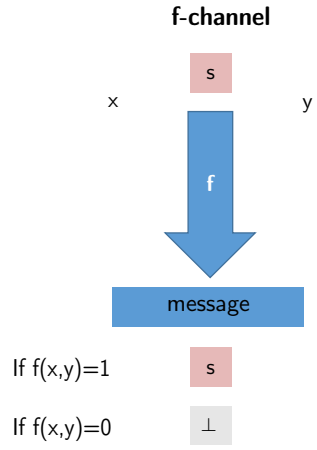
For $R \xleftarrow{R} \mathcal{R}$, write $M_A = A(x, s; R)$ and $M_B = B(y, s; R)$. The following conditions are required.

1. *Perfect correctness:* for every (x, y) with $f(x, y) = 1$ and every $s \in \mathcal{S}$,
$$\Pr[C(x, y, M_A, M_B) = s] = 1.$$
2. *Perfect privacy:* for every (x, y) with $f(x, y) = 0$ and every $s_0, s_1 \in \mathcal{S}$,
$$(A(x, s_0; R), B(y, s_0; R)) \equiv (A(x, s_1; R), B(y, s_1; R)).$$

The communication complexity is the maximum of $|M_A| + |M_B|$ over all inputs, secrets, and randomness values. The randomness complexity is $\lceil \log_2 |\mathcal{R}| \rceil$.



Goal: Charlie learns secret if and only if f(x,y)=1



5.1.1 Example: XOR

Let the secret and all arithmetic be over $\mathbb{F}_2$.

Protocol (CDS for $f(x,y) = x \oplus y$)

On inputs $x, y \in \mathbb{F}_2$ and shared secret $s \in \mathbb{F}_2$:

1. Sample $r_0, r_1 \xleftarrow{R} \mathbb{F}_2$.
2. The first sender transmits $m_A = s + r_x$.
3. The second sender transmits $m_B = r_{1-y}$.
4. If $x \oplus y = 1$, the decoder outputs $m_A + m_B$; otherwise it outputs $\perp$.

If $x \oplus y = 1$, then $x = 1 - y$ and $m_A + m_B = s$. If $x \oplus y = 0$, the two messages are independent uniform bits and hence are independent of s.

5.2 Deterministic CDS Protocols

Definition 5.2 [Deterministic CDS]. A deterministic two-party CDS is a CDS in which Alice and Bob do not share common randomness. Thus their messages are deterministic functions $A(x, s)$ and $B(y, s)$.

Theorem 5.3 [HW 3, Exercise 1: characterization]. A predicate

$$f : X_1 \times X_2 \longrightarrow \{0, 1\}$$

has a deterministic two-party CDS if and only if every 1 in its truth table lies in an all-1 row or in an all-1 column.

Proof

For necessity, suppose $f(x, y) = 1$, but row x contains a zero and column y contains a zero. Choose y_0 and x_0 such that

$$f(x, y_0) = 0, \qquad f(x_0, y) = 0.$$

Privacy at (x, y_0) implies that the deterministic message $A(x, s)$ is independent of s. Privacy at (x_0, y) implies that $B(y, s)$ is independent of s. Therefore the pair $(A(x, s), B(y, s))$ is independent of s, which contradicts correctness at the 1-entry (x, y).
For sufficiency, suppose every 1-entry has the stated property. Alice sends s when row x is all 1 and sends a fixed constant otherwise. Bob sends s when column y is all 1 and sends a fixed constant otherwise. Charlie, who knows (x, y), outputs the message belonging to an all-1 row or column when $f(x, y) = 1$. At a 0-entry, neither its row nor its column is all 1, so both messages are fixed constants and reveal nothing about s. □

Corollary 5.4 [Impossibility for AND]. The predicate

$$f(x_1, x_2) = x_1 \wedge x_2$$

cannot be realized by a deterministic two-party CDS protocol.

Proof

The unique 1-entry $(1, 1)$ lies neither in an all-1 row nor in an all-1 column. The characterization theorem applies. □

Definition 5.5 [Trivial predicate]. A predicate $f : X_1 \times X_2 \to \{0, 1\}$ is trivial if either

1. for every $x \in X_1$ and every $y, z \in X_2$, $f(x, y) = f(x, z)$; or
2. for every $x \in X_2$ and every $y, z \in X_1$, $f(y, x) = f(z, x)$.

Every predicate satisfying either condition has a deterministic CDS: the sender whose input determines f transmits s on accepting inputs and a fixed constant on rejecting inputs. The predicate OR also has a deterministic CDS by the characterization theorem.

5.3 Conditional Enforcement of PIR Query Validity

Let $Q(v, r) = (a, b)$ be the query algorithm of a two-server PIR. Define the query-validity predicate

$$V(a, b) = 1 \quad \Longleftrightarrow \quad \exists v, r \text{ such that } Q(v, r) = (a, b).$$

Lemma 5.6 [Conditional release on well-formed queries]. Let S be any server-side value and let Π_V be a CDS for V whose secret space contains S. If the two servers use (a, b) as their CDS inputs and S as the CDS secret, then the receiver reconstructs S for every (a, b) in the image of Q, while the transcript is independent of S for every (a, b) outside the image of Q.

Proof

If $(a, b) = Q(v, r)$ for some (v, r), then $V(a, b) = 1$ and CDS correctness gives reconstruction. Otherwise $V(a, b) = 0$, and CDS privacy makes the joint message distribution independent of S. □

For the Hadamard PIR, the validity predicate is

$$V(a, b) = 1 \quad \Longleftrightarrow \quad a + b \in \{e_1, \ldots, e_N\}.$$

5.4 Simple Closure Properties

5.4.1 Closure under AND

Theorem 5.7 [Closure under AND].

$$\mathrm{CDS}(f \wedge g) \leq \mathrm{CDS}(f) + \mathrm{CDS}(g).$$

Proof

Assume the secret space is a finite abelian group G. Sample $S_1 \xleftarrow{R} G$ and set $S_2 = s - S_1$. Run the two CDS protocols with independent randomness and secrets S_1, S_2. If both predicates accept, the decoder reconstructs $S_1 + S_2 = s$. If, for example, $f(x, y) = 0$, the first transcript is independent of S_1. Since S_1 is uniform, the joint transcript is independent of s even after conditioning on the second transcript. The case $g(x, y) = 0$ is symmetric. □

5.4.2 Closure under OR

Theorem 5.8 [Closure under OR].

$$\mathrm{CDS}(f \vee g) \leq \mathrm{CDS}(f) + \mathrm{CDS}(g).$$

Proof

Run the CDS protocols for f and g with the same secret s and independent randomness. If either predicate accepts, the corresponding decoder reconstructs s. If both reject, each transcript distribution is independent of s; independence of the protocol randomness implies that their joint distribution is also independent of s. □

5.5 Formula-Based CDS

The size of a Boolean formula is the number of its leaves.

Theorem 5.9 [GIKRM00]. Every Boolean formula F has a linear CDS with total message complexity

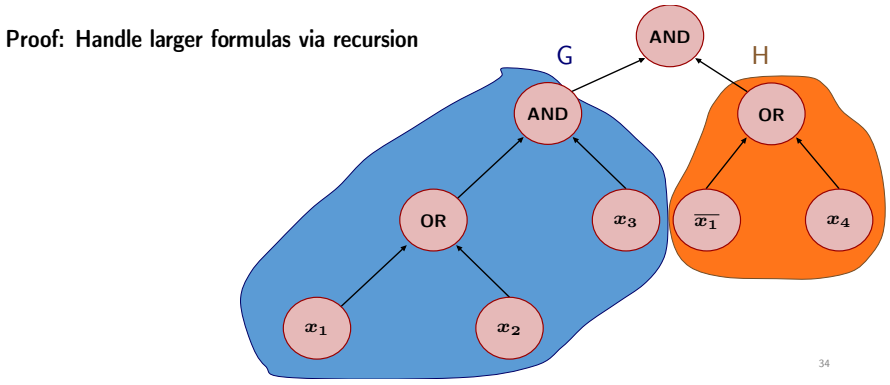
$$\mathrm{Size}(F).$$

Proof

For a size-one formula, use its one-input CDS. For a larger formula, write its root as an AND or OR of subformulas G and H. By induction, construct CDS protocols for G and H with complexities $\mathrm{Size}(G)$ and $\mathrm{Size}(H)$. Combine them using Theorem 5.7 or Theorem 5.8. The resulting complexity is

$$\mathrm{Size}(G) + \mathrm{Size}(H) = \mathrm{Size}(F).$$

□



Corollary 5.10. For every

$$f : \{0,1\}^n \times \{0,1\}^n \longrightarrow \{0,1\},$$
$$\text{Lin-CDS}(f) = O(n2^{2n}).$$

This follows by expressing a Boolean function on $2n$ variables in disjunctive normal form and applying Theorem 5.9.

5.6 CDS for General Predicates

Let $N = 2^n$. The truth table of a predicate $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$ has N rows and N columns.

5.6.1 Completeness of the Index Function

Define

$$\text{Index} : [N] \times \{0,1\}^N \longrightarrow \{0,1\}, \qquad \text{Index}(x, DB) = DB[x].$$

For every $y \in \{0,1\}^n$, define $DB_y[z] = f(z,y)$ for $z \in \{0,1\}^n$. Then $f(x,y) = \text{Index}(x, DB_y)$. Consequently, any CDS for Index gives a CDS for f without increasing its communication complexity, and

$$\text{CDS}(f) \leq \text{CDS}(\text{Index}).$$

5.6.2 Linear CDS for Index

Protocol (Primal linear CDS for Index)

Sample $r = (r_1, \ldots, r_N) \xleftarrow{R} \mathbb{F}_2^N$. The first sender transmits

$$m_A = s + r_x,$$

and the second sender transmits

$$m_B = (r_i : DB[i] = 1).$$

If $DB[x] = 1$, the decoder extracts r_x from m_B and outputs $m_A + r_x$; otherwise it outputs $\perp$.

If $DB[x] = 1$, Charlie receives r_x and recovers s. If $DB[x] = 0$, the missing key r_x perfectly masks the secret. The communication is one bit from Alice and at most N bits from Bob.

Protocol (Dual linear CDS for Index)

Sample $r \xleftarrow{R} \mathbb{F}_2^N$ and transmit

$$m_A = (r_i : i \neq x), \qquad m_B = s + \bigoplus_{i:DB[i]=0} r_i.$$

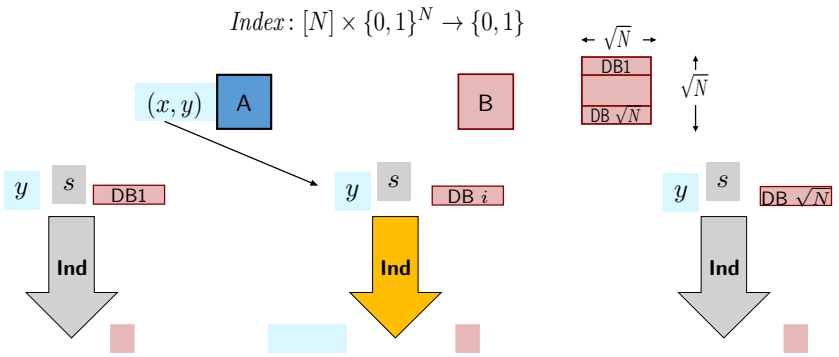
If $DB[x] = 1$, the decoder removes from m_B every mask indexed by a zero entry and outputs the result; otherwise it outputs $\perp$.

When $DB[x] = 0$, the mask r_x occurs in m_B but not in m_A, so the transcript is independent of s.

5.6.3 Balancing

Theorem 5.11 [Balanced linear CDS for Index]. Assume that N is a perfect square. Applying the primal and dual Index constructions to the two coordinates of the identification $[N] \cong [\sqrt{N}]^2$ yields a linear CDS for Index with communication

$$O(\sqrt{N}) = O(2^{n/2}).$$



5.7 Subexponential CDS

[Liu–Vaikuntanathan–Wee 18]

5.7.1 Construction from a Linear PIR Identity

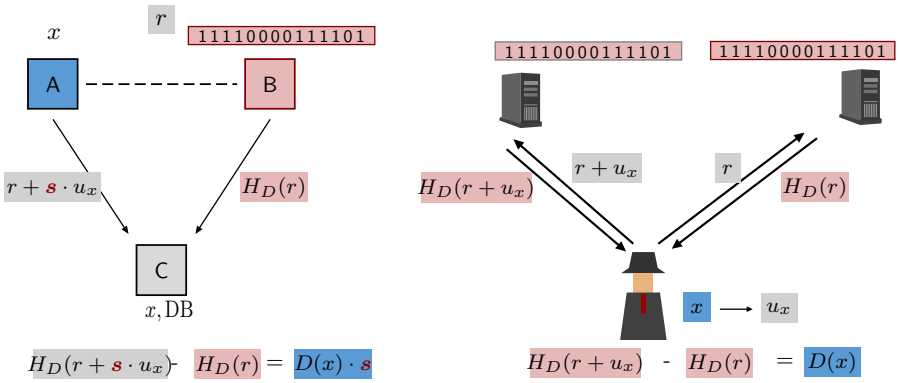
Let G be a finite abelian group. Assume that for every database D there is a homomorphism $H_D : G^m \to G$ and that every index x has an associated $u_x \in G^m$ satisfying, for every $r \in G^m$,

$$H_D(r) - H_D(r + u_x) = D[x].$$

For a scalar secret s, replace u_x by su_x. By linearity,

$$H_D(r) - H_D(r + s \cdot u_x) = D[x] \cdot s.$$

The decoder therefore obtains $D[x]s$. This identity gives a CDS whenever the two induced query marginals are independent of x and, on inputs with $D[x] = 0$, their joint distribution is independent of s.



Theorem 5.12 [Liu–Vaikuntanathan–Wee 17]. Every

$$f : \{0,1\}^n \times \{0,1\}^n \longrightarrow \{0,1\}$$

has a CDS for one-bit secrets with communication

$$2^{o(n)}.$$

5.8 Amortized CDS for General Functions

[Applebaum–Arkov–Raykov–Vasudevan 17, Applebaum–Arkov 18]

Definition 5.13 [Amortized CDS complexity]. Let $\mathrm{CDS}(f, L)$ denote the CDS communication for an L-bit secret per party. Correctness requires communicating at least the L bits that are released on accepting inputs, while independent repetition of a one-bit protocol gives the upper bound

$$L \le \mathrm{CDS}(f, L) \le L \cdot \mathrm{CDS}(f, 1).$$

Define the asymptotic overhead by

$$\overline{\mathrm{CDS}}(f) = \liminf_{L \to \infty} \frac{\mathrm{CDS}(f, L)}{L}.$$

Theorem 5.14 [Constant-rate CDS]. For every $f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$,

$$\overline{\mathrm{CDS}}(f) \le 3.$$

Definition 5.15 [Batch CDS]. Fix predicates $f_1, ldots, f_T : X \times Y \to \{0,1\}$ and a secret space $\mathcal{S}$. A batch CDS consists of two message algorithms and a decoder. For every input pair (x, y) and secret vector $\boldsymbol{s} = (s_1, ldots, s_T)$, correctness requires recovery of $(s_i)_{i:f_i(x,y)=1}$. Privacy requires that, for every two secret vectors $\boldsymbol{s}, \boldsymbol{s}'$ agreeing on all accepting coordinates, the two joint transcript distributions are identical.

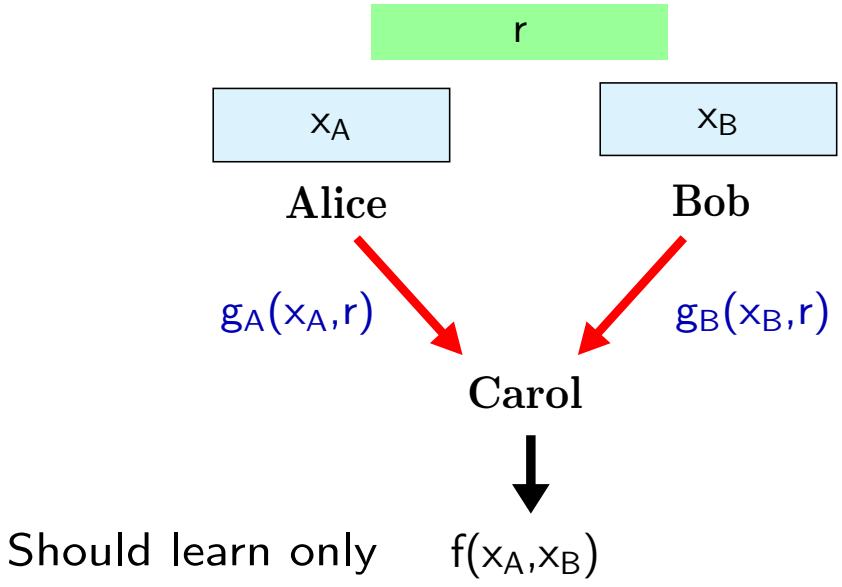
Theorem 5.16 [Batch-CDS for all functions]. There is a Batch-CDS for all functions with blow-up 1.5.

6 Private Simultaneous Messages and Randomized Encodings

6.1 Private Simultaneous Message Protocols

Definition 6.1 [Private Simultaneous Messages]. A two-party PSM for $f : X_A \times X_B \to Y$ consists of a finite shared randomness space $\mathcal{R}$, message spaces $\mathcal{M}_A, \mathcal{M}_B$, maps $g_A : X_A \times \mathcal{R} \to \mathcal{M}_A$ and $g_B : X_B \times \mathcal{R} \to \mathcal{M}_B$, a decoder $\mathrm{Dec} : \mathcal{M}_A \times \mathcal{M}_B \to Y$, and a randomized simulator $\mathrm{Sim} : Y \to \mathcal{M}_A \times \mathcal{M}_B$ satisfying:

1. *Perfect correctness:* for every x_A, x_B, r,
$$\mathrm{Dec}\big(g_A(x_A; r), g_B(x_B; r)\big) = f(x_A, x_B).$$
2. *Perfect privacy:* for every x_A, x_B and $R \xleftarrow{R} \mathcal{R}$,
$$\mathrm{Sim}(f(x_A, x_B)) \equiv \big(g_A(x_A; R), g_B(x_B; R)\big).$$



6.1.1 Example: XOR

For
$$f(x_A, x_B) = x_A \oplus x_B, \qquad x_A, x_B \in \{0, 1\},$$
use the following protocol.

Protocol (PSM for XOR)

Sample $r \xleftarrow{R} \mathbb{F}_2$ and transmit

$$m_A = x_A \oplus r, \qquad m_B = x_B \oplus r.$$

The decoder outputs $m_A \oplus m_B$.

Correctness is immediate. Conditioned on the output bit, the transcript is uniform over the two pairs having that XOR, which gives a perfect simulator.

6.2 PSM versus CDS

Theorem 6.2 [PSM implies CDS].

$$\mathrm{CDS}(f) \le \mathrm{PSM}(g),$$

where

$$g(x,(y,s)) = \begin{cases} s, & f(x,y) = 1, \\ \perp, & f(x,y) = 0. \end{cases}$$

Proof

Apply a PSM protocol satisfying Definition 6.1 to Alice's input x and Bob's input (y,s). Correctness releases s exactly when $f(x,y) = 1$, as required by Definition 5.1. When $f(x,y) = 0$, the PSM privacy simulator depends only on the output $\perp$, so the transcript is independent of s. □

6.3 PSM for Every Function [FKN94]

Let

$$f : \{0,1\}^n \times \{0,1\}^n \longrightarrow \{0,1\}, \qquad N = 2^n.$$

Protocol (Universal PSM)

Choose a uniform mask $r \in \{0,1\}^N$ and a uniform cyclic shift $s \in [N]$. The first sender transmits

$$M_A = (f(x_A,\cdot) \oplus r) \ll s.$$

The second sender transmits

$$x'_B = x_B + s \qquad \text{and} \qquad r[x'_B].$$

Theorem 6.3 [FKN universal PSM]. The construction is a PSM for every f with communication

$$N + \log N + 1 = 2^n + n + 1.$$

Proof

The decoder reads the coordinate x'_B of Alice's shifted row and XORs it with Bob's mask bit. The result is $f(x_A, x_B)$.
For privacy, given only $w = f(x_A, x_B)$, the simulator samples x'_B uniformly, samples the N-bit string M_A uniformly subject to one constraint, and sets the final bit so that

$$M_A[x'_B] \oplus r[x'_B] = w.$$

The real transcript has exactly this distribution: the shift makes x'_B uniform and the one-time pad makes every other coordinate uniform. □

6.3.1 Three-Party PSM from a Three-Dimensional Truth Table

Theorem 6.4 [HW 3, Exercise 4]. Every function

$$f : \{0,1\}^{n/3} \times \{0,1\}^{n/3} \times \{0,1\}^{n/3} \longrightarrow \{0,1\}$$

has a three-party PSM with communication

$$O(2^{2n/3}) < 2^{0.9n}.$$

Proof

Let $x, y, z \in \{0,1\}^{n/3}$ and $N = 2^{n/3}$. For every fixed x, define the $N \times N$ truth table

$$T_x[i,j] = f(x,i,j).$$

The shared randomness is

$$R \xleftarrow{R} \{0,1\}^{N \times N}, \qquad q \xleftarrow{R} \{0,1\}^{N}, \qquad s,t \xleftarrow{R} [N].$$

Party P_1 sends

$$M_1 = T_x \oplus R \ll s,t,$$

meaning that the masked table is shifted by s in the y coordinate and by t in the z coordinate.
Party P_2 sends

$$y' = y + s \qquad \text{and} \qquad M_2 = R_{y,\cdot} \oplus q \ll t.$$

Party P_3 sends

$$z' = z + t \qquad \text{and} \qquad M_3 = q[z'].$$

The referee outputs

$$M_1[y', z'] \oplus M_2[z'] \oplus M_3.$$

For correctness, since $y' = y + s$ and $z' = z + t$,

$$\begin{aligned} M_1[y', z'] &= f(x,y,z) \oplus R[y,z], \\ M_2[z'] &= R[y,z] \oplus q[z], \\ M_3 &= q[z]. \end{aligned}$$

Their XOR is $f(x, y, z)$.
For privacy, let $w = f(x, y, z)$. The simulator samples uniformly

$$y', z' \in [N], \qquad M_2 \in \{0,1\}^N, \qquad M_3 \in \{0,1\},$$

and samples $M_1 \in \{0,1\}^{N \times N}$ uniformly except for the single coordinate

$$M_1[y', z'] = w \oplus M_2[z'] \oplus M_3.$$

The masks R, q and shifts s, t make the real view uniform subject to this same equation. Hence

$$\mathrm{Sim}(f(x, y, z)) \equiv (M_1, y', M_2, z', M_3).$$

The messages have lengths N^2, $N + \log N$, and $\log N + 1$, respectively. Thus

$$\mathrm{comm} = 2^{2n/3} + 2^{n/3} + O(n) = O(2^{2n/3}) < 2^{0.9n}.$$

□

6.4 An AND PSM from Basic Principles

Protocol (PSM for multiplication)

For $f(x_A, x_B) = x_A x_B$ over a finite field, sample independent uniform a, b, r and define

$$A = x_A + a, \qquad B = x_B + b,$$
$$C = x_A b + r, \qquad D = x_B a + ab - r.$$

The first sender transmits (A, C), the second sends (B, D), and the decoder outputs

$$AB - (C + D) = x_A x_B.$$

6.5 Randomized Encoding of Functions

Definition 6.5 [Randomized encoding]. A randomized function

$$g : X \times R \longrightarrow Z$$

is a randomized encoding of $f : X \to Y$ if there exist a decoder Dec and a simulator Sim satisfying:

$$\mathrm{Dec}(g(x, r)) = f(x),$$
$$\mathrm{Sim}(f(x)) \equiv g(x, r)$$

for every x and uniform r.

Definition 6.6 [Two-decomposable randomized encoding]. A randomized encoding of $f(x, y)$ is 2-decomposable if

$$g((x, y), r) = \big(g_x(x, r), g_y(y, r)\big).$$

A two-party PSM as in Definition 6.1 is exactly a 2-decomposable randomized encoding.

6.5.1 Useful Properties

- Composition: $\mathrm{Enc}(\mathrm{Enc}(f))$ is an encoding of f.
- Concatenation: $(\mathrm{Enc}(f_1), \mathrm{Enc}(f_2))$ is an encoding of $f = (f_1, f_2)$.
- Substitution: $\mathrm{Enc}(f) \circ h$ encodes $f \circ h$.

6.5.2 Concatenation with Independent Randomness

Theorem 6.7 [HW 3, Exercise 5]. Let $f_1, f_2 : X \to Y$, and suppose

$$g_1 : X \times R_1 \to Z_1, \qquad g_2 : X \times R_2 \to Z_2$$

encode f_1 and f_2, respectively. Then

$$g(x; (r_1, r_2)) = \big(g_1(x; r_1), g_2(x; r_2)\big)$$

with independent r_1, r_2 encodes

$$f(x) = \big(f_1(x), f_2(x)\big).$$

Proof

Let (S_1, D_1) and (S_2, D_2) be simulators and decoders for the two encodings. Define

$$D(z_1, z_2) = (D_1(z_1), D_2(z_2)),$$
$$S(y_1, y_2) = (S_1(y_1), S_2(y_2)),$$

where the two simulations use independent randomness.
Correctness follows from

$$\begin{aligned} D(g(x; (r_1, r_2))) &= \big(D_1(g_1(x; r_1)), D_2(g_2(x; r_2))\big) \\ &= \big(f_1(x), f_2(x)\big) = f(x). \end{aligned}$$

For independent uniform r_1, r_2,

$$\begin{aligned} g(x; (r_1, r_2)) &= \big(g_1(x; r_1), g_2(x; r_2)\big) \\ &\equiv \big(S_1(f_1(x)), S_2(f_2(x))\big) = S(f(x)). \end{aligned}$$

□

Claim 6.8 [Reusing randomness can destroy privacy]. If $R_1 = R_2$ and

$$g'(x; r) = \big(g_1(x; r), g_2(x; r)\big),$$

then g' need not encode (f_1, f_2).

Proof

Let $X = R_1 = R_2 = \{0,1\}$ and let

$$f_1(x) = f_2(x) = 0.$$

Define

$$g_1(x;r) = r, \qquad g_2(x;r) = x + r,$$

with addition over $\mathbb{F}_2$. Each encoding is individually correct with the constant decoder $D(z) = 0$, and each is private because its output is uniform. However,

$$g'(x;r) = (r, x + r).$$

For $x = 0$ this is (r,r), while for $x = 1$ it is $(r, 1-r)$. Since $f(0) = f(1) = (0,0)$, a simulator receiving only $f(x)$ would have to produce both distinct distributions, which is impossible. □

6.6 PSM Complexity Bounds

Theorem 6.9. Every

$$f : \{0,1\}^n \times \{0,1\}^n \longrightarrow \{0,1\}$$

has a two-party PSM of complexity $O(2^{n/2})$.

6.7 Yao's Garbled-Circuit Construction

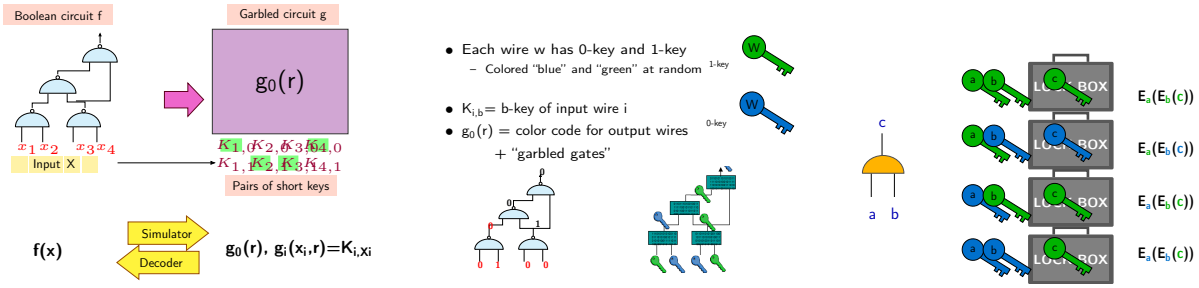
Each wire w has a 0-key and a 1-key, colored blue and green at random. The online encoding of input bit x_i is the key K_{i,x_i}. The offline part $g_0(r)$ contains the color code for output wires and the garbled gates. The decoder evaluates the garbled circuit, while the simulator generates a view from the circuit output alone.

For a gate with input-wire keys a, b and output key c, use nested one-time pads $E_a(E_b(c))$.

Theorem 6.10 [Security of formula garbling]. For a Boolean formula, nested one-time-pad garbling is a perfect randomized encoding of the formula output.

Proof

Correctness is maintained by the wire-label invariant: the decoder holds exactly one label for each evaluated wire, and the garbled gate maps the two input labels to the label representing the correct output bit. For privacy, the simulator chooses random labels only along the evaluated path and fills all unevaluated ciphertext entries with uniform strings. With one-time pads, these strings have exactly the real distribution. Formula fan-out prevents a label from being reused across independent branches. □



6.8 Encoding a Function Family

Definition 6.11 [Decomposable affine randomized encoding]. A DARE for a circuit family $\mathcal{C}$ consists of randomized algorithms $\mathrm{Off}, \mathrm{On}_1, \ldots, \mathrm{On}_n$, a decoder Dec, and a simulator Sim. On randomness R, the encoding of (C, X) is

$$(\mathrm{Off}(C; R), \mathrm{On}_1(C, X_1; R), \ldots, \mathrm{On}_n(C, X_n; R)) .$$

Each online component has the form

$$\mathrm{On}_i(C, X_i; R) = X_i a_i(C, R) + b_i(C, R).$$

Correctness requires the decoder to output $C(X)$. Privacy requires the encoding distribution to be simulatable from $(C, C(X))$ alone.

6.9 Perfect DARE for Formulas

6.9.1 DARE for Simple Functions

Addition is encoded by

$$x_1 + r \mid x_2 - r, \qquad r \xleftarrow{R} \mathbb{F}.$$

Multiplication is encoded by

$$x_1 + r_1 \mid x_2 + r_2 \mid r_2 x_1 + r_1 r_2 + r_3 \mid r_1 x_2 - r_3.$$

Indeed, subtracting the last two components from the product of the first two recovers $x_1 x_2$.

The function $x_1 x_2 + x_3$ is handled by an affinization gadget that replaces non-affine intermediate expressions by additional affine input components and an offline part.

6.9.2 Encoding the Circuit Layer by Layer

Write a layered circuit as

$$C = C_d \circ \cdots \circ C_1.$$

Start with the identity encoding of the output layer. Suppose there is a DARE for $C_1 \circ \cdots \circ C_{i-1}$. Substitute the outputs of layer C_i into this encoding and apply the affinization gadget to obtain affine encodings of the new layer inputs. Repeating this process encodes the full circuit.

The resulting key length is exponential in circuit depth, the output degree is at most three, and the construction applies to logarithmic-depth arithmetic circuits over finite fields.

Theorem 6.12 [Perfect DARE for formulas]. Every formula, equivalently every logarithmic-depth circuit after formula expansion, has a perfect decomposable affine randomized encoding. The keys may be long.

6.10 Computational DARE for General Circuits

For a general circuit, perform substitution and affinization at every layer, then apply key shrinking so that affine keys do not grow with depth.

6.10.1 Key-Shrinking Gadget

The gadget encodes a long affine function

$$yc + d$$

by a short affine function

$$ya + b$$

together with an offline string W. The values a, b, W may depend on c, d and the randomness. A decoder recovers the original affine value, and a simulator generates the encoding from that value alone.

In the Boolean case, if $y = 0$ the encoding should release d, while if $y = 1$ it should release $c + d$. The offline information contains encryptions

$$E_b(d) \qquad \text{and} \qquad E_{a+b}(c + d).$$

Theorem 6.13 [DARE for Boolean circuits from one-way functions]. Assuming one-way functions, every Boolean circuit has a computationally private DARE. The online complexity is $n\lambda$, independent of $|C|$.

PART V

Secure Computation

7 Secure Two-Party Computation over an OT Channel

7.1 Two-Party Interactive Protocols

Let $f : X_A \times X_B \to Y$. The first party holds $x_A \in X_A$ and private randomness R_A; the second holds $x_B \in X_B$ and private randomness R_B. A t-round protocol is specified by message maps of the form

$$\begin{aligned} a_1 &= A_1(x_A, R_A), \\ b_1 &= B_1(x_B, R_B, a_1), \\ &\vdots \\ a_t &= A_t(x_A, R_A, b_1, \ldots, b_t), \end{aligned}$$

and Bob outputs

$$\mathrm{Dec}(x_B, R_B, a_1, \ldots, a_t).$$

Definition 7.1 [Correctness]. The protocol is correct if Bob always outputs

$$f(x_A, x_B).$$

Definition 7.2 [Bob's view and privacy]. Bob's view is everything Bob sees:

$$\mathrm{View}_B(x_A, x_B) = (x_B, R_B, a_1, \ldots, a_t),$$

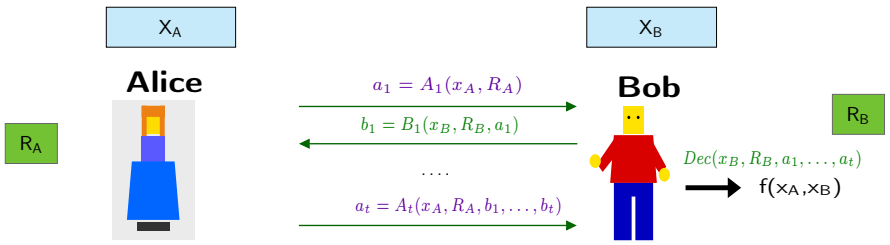
a random variable induced by (R_A, R_B). Privacy against Bob requires a simulator S_B such that for every x_A, x_B,

$$\mathrm{View}_B(x_A, x_B) \equiv S_B(x_B, f(x_A, x_B)).$$

Definition 7.3 [Alice's view and privacy]. Privacy against Alice requires a simulator S_A such that for every x_A, x_B,

$$\mathrm{View}_A(x_A, x_B) \equiv S_A(x_A, \perp).$$

Alice should not learn the output. Both privacy definitions can be relaxed to computational indistinguishability.



7.1.1 Example: XOR

Claim 7.4 [Plain-model protocol for XOR]. For

$$f(x_A, x_B) = x_A \oplus x_B,$$

there is a one-message perfectly private protocol in which only Bob receives the output.

Proof

Given the output and his own input, Bob knows Alice's input exactly. Alice sends

$$a_1 = x_A,$$

and Bob outputs

$$\mathrm{Dec}(x_B, a_1) = x_B \oplus a_1.$$

Bob's message reveals no more than is already determined by his input and output, and Alice receives no messages. □

7.2 The Impossibility of AND in the Plain Model

For

$$f(x_A, x_B) = x_A \wedge x_B,$$

Bob should not learn Alice's value when his own input is 0.

In one round, correctness for Bob's input 1 requires the message to distinguish $x_A = 0$ from $x_A = 1$. Bob can use the same distinction when his actual input is 0, violating privacy.

Theorem 7.5 [HW 4, Exercise 1]. It is impossible to privately compute, with passive security in the plain model, the function that takes a bit x_A from Alice and a bit x_B from Bob and delivers $x_A \wedge x_B$ to Bob.

Proof

Assume toward contradiction that there is a passive secure protocol for

$$f(x_A, x_B) = x_A \wedge x_B$$

where only Bob receives the output.
From privacy against Alice, since Alice gets no output,

$$\mathrm{View}_A(x_A, 0) \equiv \mathrm{View}_A(x_A, 1)$$

for every x_A.
From privacy against Bob, if $x_B = 0$, then the output is always 0. Therefore Bob must not learn x_A, so

$$\mathrm{View}_B(0, 0) \equiv \mathrm{View}_B(1, 0).$$

We prove by induction on the number of messages that also

$$\mathrm{View}_B(0, 1) \equiv \mathrm{View}_B(1, 1).$$

Base case. For the first Alice message,

$$a_1 = A_1(x_A, R_A).$$

This message does not depend on x_B, because Bob has not sent anything yet. So even if Bob's real input is 0, he receives the same kind of Alice message as he would receive if his input were 1.

Therefore, if this message lets Bob decode x_A when using $x_B = 1$, he can run the same decoding locally even when his real input is 0.
The induction uses the following invariant:

$$\mathrm{Dec}(1, R_B, a_1) = x_A$$

would allow Bob with real input 0 to recover x_A, which violates privacy.
Induction step. Assume that up to round i, Bob can reproduce the same distribution of Alice's messages in the execution with $x_B = 1$ as in the execution with $x_B = 0$. Consider Alice's next message:

$$a_{i+1} = A_{i+1}(x_A, R_A, b_1, \ldots, b_i).$$

By privacy against Alice,

$$\mathrm{View}_A(x_A, 0) \equiv \mathrm{View}_A(x_A, 1).$$

So the messages $b_1, \ldots, b_i$ that Alice sees are distributed the same whether Bob's input is 0 or 1. Hence Alice's next message a_{i+1} is also distributed the same in both cases.
Therefore, Bob with input 0 can reproduce the same distribution of Alice messages that he would get with input 1. By induction, this holds for all rounds. Thus,

$$\mathrm{View}_B(0, 1) \equiv \mathrm{View}_B(1, 1).$$

For the contradiction, correctness requires that when $x_B = 1$,

$$f(0, 1) = 0 \qquad \text{but} \qquad f(1, 1) = 1.$$

So Bob must distinguish the executions $(0, 1)$ and $(1, 1)$. But we proved

$$\mathrm{View}_B(0, 1) \equiv \mathrm{View}_B(1, 1),$$

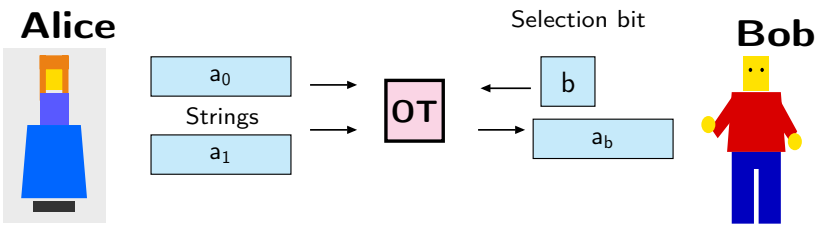
which is impossible. Therefore AND cannot be securely computed in the plain model with passive security. □

7.3 The OT-Hybrid Model

Definition 7.6 [One-out-of-two Oblivious Transfer]. Alice, the sender, inputs strings (a_0, a_1). Bob, the receiver, inputs a selection bit b and learns only

$$a_b.$$

Alice learns nothing about b, and Bob learns nothing about a_{1-b}. Parties may make an arbitrary prescribed number of calls to this ideal functionality.



7.3.1 Computing AND with OT

Theorem 7.7 [AND from one call to OT]. The function $f(x_A, x_B) = x_A \wedge x_B$, with output delivered only to Bob, has a perfectly private protocol in the OT-hybrid model using one OT call.

Proof

Use the following protocol.

Protocol (AND from OT)

The first party inputs

$$(a_0, a_1) = (0, x_A)$$

to OT, and the second party selects $b = x_B$ and outputs the received value.

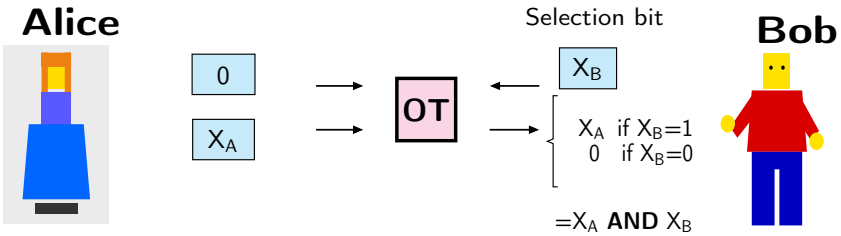
The received value is

$$a_{x_B} = \begin{cases} 0, & x_B = 0, \\ x_A, & x_B = 1, \end{cases} = x_A \wedge x_B.$$

Bob's view consists of his input and the OT answer, so

$$S_B(x_B, Y) = (x_B, Y).$$

Alice's view contains only her input, so $S_A(x_A) = x_A$. □



7.4 Inner Product via OT

Theorem 7.8 [Inner product from parallel OT]. The binary inner product can be computed with perfect privacy using n parallel calls to OT.

Proof

Alice holds $x = (x_1, \ldots, x_n)$ and Bob holds $y = (y_1, \ldots, y_n)$. They wish to compute

$$\langle x, y \rangle = x_1 y_1 + \cdots + x_n y_n \pmod 2.$$

Alice samples $r_1, \ldots, r_{n-1}$ and defines

$$r_n = -\sum_{i=1}^{n-1} r_i.$$

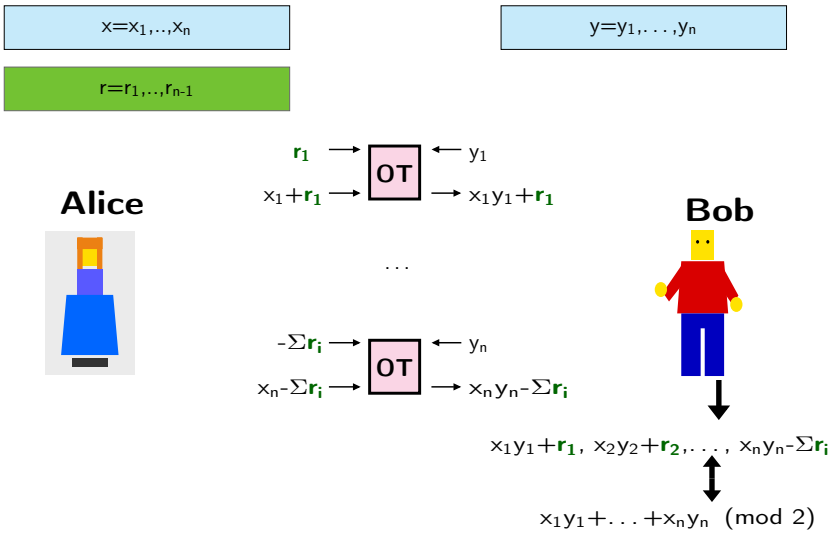
For every i, Alice acts as OT sender with

$$(a_{i,0}, a_{i,1}) = (r_i, x_i + r_i),$$

and Bob selects y_i. Bob receives

$$a_{i,y_i} = x_i y_i + r_i.$$

Summing all received values cancels the r_i and gives the inner product. Each OT hides y_i from Alice and reveals only a_{i,y_i} to Bob. The received values are uniformly masked subject to their sum, so Bob's view is simulatable from y and $\langle x, y \rangle$. □



7.4.1 Every Two-Party Function Reduces to Inner Product

Theorem 7.9 [Reduction to inner product]. Every two-party Boolean function can be expressed as an inner product of values computed locally by Alice and Bob.

Proof

Write f as a multilinear polynomial over the binary field. For example,

$$f(x,y) = x_1x_5y_6y_{10} + x_2x_3x_4y_1y_2 + x_1x_2x_{10} + x_3 + y_1y_2y_{20} + y_2y_{21}.$$

Alice locally defines

$$X_1 = x_1x_5, \quad X_2 = x_2x_3x_4, \quad X_{\text{pure}} = x_1x_2x_{10} + x_3,$$

and Bob locally defines

$$Y_1 = y_6y_{10}, \quad Y_2 = y_1y_2, \quad Y_{\text{pure}} = y_1y_2y_{20} + y_2y_{21}.$$

Then $f(x,y)$ is the inner product of

$$(X_1, X_2, X_{\text{pure}}, 1)$$

with

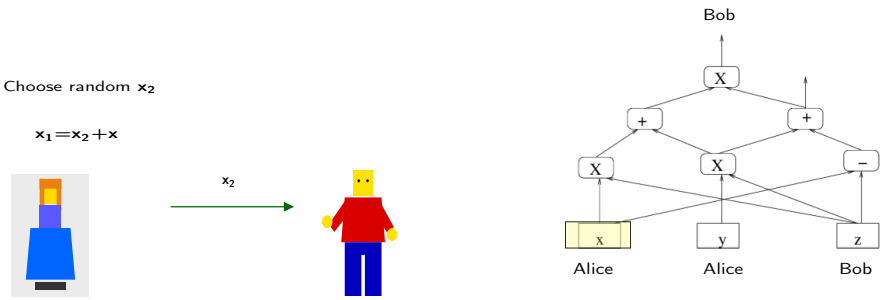
$$(Y_1, Y_2, 1, Y_{\text{pure}}).$$

The communication and computation are proportional to the number of mixed monomials, which may be exponential in the input length. The advantage is a single round of OT calls. □

7.5 The Two-Party GMW Protocol

Represent f by an arithmetic circuit over the binary field. Alice and Bob scan the circuit while maintaining the invariant that for every wire of value v, they hold random shares v_1, v_2 satisfying

$$v_1 + v_2 = v.$$



7.5.1 Input Gates

Claim 7.10 [Input sharing preserves the wire invariant]. For an Alice input x, Alice chooses random x_2, sends it to Bob, and keeps

$$x_1 = x - x_2.$$

For a Bob input y, Bob chooses random y_1, sends it to Alice, and keeps $y_2 = y - y_1$. In both cases the two shares sum to the input value.

7.5.2 Addition and Subtraction Gates

Claim 7.11 [Linear gates are local]. If $w = x - z$, the parties locally compute

$$w_1 = x_1 - z_1, \qquad w_2 = x_2 - z_2.$$

Then

$$w_1 + w_2 = (x_1 - z_1) + (x_2 - z_2) = x - z = w,$$

so the invariant is maintained.

7.5.3 Multiplication Gates

Claim 7.12 [Multiplication gate from two OT calls]. Suppose $x = x_1 + x_2$ and $z = z_1 + z_2$. The desired product is

$$v = (x_1 + x_2)(z_1 + z_2) = x_1z_1 + x_1z_2 + x_2z_1 + x_2z_2.$$

The two cross terms are computed using two OT calls.

Over a finite field $\mathbb{F}$, Alice samples $\alpha \in \mathbb{F}$ and acts as OT sender with

$$M_s = x_1 s - \alpha, \qquad s \in \mathbb{F}.$$

Bob chooses z_2 and receives

$$U = x_1 z_2 - \alpha.$$

Bob samples $\beta \in \mathbb{F}$ and acts as OT sender with

$$N_s = x_2 s - \beta.$$

Alice chooses z_1 and receives

$$V = x_2 z_1 - \beta.$$

They set

$$v_1 = x_1 z_1 + V + \alpha, \qquad v_2 = x_2 z_2 + U + \beta.$$

Then

$$v_1 + v_2 = (x_1 + x_2)(z_1 + z_2).$$

Thus (v_1, v_2) satisfies the wire-sharing invariant.

7.5.4 Output Phase

Claim 7.13 [Output reconstruction]. If the output wire has shares t_1, t_2, Alice sends t_1 to Bob, who outputs

$$t = t_1 + t_2.$$

Theorem 7.14 [Two-party GMW]. The GMW construction gives a two-party protocol for every efficiently computable function with perfect privacy in the OT-hybrid model. Its communication and computation are linear in circuit size. It extends to many parties and arithmetic circuits over larger fields. Its round complexity is the multiplicative depth of the circuit.

Proof

Correctness follows by induction over a topological ordering of the circuit, using the four gate claims above. For privacy, every intermediate wire is represented by a uniform additive sharing conditioned only on its value. The ideal OT calls expose only the selected masked cross terms. A simulator therefore samples all intermediate shares and OT outputs uniformly and fixes only the final share to match the prescribed output. Each multiplication gate uses a constant number of OTs, and gates in the same multiplicative layer run in parallel. □

7.6 From Decomposable Randomized Encodings to Protocols

A decomposable randomized encoding, also called a multiparty PSM, has the form

$$g(x, r) = \big(g_0(r), g_1(x_1, r), \ldots, g_n(x_n, r)\big).$$

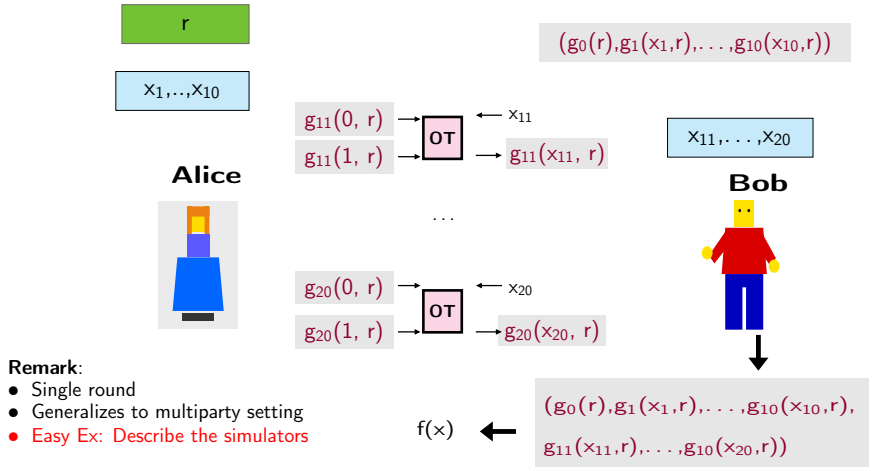
Suppose Alice holds $x_1, \ldots, x_{10}$ and Bob holds $x_{11}, \ldots, x_{20}$. Alice samples r and sends

$$g_0(r), g_1(x_1, r), \ldots, g_{10}(x_{10}, r).$$

For every Bob input bit x_j, Alice acts as OT sender with

$$\big(g_j(0, r), g_j(1, r)\big),$$

and Bob selects x_j. Bob receives all components of the encoding and decodes $f(x)$. The protocol has one round of OT calls, generalizes to the multiparty setting, and permits the offline message $g_0(r)$ to be sent before the inputs are available.



Every size-s formula has a fully decomposable perfect randomized encoding with $\mathrm{poly}(s)$ communication. Assuming PRGs, every size-s circuit has a fully decomposable computational randomized encoding with

$\mathrm{poly}(s)$ communication. Consequently, formulas and circuits have single-round OT-hybrid protocols with the corresponding privacy.

7.7 Constructing OT from Encryption with a Fake Mode

Assume public-key encryption with real and fake public keys satisfying:

- real keys and fake keys are computationally indistinguishable;
- fake keys are lossy:

$$\mathrm{Enc}_{\mathrm{fake}}(0) \equiv \mathrm{Enc}_{\mathrm{fake}}(1).$$

Examples include ElGamal, Goldwasser–Micali, and lattice-based schemes.

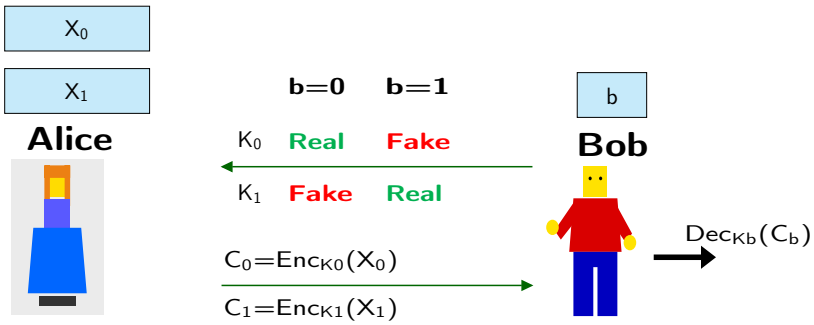
Protocol (OT from real/fake encryption)

To receive X_b, the receiver generates K_b in real mode and K_{1-b} in fake mode and sends both public keys to the sender. The sender returns

$$C_0 = \mathrm{Enc}_{K_0}(X_0), \qquad C_1 = \mathrm{Enc}_{K_1}(X_1).$$

The receiver outputs $\mathrm{Dec}_{K_b}(C_b)$.

Indistinguishability of real and fake keys hides b from Alice, and lossiness of the fake key hides X_{1-b} from Bob. The protocol is secure in the semi-honest setting.



7.7.1 Efficiency for String OT

First use OT to transfer one of two short AES keys. Then encrypt the long strings as

$$C_0 = \mathrm{AES}_{K_0}(X_0), \qquad C_1 = \mathrm{AES}_{K_1}(X_1).$$

Bob receives K_b through OT and decrypts C_b.

7.8 Precomputing OT and GMW

Protocol (Online OT from random OT)

In preprocessing, random OT gives the sender uniform (r_0, r_1) and the receiver a uniform selection bit β together with r_β. In the online phase, the sender holds (m_0, m_1) and the receiver holds b. The receiver sends the correction bit

$$d = b \oplus \beta.$$

If $d = 0$, Alice sends

$$c_0 = m_0 \oplus r_0, \qquad c_1 = m_1 \oplus r_1.$$

If $d = 1$, she sends

$$c_0 = m_0 \oplus r_1, \qquad c_1 = m_1 \oplus r_0.$$

The receiver outputs $c_b \oplus r_\beta = m_b$.

8 Passive Multi-Party Computation in the Plain and OT-Hybrid Models

8.1 The Multi-Party Protocol Model

There are n parties $P_1, \ldots, P_n$. Party P_i holds an input x_i and private randomness r_i. The parties are connected by private point-to-point channels. In the plain model these are the only available channels; in the OT-hybrid model every pair of parties may additionally call an ideal oblivious-transfer functionality.

In round ℓ, party P_j sends a message $M^{\ell}_{j:k}$ to party P_k as a function of its current view. The view of P_j after round ℓ contains

$$x_j, \quad r_j, \quad \left(M^{h}_{k:j} : k \in [n],\ 1 \le h \le \ell\right).$$

Definition 8.1 [Correctness]. Let

$$f(x_1, \ldots, x_n) = (f_1(x_1, \ldots, x_n), \ldots, f_n(x_1, \ldots, x_n)).$$

A protocol is correct if every party P_i outputs

$$y_i = f_i(x_1, \ldots, x_n).$$

Definition 8.2 [Passive t-privacy]. For a coalition $T \subseteq [n]$, let

$$\mathrm{View}_T(x_1, \ldots, x_n)$$

denote the joint view of the parties in T. A protocol is passively t-private if, for every T of size at most t, there is a simulator S_T such that

$$\mathrm{View}_T(x_1, \ldots, x_n) \equiv S_T(x_T, f_T(x_1, \ldots, x_n)).$$

Full privacy means privacy against every proper coalition $T \subsetneq [n]$.

8.1.1 The REAL/IDEAL Paradigm

In the REAL execution, the parties run the protocol and an adversary sees the views of the corrupted parties. In the IDEAL execution, every party sends its input to a trusted party, which computes f and returns the prescribed outputs. Security requires the REAL and IDEAL distributions to be identical in the perfect passive setting, or indistinguishable in the corresponding statistical or computational setting.

Definition 8.3 [Perfect passive security]. A protocol securely computes a deterministic functionality f against a passive adversary if, for every corrupted coalition T, the REAL execution is identically distributed to an IDEAL execution generated from $(x_T, f_T(x))$.

For deterministic functionalities, this formulation is equivalent to the view-simulation definition above.

8.2 Privately Computing the Sum

Let the inputs lie in a finite field $\mathbb{F}$ and let

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{n} x_i.$$

8.2.1 The Complete-Graph Protocol

Protocol (Private sum on the complete graph)

1. For every ordered pair (i, j), party P_i samples $r_{i,j} \xleftarrow{R} \mathbb{F}$ and privately sends it to P_j.
2. Party P_i broadcasts
$$c_i = x_i + \sum_{j=1}^{n} r_{j,i} - \sum_{j=1}^{n} r_{i,j}.$$
3. Every party outputs $\sum_{i=1}^{n} c_i$.

Theorem 8.4 [Correctness and privacy of the sum protocol]. The complete-graph sum protocol is correct and perfectly private against every proper passive coalition.

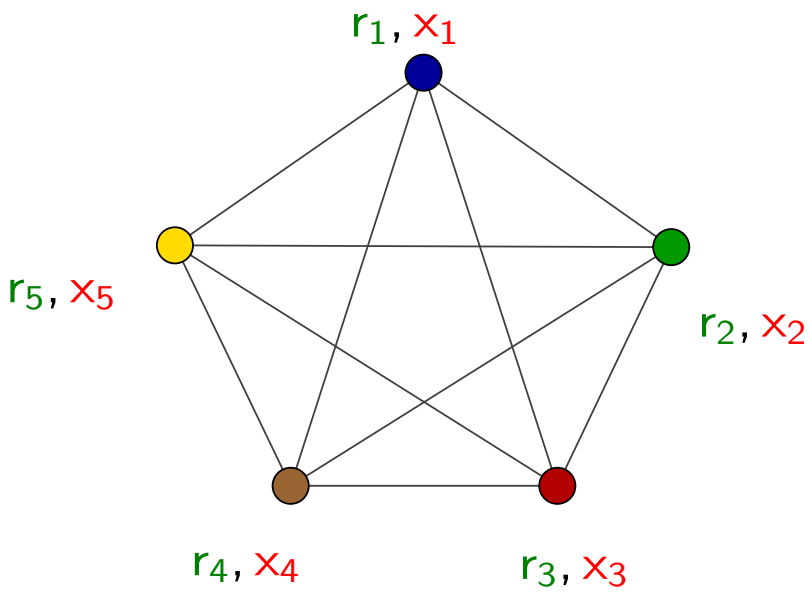
Proof

Correctness follows because every mask occurs once with a plus sign and once with a minus sign:

$$\begin{aligned}\sum_{i=1}^{n} c_i &= \sum_{i=1}^{n} x_i + \sum_{i=1}^{n}\sum_{j=1}^{n} r_{j,i} - \sum_{i=1}^{n}\sum_{j=1}^{n} r_{i,j} \\ &= \sum_{i=1}^{n} x_i.\end{aligned}$$

Fix a proper coalition T. The simulator samples all randomness held or received by parties in T exactly as in the protocol and computes their broadcasts. It then samples all but one of the honest broadcasts uniformly and chooses the final honest broadcast so that the sum of all broadcasts is the prescribed output. This is the real distribution, since the masks on edges between honest parties make the honest broadcasts uniform subject only to their required sum. □

The same protocol privately computes every linear function. It requires no oblivious transfer.



8.2.2 The Sum Protocol on an Arbitrary Graph

Let $G = ([n], E)$ be an undirected graph, and let $N(i)$ denote the neighbors of i. Party P_i sends a uniform $r_{i,j}$ to every P_j with $j \in N(i)$ and then broadcasts

$$c_i = x_i + \sum_{j \in N(i)} r_{j,i} - \sum_{j \in N(i)} r_{i,j}.$$

Definition 8.5 [T-robustness]. For a corrupted coalition $T \subseteq [n]$, write

$$\overline{T} = [n] \setminus T.$$

The graph G is T-robust if the induced graph $G[\overline{T}]$ is connected.

Theorem 8.6 [HW 4, Exercise 3]. The graph-based sum protocol is privately simulatable against the coalition T if and only if G is T-robust.

Proof

Correctness holds for every undirected graph, since

$$\sum_{i=1}^{n} \sum_{j \in N(i)} r_{j,i} = \sum_{i=1}^{n} \sum_{j \in N(i)} r_{i,j}.$$

Suppose first that $G[\overline{T}]$ is connected. The simulator samples all random values on edges incident to T and computes the broadcasts of the corrupted parties. Choose a spanning tree of $G[\overline{T}]$. The masks on its edges allow the honest broadcast vector to range uniformly over all vectors having the required sum. Consequently, the simulator samples all but one honest broadcast uniformly and fixes the last one using the public sum. This produces the real distribution.

Conversely, suppose $G[\overline{T}]$ is disconnected. Let A be one connected component and let $B = \overline{T} \setminus A$. Define

$$\delta_A = \sum_{\substack{i \in A \\ j \in N(i) \cap T}} (r_{j,i} - r_{i,j}).$$

Because no edge joins A and B, all masks internal to A cancel, and the coalition can compute

$$\sum_{i \in A} c_i - \delta_A = \sum_{i \in A} x_i.$$

Choose $\widetilde{i} \in A$ and $\widetilde{j} \in B$. The two input vectors $e_{\widetilde{i}}$ and $e_{\widetilde{j}}$ have the same corrupted inputs and the same global sum 1, but the displayed partial sum is 1 in the first execution and 0 in the second. Hence no simulator receiving only the corrupted inputs and the global output can generate both views. □

Corollary 8.7. The protocol is private against every coalition of size at most t exactly when deleting any t vertices leaves the graph connected.

8.2.3 Rushing Adversaries

Definition 8.8 [Rushing adversary]. A rushing adversary may wait within a round, observe the honest parties' messages from that round, and only then choose the corrupted parties' messages for the same round.

Theorem 8.9 [HW 4, Exercise 4]. The sum protocol is not secure against an active rushing adversary, even if only P_1 is corrupted.

Proof

The corrupted party may behave honestly in the first round. In the broadcast round it waits to observe $c_2, \ldots, c_n$ and then broadcasts

$$\widehat{c}_1 = -\sum_{i=2}^{n} c_i.$$

The reconstructed result is therefore

$$\widehat{c}_1 + \sum_{i=2}^{n} c_i = 0,$$

independently of the honest inputs.
This behavior cannot be simulated in the ideal execution. Consider two honest input vectors whose sums differ by 1 while the corrupted party's input and its view before the output are the same. In the real execution the adversary forces output 0 in both cases. In the ideal execution a single effective input for P_1 cannot make both distinct honest sums evaluate to 0. Thus the REAL and IDEAL executions cannot agree. □

8.3 Completeness of Functionalities

Definition 8.10 [Complete functionality]. A functionality is complete if access to it, together with private communication and local computation, suffices to securely realize oblivious transfer and hence every efficiently computable functionality.

Theorem 8.11 [Exam A 2025, Question 4(a)]. The two-party functionality

$$g(x_1, x_2) = (x_1 \vee x_2, \bot)$$

is complete.

Proof

Let the receiver hold the OT selection bit s, and let the sender hold messages m_0, m_1. Make two calls in which the receiver supplies the first input:

$$g(s, m_0) \qquad \text{and} \qquad g(1-s, m_1).$$

The receiver obtains $s \vee m_0$ and $(1-s) \vee m_1$. If $s = 0$, the first value is m_0 and the second is 1; if $s = 1$, the first is 1 and the second is m_1. Thus the receiver learns exactly m_s, while the sender receives no output and learns nothing about s. □

Theorem 8.12 [Exam A 2025, Question 4(b)]. The public-output functionality

$$g(x_1, (x_2, x_2')) = (x_1 \wedge x_2) \oplus x_2'$$

is complete.

Proof

The second party chooses x_2' uniformly. The public output is a one-time pad of $x_1 \wedge x_2$. The second party knows the pad and recovers the AND, whereas the first party learns nothing about it. Hence g realizes the functionality g' that delivers $x_1 \wedge x_2$ only to the second party.
To implement OT, make two calls

$$g'(m_1, s) \qquad \text{and} \qquad g'(m_0, 1-s).$$

The receiver obtains $m_1 s$ and $m_0(1-s)$ and therefore recovers

$$m_1 s \oplus m_0(1-s) = m_s.$$

The sender learns nothing about s. □

Theorem 8.13 [Exam A 2025, Question 4(c)]. The public three-party functionality

$$g(x_1, x_2, x_3) = x_1 \oplus x_2 \oplus x_3$$

is not complete.

Proof

The functionality is securely realizable in the plain model by the private sum protocol over $\mathbb{F}_2$. If it were complete, composing that plain protocol with a completeness reduction would yield a plain-model protocol for two-party AND. This contradicts the impossibility of privately computing two-party AND in the plain model. The fact that the functionality has three parties is not by itself sufficient: three-party AND is complete. □

8.4 The Multi-Party GMW Protocol in the OT-Hybrid Model

This is the multi-party extension of Theorem 7.14 and uses the OT functionality of Definition 7.6 for multiplication gates.

Represent the desired function by an arithmetic circuit over a finite field $\mathbb{F}$. For every wire carrying v, the parties maintain additive shares

$$v_1 + \cdots + v_n = v.$$

Protocol (Multi-party GMW gate evaluation)

Input sharing and linear gates are local. To multiply shared values $x = \sum_i x_i$ and $z = \sum_j z_j$, expand

$$xz = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i z_j.$$

Each diagonal term $x_i z_i$ is known locally. Every cross term $x_i z_j$ is additively shared between P_i and P_j using oblivious transfer, and the parties sum all resulting shares.

Theorem 8.14 [Multi-party GMW]. In the OT-hybrid model, every arithmetic circuit has a perfectly private multi-party protocol against every proper passive coalition. Its communication and computation are

$$O(\text{circuit-size} \cdot n^2),$$

and its round complexity is the multiplicative depth of the circuit.

8.4.1 Explicit GMW Protocol for $f(w, x, y, z) = wy + xz$

Alice holds w, x, Bob holds y, z, and Alice receives the output.

Theorem 8.15 [HW 4, Exercise 2]. The following OT-hybrid protocol securely computes

$$f(w, x, y, z) = wy + xz.$$

Proof

Alice samples $w_2, x_2 \xleftarrow{R} \mathbb{F}$, sends them to Bob, and keeps

$$w_1 = w - w_2, \qquad x_1 = x - x_2.$$

Bob samples $y_1, z_1 \xleftarrow{R} \mathbb{F}$, sends them to Alice, and keeps

$$y_2 = y - y_1, \qquad z_2 = z - z_1.$$

For shared $a = a_1 + a_2$ and $b = b_1 + b_2$, the parties produce shares of ab as follows. Alice samples $\alpha \in \mathbb{F}$ and is the OT sender with

$$M_s = a_1 s - \alpha, \qquad s \in \mathbb{F}.$$

Bob selects b_2 and receives

$$U = a_1 b_2 - \alpha.$$

Bob samples $\beta \in \mathbb{F}$ and is the OT sender with

$$N_s = a_2 s - \beta, \qquad s \in \mathbb{F}.$$

Alice selects b_1 and receives

$$V = a_2 b_1 - \beta.$$

They define

$$c_1 = a_1 b_1 + V + \alpha, \qquad c_2 = a_2 b_2 + U + \beta.$$

Then

$$\begin{aligned} c_1 + c_2 &= a_1 b_1 + a_2 b_1 - \beta + \alpha + a_2 b_2 + a_1 b_2 - \alpha + \beta \\ &= (a_1 + a_2)(b_1 + b_2) = ab. \end{aligned}$$

Run this multiplication protocol for (w, y) to obtain $p_1 + p_2 = wy$, and independently for (x, z) to obtain $q_1 + q_2 = xz$. Alice sets $t_1 = p_1 + q_1$, Bob sets $t_2 = p_2 + q_2$ and sends t_2 to Alice. Alice outputs

$$t_1 + t_2 = wy + xz.$$

For privacy against Alice, a simulator given (w, x) and $t = wy + xz$ samples

$$w_2, x_2, \alpha_p, \alpha_q, y_1, z_1, V_p, V_q \xleftarrow{R} \mathbb{F}$$

independently. It sets $w_1 = w - w_2$ and $x_1 = x - x_2$, forms Alice's two OT sender tables

$$M_s^p = w_1 s - \alpha_p, \qquad M_s^q = x_1 s - \alpha_q,$$

and treats V_p, V_q as her two reverse-OT answers. It computes

$$\begin{aligned} p_1 &= w_1 y_1 + V_p + \alpha_p, \\ q_1 &= x_1 z_1 + V_q + \alpha_q, \\ t_1 &= p_1 + q_1, \end{aligned}$$

and sets the final message to $t_2 = t - t_1$. Each sampled value has the same uniform distribution as in the real execution, and the sole final constraint is exactly correctness. Thus the simulated and real views are identically distributed. The simulator against Bob is symmetric. □

8.5 The Multi-Party BGW Protocol in the Plain Model

BGW replaces additive sharing and pairwise OT by Shamir secret sharing. Let at most t parties be corrupted and assume

$$n > 2t.$$

A wire value v is shared by choosing a random degree-t polynomial p with $p(0) = v$ and giving $p(i)$ to P_i.

8.5.1 Input and Linear Gates

Protocol (BGW input and linear gates)

An input owner Shamir-shares its input. For addition, subtraction, and multiplication by a public constant, every party performs the corresponding operation locally on its share. The resulting values remain evaluations of a degree-t polynomial whose free term is the desired wire value.

8.5.2 Multiplication and Degree Reduction

Suppose $a_i = p(i)$ and $b_i = q(i)$ are degree-t shares of a and b. Each party locally computes

$$c_i = a_i b_i.$$

These are evaluations of the degree-$2t$ polynomial $p(X)q(X)$. Since $n > 2t$, there are public interpolation coefficients $\alpha_1, \ldots, \alpha_n$ such that

$$ab = \sum_{i=1}^{n} \alpha_i c_i.$$

Protocol (BGW multiplication and degree reduction)

To reduce the degree without revealing ab, party P_i Shamir-shares the value $\alpha_i c_i$ using a fresh degree-t polynomial. Every party adds the shares it receives. The resulting vector is a fresh degree-t sharing of

$$\sum_{i=1}^{n} \alpha_i c_i = ab.$$

This step is called *Recover-and-Reshare*.

Theorem 8.16 [BGW]. If $n > 2t$, every arithmetic circuit can be evaluated in the plain model with perfect passive security against t corruptions. The communication and computation are

$$O(\text{circuit-size} \cdot n^2),$$

and the round complexity is proportional to the multiplicative depth.

The field must contain at least $n + 1$ elements so that the evaluation points $0, 1, \ldots, n$ are distinct.

8.5.3 Hybrid View and Composition

The multiplication step can be viewed as a call to an ideal functionality F_{mul}: it reconstructs shared a, b, computes ab, and returns a fresh random sharing of the product. A protocol securely realizing F_{mul} may replace each ideal call by the composition theorem. This modular view separates the circuit-level proof from the proof of the multiplication subprotocol.

9 Active Multi-Party Computation and Maurer's CNF Construction

9.1 Active Security

An active, or malicious, adversary may arbitrarily deviate from the protocol. It may coordinate all corrupted parties, change their messages, send inconsistent shares, and choose values that do not correspond to any well-defined input.

Definition 9.1 [Active t-security]. Let A be an active adversary corrupting a coalition T of at most t parties. A protocol actively t-securely realizes a functionality f if there exists an ideal-model simulator S_T such that, for every input vector

$$\boldsymbol{x} = (x_1, \dots, x_n),$$

the adversary's real view together with the outputs of the honest parties is identically distributed to the simulator's view and the honest outputs in the ideal execution with a trusted party.

Unlike the passive security of Definition 8.3, the real-model adversary may deviate arbitrarily. In the ideal model, it can essentially only change the corrupted parties' inputs, independently of the honest inputs, and change the corrupted parties' outputs.

9.1.1 Simple Examples

Claim 9.2 [Broadcast]. For

$$f(x_1, \bot, \dots, \bot) = (x_1, \dots, x_1),$$

ordinary broadcast is secure against a corrupted receiver but not against a corrupted sender.

Proof

A corrupted receiver has no secrecy requirement beyond learning x_1. A corrupted sender, however, may send different values to different receivers, whereas the ideal functionality supplies one consistent value to all honest parties. □

Claim 9.3 [Two-party sum]. Consider

$$f(x_1, x_2, \bot, \dots, \bot) = (x_1 + x_2, x_1 + x_2, \bot, \dots, \bot).$$

The direct protocol is simulatable against a corrupted second sender who sends an illegal value, but it is not actively secure against a party that chooses its input after learning the other party's input.

Proof

An illegal transmitted value can be interpreted as a fixed default input, such as zero. By contrast, choosing x_1 as a function of x_2, for example setting $x_1 = x_2$, violates independence of inputs and cannot be reproduced in the ideal model. □

9.2 Information-Theoretic Active MPC in the Plain Model

Theorem 9.4 [Active MPC thresholds]. Every efficiently computable functionality can be realized in the plain model as follows:

- BGW gives perfect active security when
$$t < \frac{n}{3};$$
- Rabin–Ben-Or gives statistical active security when
$$t < \frac{n}{2}.$$

The Rabin–Ben-Or protocol assumes an ideal broadcast channel.

9.2.1 Additional Requirements for Active Security

Starting from the GMW/BGW circuit-evaluation template, an active adversary may attack every interactive step.

- During sharing, it may distribute inconsistent shares. This requires verifiable secret sharing.
- Local addition requires no interaction and therefore introduces no new problem.
- During multiplication, it may introduce shifts or share incorrect products. This requires a secure multiplication protocol.
- During output opening, it may send false shares. This requires robust secret sharing and robust reconstruction.

9.3 The $(4, 1)$-CNF Secret-Sharing Scheme

Let the parties be P_1, P_2, P_3, P_4, and suppose at most one party is corrupted. All values lie in a finite field $\mathbb{F}$.

Definition 9.5 [$(4, 1)$-CNF sharing]. To share $s \in \mathbb{F}$, choose field elements
$$r_{1,2,3}, \quad r_{1,2,4}, \quad r_{1,3,4}, \quad r_{2,3,4}$$
subject to
$$s = r_{1,2,3} + r_{1,3,4} + r_{2,3,4} + r_{1,2,4}.$$
The value $r_{i,j,k}$ is given to exactly the three parties P_i, P_j, P_k. Thus each additive component has three copies.

Explicitly, the four local shares are

$$P_1 : (r_{1,2,3}, r_{1,2,4}, r_{1,3,4}),$$
$$P_2 : (r_{1,2,3}, r_{1,2,4}, r_{2,3,4}),$$
$$P_3 : (r_{1,2,3}, r_{1,3,4}, r_{2,3,4}),$$
$$P_4 : (r_{1,2,4}, r_{1,3,4}, r_{2,3,4}).$$

Claim 9.6 [Privacy]. Every single party has a view that is independent of s.

Proof

Each party is missing exactly one of the four additive components. That missing component is uniform subject to the sum constraint and therefore acts as a one-time pad for s. □

9.3.1 DNF and CNF Share-Size Optimization

Consider four servers (a, b, c, d) and the access structure in which a coalition T is authorized exactly when

$$|T| \geq 3 \qquad \text{or} \qquad (a \in T \text{ and } b \in T).$$

Definition 9.7 [DNF secret sharing]. For every minimal authorized set, independently share the same secret among the members of that set using additive secret sharing.

Claim 9.8 [Exam solution: DNF share count]. The minimal authorized sets are

$$(ab), \qquad (bcd), \qquad (acd).$$

The DNF construction has total share size

$$2 + 3 + 3 = 8$$

field elements.

Proof

Apply a 2-out-of-2 additive sharing to (ab) and independent 3-out-of-3 additive sharings to (bcd) and (acd). Every authorized coalition contains a minimal authorized set and can reconstruct one copy of s. An unauthorized coalition lacks at least one uniform additive share from every copy. □

Definition 9.9 [CNF secret sharing]. For every maximal unauthorized set U, create one additive component r_U. Additively share s as the sum of all these components, and give r_U to every party outside U.

Claim 9.10 [Exam solution: CNF share count]. The maximal unauthorized sets are

$$(bc), \qquad (bd), \qquad (ac), \qquad (ad), \qquad (cd).$$

Choose

$$s = r_{bc} + r_{bd} + r_{ac} + r_{ad} + r_{cd}$$

and give each party the components whose subscript does not contain its name. Every component is stored twice, so the total share size is

$$5 \cdot 2 = 10$$

field elements.

For example, server a stores

$$r_{bc}, \quad r_{bd}, \quad r_{cd}.$$

Theorem 9.11 [A six-element construction]. The same access structure has a secret-sharing scheme with total share size 6.

Proof

Independently use a 2-out-of-2 sharing between a and b and a 3-out-of-4 Shamir sharing among a, b, c, d. The first sharing uses two field elements and realizes the authorized set (ab). The second uses four field elements and realizes every coalition of size at least three. The total is

$$2 + 4 = 6.$$

□

Theorem 9.12 [The single-element server trick]. For every monotone access function, one distinguished server a can store only one field element.

Proof

Let x_1 indicate whether server a belongs to the coalition, and define the restricted access functions

$$f_0(x_2, \ldots, x_n) = f(0, x_2, \ldots, x_n), \qquad f_1(x_2, \ldots, x_n) = f(1, x_2, \ldots, x_n).$$

Choose a uniform $r \in \mathbb{F}$ and store only r at server a. Among the other servers, independently share s according to f_0 and share $s - r$ according to f_1.
If a is absent and the coalition is authorized, then $f_0 = 1$ and the coalition reconstructs s. If a is present and the coalition is authorized, then $f_1 = 1$; it reconstructs $s - r$ and adds the stored value r. An unauthorized coalition without a either learns neither value or only the uniformly masked value $s - r$. An unauthorized coalition containing a satisfies $f_1 = 0$ and, by monotonicity, also $f_0 = 0$, so it cannot reconstruct either sharing. □

For the DNF scheme above, the same optimization can be viewed by letting server a hold random values r_1, r_2 and setting $r_2 = r_1$. This reduces the share size of a to one field element and is a special case of the f_0, f_1 construction.

9.4 Robust Secret Sharing and Reconstruction

Definition 9.13 [t-robust secret sharing]. A secret-sharing scheme is t-robust if, after s has been honestly shared, a t-bounded adversary cannot prevent the correct reconstruction of s.

The adversary's goal is to create ambiguity with nonzero probability. It may be assumed to know all local shares, while the referee does not know which party is corrupted.

Theorem 9.14 [Robust reconstruction of $(4, 1)$-CNF sharing]. The $(4, 1)$-CNF scheme is robust against one corrupted party.

Proof

For every triple (i, j, k), the referee receives three reported copies of $r_{i,j,k}$ and recovers the component by majority vote. At most one copy is corrupted, so the majority value is correct. The referee then outputs

$$s = r_{1,2,3} + r_{1,3,4} + r_{2,3,4} + r_{1,2,4}.$$

□

9.5 Verifiable Secret Sharing for $(4, 1)$-CNF

Theorem 9.14 assumes that the dealer shared the secret honestly. Verifiable secret sharing must additionally handle a corrupted dealer who sends inconsistent copies.

Definition 9.15 [Conflict]. A conflict on $r_{i,j,k}$ occurs when two parties that should hold the same component report different received values for it.

Protocol (Conflict-based $(4, 1)$-VSS verification)

1. *Conflict identification.* For every pair (i, j), parties P_i and P_j check the consistency of all components they both hold. They broadcast *OK* if the values agree; otherwise they broadcast

   $$\text{conflict}, (i, j, k).$$

   The comparison preserves secrecy because both parties already know the tested component. A broadcast channel guarantees that every party hears the same report and can itself be implemented from robust secret sharing.

2. *Dealer resolution.* For each conflict concerning $r_{i,j,k}$, the dealer broadcasts the intended value $r_{i,j,k}$. Revealing this component is safe: a conflict implies that the dealer or one of the comparing parties is corrupted, and in either case the adversary already knows $r_{i,j,k}$.

Theorem 9.16 [Maurer $(4, 1)$-VSS]. Conflict identification followed by dealer resolution realizes the ideal $(4, 1)$ verifiable-secret-sharing functionality against one active corruption.

Proof

If the dealer is honest, every honest recipient receives consistent components; a corrupted recipient can only create a conflict involving a component it already knows, and the dealer's broadcast fixes the value. If the dealer is corrupted, the public conflict resolutions define one consistent additive sharing to which all honest parties agree. Robust majority reconstruction then produces one well-defined value. The only components revealed during verification are already known to the adversary, so privacy is preserved. □

9.6 Active $(4,1)$-MPC

Protocol (Active four-party circuit evaluation)

1. inputs are shared by $(4,1)$-VSS;
2. addition is performed locally and non-interactively;
3. multiplication gates are evaluated by the monomial-sharing procedure specified in the following subsection;
4. outputs are opened by robust majority reconstruction.

9.6.1 Secure Multiplication

Let

$$a = a_{1,2,3} + a_{1,3,4} + a_{2,3,4} + a_{1,2,4},$$
$$b = b_{1,2,3} + b_{1,3,4} + b_{2,3,4} + b_{1,2,4}.$$

The goal is to generate components

$$c_{1,2,3}, c_{1,3,4}, c_{2,3,4}, c_{1,2,4}$$

such that

$$ab = c_{1,2,3} + c_{1,3,4} + c_{2,3,4} + c_{1,2,4}.$$

Expanding the product gives

$$ab = \sum_{I,J \in \binom{[4]}{3}} a_I b_J.$$

Claim 9.17 [Multiplication from shared monomials]. If every monomial $a_I b_J$ is correctly $(4,1)$-CNF-shared, then locally adding all corresponding share components produces a valid sharing of ab.

Proof

The $(4,1)$-CNF sharing map is linear. Summing the four-component sharings of all monomials therefore yields a sharing whose reconstructed value is

$$\sum_{I,J} a_I b_J = ab.$$

□

9.6.2 Subprotocol for Sharing a Monomial

For every $I, J \in \binom{[4]}{3}$, the intersection $I \cap J$ contains at least two parties. Choose two parties in this intersection. Both know a_I and b_J, so both can compute $a_I b_J$.

Claim 9.18 [Actively secure monomial sharing]. The parties can generate a correct VSS sharing of $a_I b_J$ despite one active corruption.

Proof

Let the two selected parties VSS-share values v and w, respectively, where an honest selected party uses

$$v = a_I b_J \qquad \text{or} \qquad w = a_I b_J.$$

At least one selected party is honest. The parties securely test whether $v = w$ by locally subtracting the two sharings and robustly opening the difference.
If $v = w$, retain either sharing. If $v \neq w$, one selected party is corrupted. Robustly recover a_I and b_J, compute $a_I b_J$, and locally set the shares to the canonical CNF sharing

$$\text{CNF-Share}(a_I b_J; 0, \ldots, 0).$$

Thus the retained sharing always reconstructs to the correct monomial. □

Theorem 9.19 [Active $(4,1)$-MPC]. Every efficiently computable functionality has a four-party protocol with perfect active security against one corrupted party and polynomial complexity in the circuit size.

Proof

Verifiable sharing fixes one consistent input for every party. Local addition preserves valid sharings. The multiplication protocol produces a valid sharing of every multiplication-gate output, and robust reconstruction prevents a corrupted party from changing the final output. Each interactive step is simulatable by the corresponding ideal VSS, multiplication, or reconstruction functionality. Sequential composition therefore gives perfect active security for the full circuit evaluation. □

PART VI

Proof Systems

10 Probabilistic Proof Systems

10.1 Classical Proof Systems

Definition 10.1 [Proof system for a language]. Let $L \subseteq \{0,1\}^*$. A deterministic verifier $V : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$ is a proof system for L if

$$\begin{aligned} \textit{Completeness: } & \forall x \in L,\ \exists \pi :\ V(x,\pi) = 1, \\ \textit{Soundness: } & \forall x \notin L,\ \forall \pi :\ V(x,\pi) = 0. \end{aligned}$$

10.2 NP Proofs

Definition 10.2 [Polynomial-time verifier and NP]. A language L belongs to NP if there exist a deterministic polynomial-time verifier V and a polynomial p such that

$$\begin{aligned} x \in L &\Longrightarrow \exists \pi, |\pi| \le p(|x|) : V(x,\pi) = 1, \\ x \notin L &\Longrightarrow \forall \pi, |\pi| \le p(|x|) : V(x,\pi) = 0. \end{aligned}$$

For example,

$$\text{CIRSAT} = \{C : C \text{ is a Boolean circuit and } \exists y :\ C(y) = 1\} \in \text{NP}.$$

Moreover, CIRSAT is NP-complete: for every $L \in$ NP, we can translate in polynomial time any x into a circuit C_x such that

$$x \in L \quad \Longleftrightarrow \quad C_x \in \text{CIRSAT}.$$

Define

$$\text{coNP} = \{L : \overline{L} \in \text{NP}\}$$

and EXP, the class of languages decidable in exponential time.

10.3 Interactive Proofs

Interactive proofs replace the static proof π by an interactive prover P and allow the verifier to use randomness.

Definition 10.3 [Interactive proof]. An interactive proof system (P, V) for L satisfies:

Completeness: for every $x \in L$,

$$\Pr_V[\langle P(x) \leftrightarrow V(x)\rangle_V = 1] = 1.$$

Soundness: for every cheating prover P^* and every $x \notin L$,

$$\Pr_V[\langle P^*(x) \leftrightarrow V(x)\rangle_V = 1] < \frac{1}{2}.$$

The parameters $(1, 1/2)$ can be replaced by $(1 - \epsilon_s, \epsilon_c)$ whenever

$$1 - \epsilon_s > \epsilon_c + \frac{1}{\text{poly}(|x|)}.$$

10.4 Zero Knowledge

Let (P, V) be an interactive proof system for L. By interacting with $P(x)$, the verifier can learn that $x \in L$. Zero knowledge asks what else the verifier can learn, including when the verifier cheats.

Definition 10.4 [Zero knowledge]. An interactive proof (P, V) for L is zero knowledge if, for every cheating verifier V^*, there exists a simulator S, with the same complexity as V^*, such that for every $x \in L$,

$$\langle P(x) \leftrightarrow V^*(x) \rangle_V \equiv S(V^*, x).$$

10.5 Graph Isomorphism Examples

Graphs G, H are isomorphic if we can rename the vertices of G to obtain H. Define

$$\mathsf{GI} = \{(G, H) : G \simeq H\}.$$

10.5.1 An Interactive Proof beyond NP

Theorem 10.5 [GMW86]. $\overline{\mathsf{GI}}$ has an interactive proof with a polynomial-time verifier.

Proof

Protocol (Graph non-isomorphism)

Given (G_0, G_1), the verifier samples

$$b \xleftarrow{R} \{0, 1\},$$

samples a random graph H such that $H \simeq G_b$, and sends H to the prover. Given G_0, G_1, H, the prover computes b' such that $H \simeq G_{b'}$ and sends b' to the verifier. The verifier accepts if $b' = b$ and rejects otherwise.

If $G_0 \not\simeq G_1$, the graph H identifies b uniquely, giving perfect completeness. If $G_0 \simeq G_1$, the distribution of H is independent of b, so any prover guesses b with probability at most $1/2$. □

10.5.2 A Zero-Knowledge Interactive Proof

Theorem 10.6 [GMR85]. GI has a zero-knowledge interactive proof.

Proof

Protocol (Graph isomorphism)

Given isomorphic graphs (G_0, G_1), the prover samples a random graph H such that $H \simeq G_0$ and sends H to the verifier. The verifier samples

$$b \xleftarrow{R} \{0,1\}$$

and sends b to the prover. The prover sends a permutation σ such that

$$\sigma(H) = G_b.$$

The verifier accepts if $\sigma(H) = G_b$ and rejects otherwise.

Completeness and soundness are interpreted according to Definition 10.3. Completeness follows from $G_0 \simeq G_1$. For a non-isomorphic pair, a single H cannot be isomorphic to both graphs, so a cheating prover answers a random challenge with probability at most $1/2$. A simulator chooses the challenge and permutation first and constructs the corresponding H; repeating until the verifier supplies the chosen challenge produces the same view as the real interaction. □

10.6 The Class IP

Definition 10.7 [IP and PSPACE]. IP is the class of languages having an interactive proof with a polynomial-time verifier. PSPACE is the class of languages decidable in polynomial space.

Theorem 10.8 [LFKN90, S90].

$$\mathsf{IP} = \mathsf{PSPACE}.$$

Moreover, the prover runs in exponential time and polynomial space, the round complexity is polynomial, and the verifier uses public randomness.

10.7 Two-Prover Interactive Proofs and PCPs

A two-prover interactive proof replaces the prover by P_1, P_2. The verifier may question both provers in multiple rounds, but the provers may not communicate after the protocol starts.

Definition 10.9 [MIP_2]. MIP_2 is the class of languages having a two-prover interactive proof.

NEXP is analogous to NP except that the verifier may run in exponential time. More generally, $\mathsf{NTIME}(T)$ permits a verifier running in time $T(|x|)$.

Theorem 10.10 [BFL90].

$$\mathsf{MIP}_2 = \mathsf{NEXP}.$$

Moreover, for $L \in \mathsf{NTIME}(T)$, the provers run in time $\mathrm{poly}(T)$, the verifier runs in time

$$n \cdot \mathrm{polylog}(T),$$

and the protocol has one round, $\mathrm{polylog}(T)$ communication, public randomness, and zero knowledge.

MIPs are closely related to probabilistically checkable proofs (PCPs): a PCP has a non-interactive proof π, and the verifier queries only a few random locations of π. A strong version of the BFL theorem is known as the PCP theorem, with applications to hardness of approximation, cryptography, and more.

10.8 Doubly Efficient Interactive Proofs

Definition 10.11 [Doubly efficient IP]. An interactive proof (P, V) for $L \in \mathsf{NTIME}(T)$ is doubly efficient if the prover time is $\mathrm{poly}(T)$ and the verifier time is $o(T)$.

Theorem 10.12 [GKR08]. There is a doubly efficient interactive proof for every language decidable by a uniform circuit of depth d, with verification time

$$n \cdot \mathrm{poly}(d),$$

where n is the input size. The proof has

$$O(d \log n)$$

rounds and public randomness.

Theorem 10.13 [RRR16]. There is a doubly efficient interactive proof for every language decidable in polynomial time and space $S = n^{\epsilon}$, with verification time

$$O(n + \mathrm{poly}(S)).$$

The proof has $O(1)$ rounds, $\mathrm{poly}(S)$ communication, and public randomness.

10.9 Interactive Proofs and Cryptography

Definition 10.14 [Interactive argument]. An interactive argument (P, V) for L satisfies completeness and computational soundness: for every polynomial-time P^* and every $x \notin L$,

$$\Pr_V[\langle P^*(x) \leftrightarrow V(x)\rangle_V = 1] < \frac{1}{2}.$$

It is computational zero knowledge if, for every polynomial-time V^*, there exists a polynomial-time S such that, for every $x \in L$,

$$\langle P(x) \leftrightarrow V^*(x)\rangle_V \approx_c S(V^*, x).$$

10.9.1 Succinct Arguments for NP

Theorem 10.15 [K92]. Assuming collision-resistant hashing, there is a doubly efficient interactive argument for every $L \in \mathsf{NP}$ satisfying:

- the prover runs in polynomial time given the witness;
- the verifier runs in time $n \cdot \mathrm{poly}(\lambda, \log T)$, where n is the input size, λ is the security parameter, and T is the running time of the NP verifier;
- the protocol has four rounds, $\mathrm{poly}(\lambda, \log T)$ communication, and public randomness.

The proof uses collision-resistant hashing to compile a PCP into an interactive argument.

10.9.2 Zero Knowledge for NP

Theorem 10.16 [GMR85]. Assuming one-way functions, there is a zero-knowledge interactive argument for every $L \in$ NP in which the prover runs in polynomial time given the witness, the verifier runs in polynomial time, the number of rounds is polynomial, and the randomness is public.

11 IP = PSPACE and the Sum-Check Protocol

11.1 Flawed Interactive Proofs for #SAT

Consider the claim

$$\sum_{x_1,\ldots,x_n\in\{0,1\}} \phi(x_1, x_2, \ldots, x_n) = s.$$

For $b \in \{0,1\}$, the prover sets

$$\phi_b^1(x_2, \ldots, x_n) = \phi(b, x_2, \ldots, x_n)$$

and computes

$$s_b^1 = \sum_{x_2,\ldots,x_n\in\{0,1\}} \phi_b^1(x_2, \ldots, x_n).$$

The prover sends s_0^1, s_1^1 to the verifier, which checks

$$s_0^1 + s_1^1 = s.$$

Recursing on both statements

$$s_b^1 = \sum_{x\in\{0,1\}^{n-1}} \phi_b^1(x), \qquad b \in \{0,1\},$$

makes the verifier run in time $2^{O(n)}$.

In the second flawed protocol, the verifier samples

$$b_1 \xleftarrow{R} \{0,1\}$$

and recurses only on the statement

$$s_{b_1}^1 = \sum_{x\in\{0,1\}^{n-1}} \phi_{b_1}^1(x).$$

The soundness error is $1 - 2^{-n}$.

11.2 Arithmetization

Claim 11.1 [Arithmetization]. Given a formula ϕ with n variables and a field $\mathbb{F}$, we can efficiently compute a polynomial

$$p \in \mathbb{F}[x_1, \ldots, x_n]$$

of individual degree at most $|\phi|$ such that, for every $x_1, \ldots, x_n \in \{0,1\}$,

$$p(x_1, \ldots, x_n) = \phi(x_1, \ldots, x_n).$$

The proof replaces gates according to

$$\neg x \longmapsto 1 - x, \qquad x \wedge y \longmapsto x \cdot y.$$

Let $\mathbb{F} = \mathbb{Z}_q$ for $q > 2^n$. It is sufficient to prove

$$s = \sum_{x \in \{0,1\}^n} p(x).$$

11.3 The Sum-Check Protocol

Let $s \in \mathbb{F}$ and let

$$p \in \mathbb{F}[x_1, \ldots, x_n]$$

have individual degree at most d. The asserted identity is

$$s = \sum_{x \in \{0,1\}^n} p(x).$$

Theorem 11.2 [Sum-check parameters]. The sum-check protocol has the following parameters:

- the prover runs in time
$$2^n \cdot |p| \cdot \mathrm{poly}(n, d, \log |\mathbb{F}|),$$
where $|p| \leq d^n \log |\mathbb{F}|$;
- the verifier runs in time
$$\mathrm{poly}(n, d, \log |\mathbb{F}|)$$
plus one evaluation of p;
- perfect completeness and soundness error
$$\frac{nd}{|\mathbb{F}|};$$
- n rounds and public randomness.

If

$$|\mathbb{F}| > \max\{2n|\phi|, 2^n\},$$

then the resulting interactive proof for #SAT has soundness error at most $1/2$.

Protocol (Sum-check)

Set $s_0 = s$. For $i = 1, \ldots, n$:

1. The prover sends a univariate polynomial p_i of degree at most d. For the honest prover,
$$p_i(Y) = \sum_{x_{i+1}, \ldots, x_n \in \{0,1\}} p(r_1, \ldots, r_{i-1}, Y, x_{i+1}, \ldots, x_n).$$

2. The verifier rejects unless
$$p_i(0) + p_i(1) = s_{i-1}.$$
3. The verifier samples $r_i \xleftarrow{R} \mathbb{F}$, sets $s_i = p_i(r_i)$, and sends r_i to the prover.

After round n, the verifier accepts if and only if
$$s_n = p(r_1, \ldots, r_n).$$

11.3.1 Completeness

Theorem 11.3 [Perfect completeness of sum check]. The honest prover is accepted with probability 1.

Proof

For every $i \leq n$, the prover sets
$$p_i(y) = \sum_{x_{i+1},\ldots,x_n \in \{0,1\}} p(r_1, \ldots, r_{i-1}, y, x_{i+1}, \ldots, x_n).$$

Therefore, for every $i \leq n$,
$$\begin{aligned} s_{i-1} &= p_{i-1}(r_{i-1}) \\ &= \sum_{x_i,\ldots,x_n \in \{0,1\}} p(r_1, \ldots, r_{i-1}, x_i, \ldots, x_n) \\ &= p_i(0) + p_i(1). \end{aligned}$$

Moreover,
$$p_n(r_n) = p(r_1, \ldots, r_n).$$

All verification checks pass. □

11.4 The Schwartz–Zippel Lemma

Lemma 11.4 [Schwartz–Zippel]. Let $\mathbb{F}$ be a finite field, and let
$$f(x_1, \ldots, x_m) \in \mathbb{F}[x_1, \ldots, x_m]$$
be a nonzero polynomial of total degree at most d. Then
$$\Pr_{a_1,\ldots,a_m \xleftarrow{R} \mathbb{F}} [f(a_1, \ldots, a_m) = 0] \leq \frac{d}{|\mathbb{F}|},$$
where each a_i is chosen independently and uniformly at random from $\mathbb{F}$.

Proof

We prove the lemma by induction on m.
Basis. If $m = 1$, then a nonzero univariate polynomial has at most $\deg f$ roots in $\mathbb{F}$. Thus

$$\Pr_{a \xleftarrow{R} \mathbb{F}}[f(a) = 0] \leq \frac{|\{a \in \mathbb{F} : f(a) = 0\}|}{|\mathbb{F}|} \leq \frac{\deg f}{|\mathbb{F}|} \leq \frac{d}{|\mathbb{F}|}.$$

Step. Assume the lemma holds for $m - 1$. Since we sample a uniformly from $\mathbb{F}^m$, write

$$a = (b, c), \qquad b \xleftarrow{R} \mathbb{F}^{m-1}, \qquad c \xleftarrow{R} \mathbb{F}.$$

Let k be the maximum degree of x_m in f. Write f as a polynomial in x_m with coefficients in $\mathbb{F}[x_1, \ldots, x_{m-1}]$:

$$f(x_1, \ldots, x_m) = \sum_{i=0}^{k} g_i(x_1, \ldots, x_{m-1}) x_m^i.$$

Since k is the maximum degree of x_m, the polynomial g_k is nonzero. Moreover, because the total degree of f is at most d, the total degree of g_k is at most $d - k$.
By the law of total probability, condition on whether b is a root of g_k:

$$\begin{aligned}\Pr[f(b, c) = 0] = {} & \Pr[g_k(b) = 0] \Pr[f(b, c) = 0 \mid g_k(b) = 0] \\ & + \Pr[g_k(b) \neq 0] \Pr[f(b, c) = 0 \mid g_k(b) \neq 0].\end{aligned}$$

Replacing the unneeded probabilities by 1 gives

$$\Pr[f(a) = 0] \leq \Pr[g_k(b) = 0] + \Pr[f(b, c) = 0 \mid g_k(b) \neq 0].$$

For the first term, g_k is a nonzero polynomial in $m - 1$ variables of total degree at most $d - k$. By the induction hypothesis,

$$\Pr_{b \xleftarrow{R} \mathbb{F}^{m-1}}[g_k(b) = 0] \leq \frac{d - k}{|\mathbb{F}|}.$$

For the second term, condition on a fixed b such that $g_k(b) \neq 0$. Then $f(b, x_m)$ is a nonzero univariate polynomial in x_m of degree exactly k, since its leading coefficient $g_k(b)$ is nonzero. By the base case, the probability that a random $c \xleftarrow{R} \mathbb{F}$ is a root is at most

$$\frac{k}{|\mathbb{F}|}.$$

Therefore,

$$\Pr[f(a) = 0] \leq \frac{d - k}{|\mathbb{F}|} + \frac{k}{|\mathbb{F}|} = \frac{d}{|\mathbb{F}|}.$$

□

11.5 Soundness of Sum Check

Assume

$$s_0 \neq \sum_{x \in \{0,1\}^n} p(x).$$

Let P^* be a cheating prover, and let

$$\widetilde{p}_1, r_1, \ldots, \widetilde{p}_n, r_n$$

be the transcript of (P^*, V). Let p_i denote the message sent by the honest prover:

$$p_i(y) = \sum_{x_{i+1},\ldots,x_n \in \{0,1\}} p(r_1, \ldots, r_{i-1}, y, x_{i+1}, \ldots, x_n).$$

If $p_1 \equiv \widetilde{p}_1$, then

$$\widetilde{p}_1(0) + \widetilde{p}_1(1) \neq s_0$$

and the verifier rejects in the first round. If $p_1 \not\equiv \widetilde{p}_1$, then $p_1 - \widetilde{p}_1$ is a nonzero degree-d polynomial. By Lemma 11.4,

$$\Pr_{r_1 \xleftarrow{R} \mathbb{F}} [p_1(r_1) = \widetilde{p}_1(r_1)] \leq \frac{d}{|\mathbb{F}|}.$$

Thus

$$s_1 = \widetilde{p}_1(r_1) \neq p_1(r_1) = \sum_{x_2,\ldots,x_n \in \{0,1\}} p(r_1, x_2, \ldots, x_n)$$

except with probability $d/|\mathbb{F}|$.

Similarly, conditioned on

$$s_{i-1} \neq \sum_{x_i,\ldots,x_n \in \{0,1\}} p(r_1, \ldots, r_{i-1}, x_i, \ldots, x_n),$$

either $p_i \equiv \widetilde{p}_i$ and the verifier rejects in round i, or $p_i \not\equiv \widetilde{p}_i$ and the false claim survives the random point with probability at most $d/|\mathbb{F}|$. Therefore the verifier rejects in the last round except with probability

$$\frac{nd}{|\mathbb{F}|}.$$

11.6 Proof of IP = PSPACE

Let TQBF be the language of true quantified Boolean formulas:

$$\begin{aligned}\text{TQBF} = \{\phi : &\phi \text{ is a Boolean formula and} \\ &\forall x_1 \exists x_2 \forall x_3 \cdots \exists x_n \, \phi(x_1, \ldots, x_n) = 1\}.\end{aligned}$$

11.7 IP $\subseteq$ PSPACE

Let (P, V) be an interactive proof for L. For every prover A, define

$$\alpha_A(x) = \Pr[\langle A(x) \leftrightarrow V(x)\rangle_V = 1].$$

By completeness and soundness, we can decide whether $x \in L$ by computing

$$\max_A \alpha_A(x).$$

We cannot enumerate all cheating provers A in polynomial space. Instead, we implement a prover that maximizes α_A in polynomial space by enumerating all possible transcripts recursively.

Assume without loss of generality that (P, V) has 2ℓ messages. Let

$$V_{r,x}(p_1, \ldots, p_i) = v_{i+1}$$

be the verifier's next-message function with randomness r and input x. Let

$$\alpha(m_1, \ldots, m_i)$$

be the probability that the optimal prover convinces V to accept, conditioned on the first i messages being $m_1, \ldots, m_i$. Then:

- after the last prover message, α is the conditional expectation of the verifier's acceptance predicate;
- after a prover message, the next verifier message is averaged over the randomness r consistent with the transcript;
- before a prover message,
$$\alpha(v_1, \ldots, v_i) = \max_{p_i} \alpha(v_1, \ldots, v_i, p_i).$$

All these quantities can be computed recursively in polynomial space.

11.8 TQBF Is PSPACE-Complete

Theorem 11.5 [Local transition formula]. Let M be a program running in time t and space s on inputs of size n. There exist $s' = \mathrm{poly}(s)$ and a $\mathrm{poly}(s)$-size Boolean formula

$$\phi_{M,n} : \{0,1\}^{2s'} \longrightarrow \{0,1\}$$

such that

$$\phi_{M,n}(C, C') = 1$$

if and only if M transitions from configuration C to C' in one step.

For every $x \in \{0,1\}^n$, $M(x) = 1$ if and only if there exist configurations $C_1, \ldots, C_t \in \{0,1\}^{s'}$ such that

- $C_1 = x0^{s'-|x|}$;

- for every $i < t$, $\phi_{M,n}(C_i, C_{i+1}) = 1$;
- C_t is an accepting configuration.

TQBF $\in$ PSPACE via the naive recursive algorithm. For a polynomial-space program M, construct QBFs ϕ_i such that $\phi_i(C, C')$ is true if and only if M transitions from C to C' in 2^i steps. Set

$$\phi_0 = \phi_{M,n}.$$

The first attempted recursion is

$$\phi_i(C, C') = \exists D : \phi_{i-1}(C, D) \wedge \phi_{i-1}(D, C'),$$

but ϕ_i is exponential in i. Instead use

$$\phi_i(C, C') = \exists D \, \forall E, E' : \big((E, E') = (C, D) \vee (E, E') = (D, C')\big) \longrightarrow \phi_{i-1}(E, E').$$

11.9 Interactive Proof for TQBF

First arithmetize TQBF and then use a variant of sum check. Given a formula ϕ with n variables, compute a polynomial

$$p \in \mathbb{F}[x_1, \ldots, x_n]$$

of degree at most $|\phi|$ such that

$$A_{x_1} E_{x_2} A_{x_3} \cdots E_{x_n} p(x_1, \ldots, x_n) = \forall x_1 \exists x_2 \forall x_3 \cdots \exists x_n \phi(x_1, \ldots, x_n),$$

where

$$A_y q(y, x_1, \ldots, x_m) = q(0, x_1, \ldots, x_m) q(1, x_1, \ldots, x_m),$$
$$E_y q(y, x_1, \ldots, x_m) = 1 - (1 - q(0, x_1, \ldots, x_m))(1 - q(1, x_1, \ldots, x_m)).$$

11.9.1 The Degree Problem

The goal is to eliminate quantifiers one by one. The direct univariate polynomial

$$p_1(y) = E_{x_2} A_{x_3} \cdots E_{x_n} p(y, x_2, \ldots, x_n)$$

has high degree. The solution uses the fact that x^d and x agree on $\{0, 1\}$.

Definition 11.6 [Degree-reduction operator]. Define

$$R_y q(y, x_1, \ldots, x_m) = (1 - y) q(0, x_1, \ldots, x_m) + y q(1, x_1, \ldots, x_m).$$

The polynomial $R_y q$ has degree one in y and agrees with q on Boolean inputs.

It is sufficient to prove

$$A_{x_1} R_{x_1} E_{x_2} R_{x_1} R_{x_2} A_{x_3} R_{x_1} R_{x_2} R_{x_3} \cdots E_{x_n} R_{x_1} R_{x_2} \cdots R_{x_n} p(x_1, \ldots, x_n) = 1.$$

There are

$$m = O(n^2)$$

operators. The protocol eliminates one operator per round and eliminates A, E operators as in sum check.

11.9.2 Eliminating Degree-Reduction Operators

Before round k, we need to prove a statement of the form

$$R_{x_i} Q_{k+1} \cdots Q_m p(r_1, \ldots, r_j, x_{j+1}, \ldots, x_n) = s_{k-1}$$

for some $i \leq j$. The prover computes a univariate polynomial p_k of degree at most $\max\{d, 2\}$:

$$p_k(y) = Q_{k+1} \cdots Q_m p(r_1, \ldots, r_{i-1}, y, r_{i+1}, \ldots, r_j, x_{j+1}, \ldots, x_n).$$

The prover sends p_k to the verifier. The verifier checks

$$R_y p_k(r_i) = s_{k-1},$$

samples a new $r_i \xleftarrow{R} \mathbb{F}$, sets

$$s_k = p_k(r_i),$$

and sends r_i to the prover.

11.9.3 Efficiency

Given $p \in \mathbb{F}[x_1, \ldots, x_n]$ of degree d:

- the prover runs in time
$$2^{n^2} \cdot |p| \cdot \mathrm{poly}(n, d, \log |\mathbb{F}|),$$
where $|p| \leq d^n \log |\mathbb{F}|$;
- the verifier runs in time $\mathrm{poly}(n, d, \log |\mathbb{F}|)$ plus one evaluation of p;
- the protocol has n rounds and public randomness.

For a program M running in time t and space s, the parameters satisfy

$$n = \mathrm{poly}(s, \log t), \qquad |\mathbb{F}|, |p|, d = \mathrm{poly}(s).$$

12 Probabilistically Checkable Proofs and Succinct Arguments

12.1 Probabilistically Checkable Proofs

Definition 12.1 [PCP system]. Let $L \subseteq \{0,1\}^*$. A PCP system consists of a prover P that produces an oracle string Π and a probabilistic oracle verifier V. For functions $r, q : \mathbb{N} \to \mathbb{N}$ and soundness error ε, the verifier uses at most $r(|x|)$ random bits, makes at most $q(|x|)$ oracle queries, and satisfies:

1. *Completeness:* for every $x \in L$, there exists Π such that
$$\Pr[V^{\Pi}(x) = 1] = 1.$$
2. *Soundness:* for every $x \notin L$ and every oracle string Π^*,
$$\Pr[V^{\Pi^*}(x) = 1] \leq \varepsilon.$$

12.2 From Interactive Proofs to PCPs

The construction converts an interactive proof satisfying Definition 10.3 into a non-interactive proof satisfying Definition 12.1.

Let L have an interactive proof (P, V). On inputs of length n, assume that V uses $r(n)$ random bits, communicates at most $m(n)$ bits with the prover, and that the protocol has perfect completeness and soundness error at most $1/2$.

Theorem 12.2 [HW 5, Exercise 6]. The interactive proof gives a PCP for L with proof length at most
$$m(n)2^{m(n)+1},$$
query complexity at most $m(n)$, and soundness error at most $1/2$.

Proof

Construct the proof string π as a lookup table indexed by every possible verifier-message history
$$h \in \{0,1\}^{\leq m(n)}.$$
For every history h, the entry $\pi[h]$ contains the bits that the deterministic honest prover P would send in response.
The PCP verifier $V_{\mathrm{PCP}}^{\pi}(x)$ samples a random tape
$$r \xleftarrow{R} \{0,1\}^{r(n)}$$
and initializes a simulated verifier $V(x; r)$ with history $h = \emptyset$. In each round, when V outputs a message, V_{PCP} appends it to h, queries $\pi[h]$ for the prover's response, and feeds this response back to V. The PCP verifier accepts if and only if the simulated verifier accepts.
Length of the proof string. The verifier sends at most $m(n)$ bits throughout the interactive protocol.

Thus the number of possible histories is at most

$$\sum_{i=0}^{m(n)} 2^i = 2^{m(n)+1} - 1.$$

For each history, the prover's response is at most $m(n)$ bits long. Therefore the total proof length is bounded by

$$m(n) \cdot 2^{m(n)+1}$$

bits.

Number of queries. The verifier queries only prover responses along one execution path. Since the total communication in one execution is at most $m(n)$ bits, it reads at most $m(n)$ proof bits.

Soundness. Assume $x \notin L$. Every proof string π^* defines a fixed deterministic cheating prover strategy P^* for the interactive protocol. By construction,

$$V_{\mathrm{PCP}}^{\pi^*}(x) = 1$$

if and only if $V(x; r)$ accepts when interacting with P^*. Hence the acceptance probability is exactly

$$\Pr[(P^*, V)(x) = 1] \leq \frac{1}{2},$$

by the soundness of the original interactive proof. □

12.3 PCP Existence Theorems

Theorem 12.3 [Babai–Fortnow–Lund 91]. There exists a PCP for NEXP in which the verifier runs in polynomial time.

Theorem 12.4 [PCP theorem; AS92, ALMSS92]. There exists a PCP for NP in which the verifier makes

$$O(1)$$

queries and uses

$$O(\log n)$$

random bits.

Theorem 12.5 [Babai–Fortnow–Lund–Szegedy 91]. There exists a PCP for NP in which:

- P runs in polynomial time given the witness;
- V runs in time $n \cdot \mathrm{polylog}(n)$ and makes $\mathrm{polylog}(n)$ queries.

Fix $L \in$ NP and an NP verifier M running in time

$$T = \mathrm{poly}(n).$$

12.4 3CNF Formulas and Cook–Levin

Let $x_1, \dots, x_n$ be Boolean variables. For every i, the expressions x_i and $\overline{x_i}$ are literals. An or-clause is a disjunction of literals, for example

$$x_1 \vee \overline{x_5} \vee x_9 \vee \overline{x_{17}}.$$

A conjunctive-normal-form formula is a conjunction of or-clauses. It is a 3CNF formula if every or-clause contains exactly three literals.

Theorem 12.6 [Cook–Levin]. Fix $L \in$ NP and let M be its verifier running in time $T = \mathrm{poly}(n)$. There is a 3CNF formula

$$\phi_M(y_1, \dots, y_N), \qquad N = \mathrm{poly}(T),$$

such that for every $x \in \{0,1\}^n$, the following are equivalent:

- $x \in L$;
- there exists w such that $M(x, w) = 1$;
- there exists an assignment $\sigma : [N] \to \{0,1\}$ such that

  $$x = (\sigma(1), \dots, \sigma(n))$$

  and

  $$\phi_M(\sigma(1), \dots, \sigma(N)) = 1.$$

Let ϕ_x be a 3CNF formula satisfying

$$\phi_x(y_1, \dots, y_N) = 1 \quad \Longleftrightarrow \quad x = (y_1, \dots, y_n).$$

Then

$$x \in L \quad \Longleftrightarrow \quad \phi_{M,x} = \phi_M \wedge \phi_x \text{ is satisfiable.}$$

12.5 A Succinct Representation of ϕ_M

For the PCP verifier to run quickly, it needs a succinct representation of ϕ_M. Let

$$\varphi_M : [N]^3 \times \{0,1\}^3 \longrightarrow \{0,1\}$$

satisfy

$$\varphi_M(i_1, i_2, i_3, b_1, b_2, b_3) = 1$$
$$\Longleftrightarrow \quad \phi_M \text{ contains the clause } (y_{i_1} = b_1) \vee (y_{i_2} = b_2) \vee (y_{i_3} = b_3).$$

By the Cook–Levin proof, φ_M is computed by a Boolean circuit of size $\mathrm{polylog}(N)$ and depth $O(\log \log N)$.

Similarly, let

$$\varphi_x : [N]^3 \times \{0,1\}^3 \longrightarrow \{0,1\}$$

describe ϕ_x. Then

$$\varphi_{M,x} = \varphi_M + \varphi_x$$

describes

$$\phi_{M,x} = \phi_M \wedge \phi_x.$$

12.6 Low-Degree Extensions

Let $\mathbb{F}$ be a field of size $\mathrm{polylog}(N)$, and choose

$$\{0,1\} \subseteq H \subset \mathbb{F}, \qquad |H| = \log N.$$

Let m satisfy

$$|H|^m = N.$$

Identify the indexes in $[N]$ with elements of H^m, and view an assignment as a function

$$\sigma : H^m \longrightarrow \{0,1\}.$$

Claim 12.7 [Low-degree extension]. Fix a field $\mathbb{F}$, a set $H \subseteq \mathbb{F}$, and a function

$$\sigma : H^m \longrightarrow \mathbb{F}.$$

There exists a unique polynomial

$$\widetilde{\sigma} : \mathbb{F}^m \longrightarrow \mathbb{F}$$

of individual degree $|H| - 1$ satisfying

$$\forall x \in H^m : \quad \sigma(x) = \widetilde{\sigma}(x).$$

Proof

For $a \in H$, define the degree-$|H| - 1$ Lagrange polynomial

$$L_a(x) = \prod_{b \in H \setminus \{a\}} \frac{x - b}{a - b}.$$

Then $L_a(a) = 1$ and $L_a(b) = 0$ for every $b \in H \setminus \{a\}$. Define

$$\widetilde{\sigma}(x_1, \ldots, x_m) = \sum_{a_1, \ldots, a_m \in H} \sigma(a_1, \ldots, a_m) \prod_{i=1}^{m} L_{a_i}(x_i).$$

This polynomial agrees with σ on H^m. Uniqueness follows by a counting argument. □

12.7 Arithmetizing Satisfiability

View

$$\varphi_M, \varphi_x : H^{3m+3} \longrightarrow \{0,1\}.$$

If $b_1, b_2, b_3 \notin \{0,1\}$, define both functions to be zero. Let

$$\widetilde{\sigma} : \mathbb{F}^m \longrightarrow \mathbb{F}$$

be the LDE of σ, and let

$$\widetilde{\varphi}_x : \mathbb{F}^{3m+3} \longrightarrow \mathbb{F}$$

be the LDE of φ_x. It can be evaluated in time $O(n)$. There exists a polynomial

$$\widetilde{\varphi}_M : \mathbb{F}^{3m+3} \longrightarrow \mathbb{F}$$

of degree

$$\delta = \text{polylog}(N)$$

that agrees with φ_M on H^{3m+3} and can be evaluated in time $\text{polylog}(N)$ by arithmetizing φ_M.

Claim 12.8 [Polynomial satisfiability condition]. $\phi_{M,x}$ is satisfiable if and only if there exists $\widetilde{\sigma} : \mathbb{F}^m \to \mathbb{F}$ such that, for every

$$z_1, z_2, z_3, b_1, b_2, b_3 \in H^{3m+3},$$

$$\widetilde{\varphi}_{M,x}(z_1, z_2, z_3, b_1, b_2, b_3) \cdot \prod_{i\in[3]} (\widetilde{\sigma}(z_i) - b_i) = 0.$$

To prove that such a $\widetilde{\sigma}$ exists, we use sum-check machinery.

12.8 The PCP Prover and Verifier

Given x, w such that $M(x, w) = 1$, the prover computes an assignment σ satisfying $\phi_{M,x}$. Let $\widetilde{\sigma}$ be its LDE and define

$$Q(z_1, z_2, z_3, b_1, b_2, b_3) = \widetilde{\varphi}_{M,x}(z_1, z_2, z_3, b_1, b_2, b_3) \prod_{i\in[3]} (\widetilde{\sigma}(z_i) - b_i).$$

Protocol (PCP prover)

The proof oracle contains:

- the truth table of $\widetilde{\sigma}$;
- the truth table of Q;
- a PCP proof that Q vanishes on H^{3m+3}, called a zero sub-cube proof.

Protocol (PCP verifier)

The verifier performs the following checks:

- check that $\widetilde{\sigma}$ has the required low degree and Q has degree $\delta + 1$, using low-degree tests;
- check that Q vanishes on H^{3m+3} using the zero sub-cube proof;
- check consistency between Q and $\widetilde{\sigma}$ by sampling

$$z_1, z_2, z_3, b_1, b_2, b_3 \xleftarrow{R} \mathbb{F}^{3m+3},$$

querying $Q(z_1, z_2, z_3, b_1, b_2, b_3)$ and $\widetilde{\sigma}(z_1), \widetilde{\sigma}(z_2), \widetilde{\sigma}(z_3)$, computing $\widetilde{\varphi}_{M,x}(z_1, z_2, z_3, b_1, b_2, b_3)$, and checking

$$Q(z_1, z_2, z_3, b_1, b_2, b_3) = \widetilde{\varphi}_{M,x}(z_1, z_2, z_3, b_1, b_2, b_3) \prod_{i \in [3]} (\widetilde{\sigma}(z_i) - b_i).$$

By the Schwartz–Zippel lemma, consistency can be tested at a random point. The soundness error is

$$\frac{(\delta + 1)(3m + 3)}{|\mathbb{F}|}.$$

12.9 Zero Sub-Cube Proofs

Let

$$Q \in \mathbb{F}[x_1, \ldots, x_\ell]$$

have individual degree d.

Definition 12.9 [Zero sub-cube proof]. The proof has completeness

$$Q|_{H^\ell} \equiv 0 \quad \Longrightarrow \quad \Pr_{\Pi \leftarrow P(Q)}[V(Q, \Pi) = 1] = 1,$$

and soundness

$$Q|_{H^\ell} \not\equiv 0 \quad \Longrightarrow \quad \forall \Pi^* : \Pr[V(Q, \Pi^*) = 1] \leq \frac{1}{2}.$$

The idea is that $Q|_{H^\ell} \equiv 0$ if and only if $\widetilde{Q} \equiv 0$, where $\widetilde{Q}$ is the LDE of Q. The problem is checking that $\widetilde{Q}$ is really the LDE of Q. The variables are linearized one by one, as in the IP = PSPACE protocol.

12.9.1 The Prover

Set $Q_0 = Q$. For every $i \in [\ell]$, set

$$Q_i(z_1, \ldots, z_\ell) = \sum_{h \in H} L_h(z_i) \cdot Q_{i-1}(z_1, \ldots, z_{i-1}, h, z_{i+1}, \ldots, z_\ell).$$

The proof contains the truth tables of

$$Q_0, \ldots, Q_\ell.$$

The polynomial Q_ℓ is the LDE of Q.

12.9.2 The Verifier

Protocol (Random-line low-degree test)

The verifier samples

$$z_1, \dots, z_\ell \xleftarrow{R} \mathbb{F}.$$

For every $i \le \ell$, it:

- checks that Q_i has degree at most d using a low-degree test;
- queries $Q_i(z_1, \dots, z_\ell)$;
- queries
$$Q_{i-1}(z_1, \dots, z_{i-1}, h, z_{i+1}, \dots, z_\ell)$$
for every $h \in H$;
- checks the interpolation identity defining Q_i;
- finally checks
$$Q_\ell(z_1, \dots, z_\ell) = 0.$$

By Schwartz–Zippel, the soundness error is

$$\frac{\ell^2 d}{|\mathbb{F}|}.$$

12.10 Low-Degree Tests

Let $Q^* : \mathbb{F}^\ell \to \mathbb{F}$ be a function. Exact testing with $o(|\mathbb{F}|^\ell)$ queries is impossible. The soundness requirement is therefore relaxed to testing whether Q^* is close to a low-degree polynomial.

Definition 12.10 [Low-degree test]. The test has perfect completeness when Q^* has total degree at most d. For soundness, if every polynomial Q of total degree at most d satisfies

$$\Pr_{x \leftarrow \mathbb{F}^\ell}[Q^*(x) = Q(x)] \le 1 - \delta,$$

then

$$\Pr[V^{Q^*} = 1] < 1 - \epsilon.$$

The verifier samples

$$a, b \xleftarrow{R} \mathbb{F}^\ell,$$

queries $Q^*(ax + b)$ for every $x \in \mathbb{F}$, and checks that

$$f(x) = Q^*(ax + b)$$

has degree at most d.

Theorem 12.11 [Rubinfeld–Sudan 92]. Fix $\mathbb{F} = \mathbb{F}_q$ and $d < q - 1$. The random-line protocol is a low-degree test for every

$$\delta = O(1/d^2) \qquad \text{and} \qquad \epsilon = \delta/2.$$

12.11 Local Correction

In the PCP construction, the verifier can test only that Q^* is close to a low-degree polynomial Q. A correction algorithm emulates queries to Q.

Protocol (Local correction at b)

1. sample $a \xleftarrow{R} \mathbb{F}^\ell$;
2. query $Q^*(ax+b)$ for every $x \in [d+1] \subset \mathbb{F}$;
3. interpolate the degree-d polynomial f satisfying
$$f(x) = Q^*(ax+b) \qquad \text{for every } x \in [d+1];$$
4. return $f(0)$.

Claim 12.12 [Local correction]. Let $Q^* : \mathbb{F}^\ell \to \mathbb{F}$ and let Q be a polynomial of total degree at most d satisfying
$$\Pr_{x \leftarrow \mathbb{F}^\ell}[Q^*(x) = Q(x)] \geq 1 - \delta.$$
Then, for every $b \in \mathbb{F}^\ell$, the correction algorithm returns $Q(b)$ with probability at least
$$1 - (d+1)\delta.$$

12.12 Succinct Arguments

Definition 12.13 [Succinct interactive argument]. An interactive argument (P, V) for $L \in$ NP satisfies:

Completeness: for every $\lambda \in \mathbb{N}$, every $x \in L$, and every witness w,
$$\Pr_V[\langle P(\lambda, x, w) \leftrightarrow V(\lambda, x)\rangle_V = 1] = 1.$$

Soundness: for every $\lambda \in \mathbb{N}$, every cheating prover P^* of size $\mathrm{poly}(\lambda)$, and every $x \notin L$,
$$\Pr_V[\langle P^* \leftrightarrow V(\lambda, x)\rangle_V = 1] < \frac{1}{2}.$$

The argument is succinct if its communication is shorter than the witness.

Theorem 12.14 [Kilian 92]. Assuming collision-resistant hashing, there exists a succinct argument for NP with the following parameters:

- prover time $\mathrm{poly}(\lambda, n)$;
- verifier time $n \cdot \mathrm{poly}(\lambda, \log n)$;
- communication $\mathrm{poly}(\lambda, \log n)$;
- four messages and public randomness.

12.12.1 Compiling a PCP

The compiler begins with a proof satisfying Definition 12.1 and produces an argument satisfying Definition 12.13.

A direct protocol lets the verifier generate PCP queries $q_1, \ldots, q_m$ and lets the prover answer

$$a_i = \pi[q_i].$$

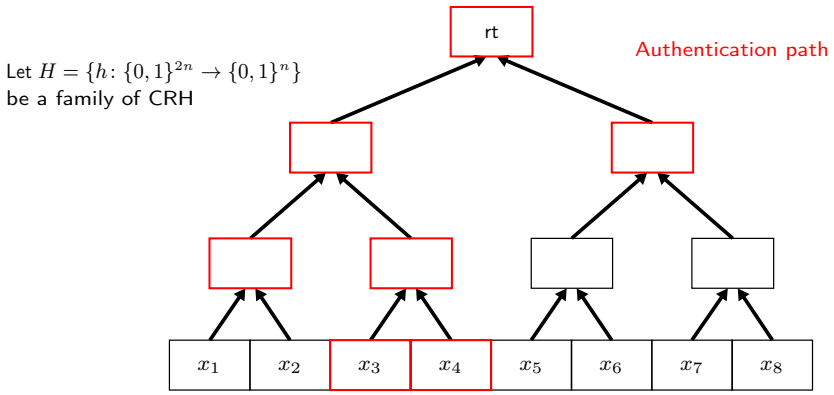
This is not sound if a cheating prover chooses π as a function of the queries. The prover must commit to π before receiving $q_1, \ldots, q_m$, and both the commitment and the openings of $a_1, \ldots, a_m$ must be succinct.

12.12.2 Merkle Hash Trees

Let

$$\mathcal{H} = \{h : \{0,1\}^{2n} \to \{0,1\}^n\}$$

be a family of collision-resistant hash functions. Place the proof blocks $x_1, \ldots, x_8$ at the leaves of a binary tree, hash pairs of children up the tree, and use the root rt as the commitment. An authentication path contains the sibling hashes needed to recompute rt from an opened leaf.



12.12.3 Kilian's Succinct Argument

Protocol (Kilian's argument)

1. the verifier sends a hash function h;
2. the prover computes the PCP proof

   $$\pi \leftarrow P_{\mathrm{PCP}}(x, w),$$

   builds its Merkle tree, and sends the root rt;
3. the verifier generates

   $$q_1, \ldots, q_m \leftarrow V_{\mathrm{PCP}}(x)$$

and sends the queries;

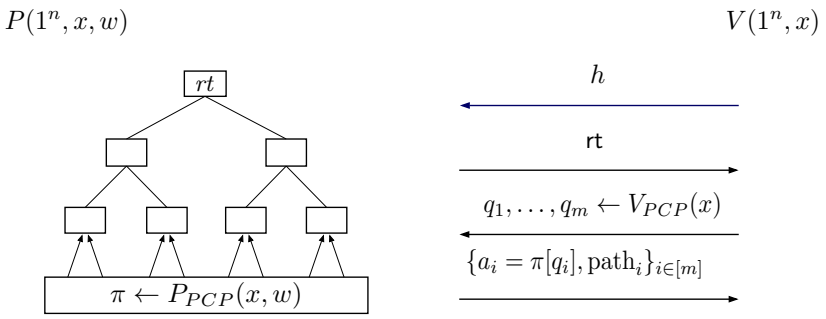
4. the prover sends
$$(a_i = \pi[q_i], \mathrm{path}_i)_{i\in[m]}.$$

The verifier checks every authentication path against rt and then applies the PCP decision predicate to the authenticated answers.

Theorem 12.15 [Soundness of Kilian's compilation]. If the underlying PCP is sound and the hash family is collision resistant, then Kilian's protocol is a computationally sound succinct argument.

Proof

Suppose a polynomial-size prover convinces the verifier on a false statement with probability ϵ. Fix a hash function h for which the conditional acceptance probability is at least $\epsilon/2$. If every PCP location had one answer occurring with overwhelming conditional probability, those answers would define a fixed PCP proof accepted with non-negligible probability, contradicting PCP soundness. Consequently, there is an index i for which both bit values occur in accepting openings with probability at least $\epsilon/(4\ell)$, where ℓ is the PCP proof length.
Repeating the interaction $O(\epsilon^{-1}\ell)$ times yields two different valid authentication paths for the same index and root with constant probability. At the first tree node where the authenticated values differ, the two paths give a collision in h, contradicting collision resistance. □



13 Zero Knowledge, Composition, and the Fiat–Shamir Heuristic

13.1 Freivalds' Test for Matrix Multiplication

Let $\mathbb{F}$ be a finite field and define

$$L_{\mathrm{MM}} = \{(A, B, C) \in (\mathbb{F}^{n\times n})^3 : AB = C\}.$$

Protocol (Freivalds' test)

Given $A = (a_{ij})$, $B = (b_{ij})$, and $C = (c_{ij})$:

1. sample $r \xleftarrow{R} \mathbb{F}$;
2. calculate
$$x = (1, r, \ldots, r^{n-1})^\top;$$
3. compute $w = Bx$, $u = Aw$, and $v = Cx$;
4. accept if and only if $u_i = v_i$ for every $i \in [n]$.

Claim 13.1 [HW 5, Exercise 1: complexity]. The algorithm uses $O(n^2)$ field operations.

Proof

Computing x takes $O(n \log n)$ field operations using fast modular powering. Each dot product takes $O(n)$ operations, and the algorithm computes n dot products for each of w, u, v. Thus the total is

$$O(n \log n) + 3n \cdot O(n) = O(n^2).$$

□

Let $\mathcal{A}(A, B, C; r)$ denote the algorithm's output.

Theorem 13.2 [Perfect completeness]. If $(A, B, C) \in L_{\mathrm{MM}}$, then

$$\Pr_{r\xleftarrow{R}\mathbb{F}} [\mathcal{A}(A, B, C; r) = 1] = 1.$$

Proof

If $AB = C$, then for every $r \in \mathbb{F}$,

$$u = Aw = A(Bx) = (AB)x = Cx = v.$$

Hence every coordinate check passes. □

Theorem 13.3 [Soundness]. If $(A, B, C) \notin L_{\mathrm{MM}}$, then

$$\Pr_{r\xleftarrow{R}\mathbb{F}} [\mathcal{A}(A, B, C; r) = 0] \geq 1 - \frac{n-1}{|\mathbb{F}|}.$$

Proof

There is an i such that $(AB)_i \neq C_i$. If the algorithm accepts, then $u_i = v_i$, so

$$\sum_{j=1}^{n} \left(\sum_{k=1}^{n} a_{ik} b_{kj} - c_{ij} \right) r^{j-1} = 0.$$

Define

$$p(X) = \sum_{j=1}^{n} \left(\sum_{k=1}^{n} a_{ik} b_{kj} - c_{ij} \right) X^{j-1}.$$

Because $(AB)_i \neq C_i$, there is an index j for which

$$\sum_{k=1}^{n} a_{ik} b_{kj} - c_{ij} \neq 0.$$

This value is the coefficient of X^{j-1} in p, so p is nonzero. Moreover, p has degree at most $n-1$. Therefore, by Schwartz–Zippel,

$$\Pr[\mathcal{A}(A, B, C; r) = 1] \leq \Pr[p(r) = 0] \leq \frac{n-1}{|\mathbb{F}|}.$$

Taking the complement proves the claim. □

The prover may send $\perp$, after which the verifier alone executes the test. This is an interactive proof for L_{MM} with perfect completeness and soundness error $(n-1)/|\mathbb{F}|$.

13.2 Zero-Error Interactive Proofs

Theorem 13.4 [HW 5, Exercise 2]. If a language $L \subseteq \Sigma^*$ has an interactive proof with perfect completeness and perfect soundness, then

$$L \in \mathsf{NP}.$$

Proof

Let (P, V) be such a proof, and suppose that on input x the verifier uses $r(|x|)$ random bits. For a tape $\rho \in \{0,1\}^{r(|x|)}$, let V_ρ denote the deterministic verifier obtained by fixing V's random tape to ρ.
For $x \in L$, perfect completeness says

$$\begin{aligned} 1 &= \Pr_{\rho, R_P} [\langle P(R_P), V_\rho \rangle(x) = 1] \\ &= 2^{-r(|x|)} \sum_{\rho} \Pr_{R_P} [\langle P(R_P), V_\rho \rangle(x) = 1]. \end{aligned}$$

Every summand lies in $[0,1]$, so every summand equals 1. For $x \notin L$ and every cheating prover $\widehat{P}$,

perfect soundness gives

$$\begin{aligned} 0 &= \Pr_{\rho, R_{\widehat{P}}}[\langle \widehat{P}(R_{\widehat{P}}), V_\rho\rangle(x) = 1] \\ &= 2^{-r(|x|)} \sum_{\rho} \Pr_{R_{\widehat{P}}}[\langle \widehat{P}(R_{\widehat{P}}), V_\rho\rangle(x) = 1]. \end{aligned}$$

All terms are nonnegative, so each is 0. Consequently, fixing any tape, say $\rho_0 = 0^{r(|x|)}$, preserves both properties. Write $V_0 := V_{\rho_0}$.
Suppose the protocol has k prover turns. For prover messages $a_1, \ldots, a_k$, use the notation

$$\mathrm{Exec}_{V_0}(x; a_1, \ldots, a_k)$$

for the unique execution in which V_0 computes each of its messages from x and the transcript so far, and receives a_i on its ith prover turn. Define an NP verifier V_{NP} whose witness is the polynomial-length sequence

$$w = a_1 \sqcup \cdots \sqcup a_k.$$

It simulates $\mathrm{Exec}_{V_0}(x; a_1, \ldots, a_k)$ and accepts exactly when V_0 accepts.
If $x \in L$, the fixed-tape completeness statement above guarantees an accepting execution; its prover-message sequence is a witness accepted by V_{NP}. If $x \notin L$, every alleged witness w defines a deterministic cheating prover P^*_w: on the transcript prefixes occurring in the simulated execution it sends the next prescribed a_i, and on all other prefixes it acts arbitrarily. Hence

$$V_{\mathsf{NP}}(x, w) = \langle P^*_w, V_0\rangle(x) = 0$$

by fixed-tape perfect soundness. Thus V_{NP} is a polynomial-time NP verifier for L. □

13.3 Sequential Repetition

Let $\Pi = (P, V)$ have perfect completeness and soundness error $1/2$. The n-fold sequential repetition $\Pi^{(n)}$ executes Π on the same input n times in sequence with fresh verifier randomness and accepts only if every execution accepts.

Theorem 13.5 [HW 5, Exercise 3]. $\Pi^{(n)}$ has perfect completeness and soundness error at most

$$2^{-n}.$$

Proof

We prove the theorem by induction on n.
Base. For $n = 1$, the protocol is Π and has perfect completeness and soundness error $1/2 = 2^{-1}$.
Step. Assume the claim for $\Pi^{(n)}$. Perfect completeness of $\Pi^{(n+1)}$ is formalized as follows. Let A be the event that the first n repetitions accept and let B be the event that repetition $n+1$ accepts. For $x \in L$, the induction hypothesis gives $\Pr[A] = 1$. Each repetition is a fresh execution of the honest protocol, so perfect completeness of Π gives $\Pr[B \mid H = h] = 1$ after every history h of the first n repetitions having positive probability. Therefore

$$\Pr[\Pi^{(n+1)}(x) = 1] = \Pr[A \cap B] = 1.$$

For soundness, fix $x \notin L$ and an arbitrary cheating prover. Let W be the set of histories h of the first n repetitions on which they all accept. By total probability,

$$\Pr[\Pi^{(n+1)}(x) = 1] = \sum_{h \in W} \Pr[\Pi^{(n+1)}(x) = 1 \mid h] \Pr[h].$$

After fixing $h \in W$, the original prover together with its state after h induces a cheating strategy P_h^* for the fresh $(n+1)$st execution. The base soundness bound therefore gives

$$\Pr[\Pi^{(n+1)}(x) = 1 \mid h] \leq \frac{1}{2}.$$

Therefore

$$\begin{aligned} \Pr[\Pi^{(n+1)}(x) = 1] &\leq \frac{1}{2} \sum_{h \in W} \Pr[h] \\ &= \frac{1}{2} \Pr[\Pi^{(n)}(x) = 1] \\ &\leq \frac{1}{2} \cdot 2^{-n} = 2^{-(n+1)}. \end{aligned}$$

□

13.4 Quadratic Non-Residuosity

Let $N = pq$ be the product of two distinct odd primes, and define

$$\mathbb{Z}_N^* = \{x \in \mathbb{Z}_N : \gcd(x, N) = 1\}.$$

Define

$$\mathsf{QR}_N = \{x \in \mathbb{Z}_N^* : \exists a \in \mathbb{Z}_N^*,\ a^2 \equiv x \pmod N\}$$

and

$$\mathsf{QNR}_N = \mathbb{Z}_N^* \setminus \mathsf{QR}_N.$$

Claim 13.6. $(\mathbb{Z}_N^*, \cdot_N)$ is a group and

$$\mathsf{QR}_N \trianglelefteq \mathbb{Z}_N^*.$$

Proof

Closure, associativity, and the identity 1 are immediate. For every $a \in \mathbb{Z}_N^*$, Bézout's identity gives $\alpha a + \beta N = 1$, so α is the multiplicative inverse of a modulo N.
If $x \equiv a^2 \pmod N$ and $y \equiv b^2 \pmod N$, then

$$xy \equiv (ab)^2 \pmod N,$$

so $xy \in \mathsf{QR}_N$. Moreover,

$$x^{-1} \equiv (a^{-1})^2 \pmod N,$$

so $x^{-1} \in \mathsf{QR}_N$. Thus QR_N is a subgroup; since $\mathbb{Z}_N^*$ is abelian, it is normal. □

13.4.1 Interactive Protocol

Protocol (Quadratic non-residuosity)

On common input (N, x):

1. the verifier samples
$$y \xleftarrow{R} \mathsf{QR}_N, \qquad b \xleftarrow{R} \{0,1\},$$
computes
$$z = yx^b \pmod N,$$
and sends z to the prover;
2. given the factorization $N = pq$, the prover computes
$$z_p = z^{(p-1)/2} \pmod p, \qquad z_q = z^{(q-1)/2} \pmod q,$$
and sends
$$b' = \begin{cases} 0, & z_p = z_q = 1, \\ 1, & \text{otherwise;} \end{cases}$$
3. the verifier accepts if and only if $b' = b$.

Theorem 13.7 [HW 5, Exercise 4: completeness]. If $x \in \mathsf{QNR}_N$, then

$$\Pr[\langle P, V\rangle(x) = 1] = 1.$$

Proof

If $b = 0$, then $z = y \in \mathsf{QR}_N$, and the honest prover returns $b' = 0$. If $b = 1$, then $z = yx$. Since QR_N is a subgroup, multiplying a non-residue by a residue gives a non-residue, so the prover returns $b' = 1$. In both cases, $b' = b$. □

Theorem 13.8 [HW 5, Exercise 4: soundness]. If $x \in \mathsf{QR}_N$, then for every P^*,

$$\Pr[\langle P^*, V\rangle(x) = 1] \le \frac{1}{2}.$$

Proof

If $b = 0$, then $z = y$ is uniform in QR_N. If $b = 1$, then $z = yx$. Right multiplication by the fixed residue x is a bijection on QR_N, so yx is also uniform in QR_N. More explicitly, for every $z_0 \in \mathsf{QR}_N$ and $c \in \{0,1\}$,

$$\Pr[Z = z_0 \mid B = c] = \frac{1}{|\mathsf{QR}_N|}.$$

Hence Z and B are independent. Let R be the internal randomness of an arbitrary prover P^*; it is sampled independently of (B, Y), so (Z, R) is also independent of B. Since the reply

$$B' = P^*(x, Z; R)$$

is a function of (Z, R), it is independent of B. As B is uniform,

$$\Pr[B' = B] = \sum_{c \in \{0,1\}} \Pr[B' = c] \Pr[B = c] = \frac{1}{2}.$$

Thus every prover succeeds with probability exactly $1/2$. □

Theorem 13.9 [Conditional failure of zero knowledge]. Assume that quadratic residuosity remains hard for probabilistic polynomial-time algorithms even when they are given a valid common input (N, x) with $x \in \mathsf{QNR}_N$ but not the factorization of N. Then the quadratic non-residuosity protocol is not computational zero knowledge against malicious verifiers.

Proof

For $w \in \mathbb{Z}_N^*$, define a malicious verifier V_w^* with w hardwired into its description. It ignores the prescribed first-message algorithm and sends $Z = w$. The honest prover returns the deterministic bit

$$B' = \begin{cases} 0, & w \in \mathsf{QR}_N, \\ 1, & w \notin \mathsf{QR}_N. \end{cases}$$

Thus the real view of V_w^* contains the answer to the quadratic residuosity instance w.
Suppose, toward a contradiction, that a probabilistic polynomial-time simulator S produced a computationally indistinguishable view for every V_w^*. Run S on (N, x) and V_w^*, read the simulated reply bit, and use it to decide whether $w \in \mathsf{QR}_N$. If the simulated bit differed from the real deterministic bit with nonnegligible probability, the polynomial-time test that reads this bit would distinguish the simulated and real views. Hence this procedure decides quadratic residuosity with all but negligible error, contradicting the stated hardness assumption.
The conclusion is therefore conditional on computational hardness. Without such an assumption, the interaction still demonstrates that a malicious verifier learns a residuosity bit, but this observation alone is not a formal impossibility proof for polynomial-time simulation. □

13.5 Zero-Knowledge Proofs

Definition 13.10 [Computational zero knowledge]. An interactive proof (P, V) for L is zero knowledge if, for every cheating verifier V^*, there is a simulator S with the same complexity as V^* such that, for every $x \in L$,

$$\langle P(x) \leftrightarrow V^*(x) \rangle_{V^*} \approx_c S(V^*, x).$$

Theorem 13.11 [GMW87]. Assuming one-way functions, there is a zero-knowledge interactive argument for every $L \in \mathsf{NP}$.

It is sufficient to construct a zero-knowledge interactive proof for one NP-complete language. The GMW protocol uses

$$\mathsf{3COL} = \{(U, E) : \exists \sigma \in [3]^U,\ \forall (u, v) \in E,\ \sigma(u) \neq \sigma(v)\}.$$

13.6 Commitments

Definition 13.12 [Commitment scheme]. A non-interactive commitment scheme consists of the algorithms

$$\begin{aligned} &\mathrm{Commit}(1^\lambda, m; r) \quad \text{(probabilistic)},\\ &\mathrm{Verify}(1^\lambda, c, m, r) \quad \text{(deterministic)}. \end{aligned}$$

It satisfies:

1. *Correctness:* for every m, r,
$$\mathrm{Verify}(1^\lambda, \mathrm{Commit}(1^\lambda, m; r), m, r) = 1.$$
2. *Computational hiding:* for every equal-length m_0, m_1,
$$\{\mathrm{Commit}(1^\lambda, m_0; R)\}_\lambda \approx_c \{\mathrm{Commit}(1^\lambda, m_1; R)\}_\lambda.$$
3. *Binding:* no probabilistic polynomial-time adversary outputs (c, m_0, r_0, m_1, r_1) with $m_0 \neq m_1$ and
$$\mathrm{Verify}(1^\lambda, c, m_0, r_0) = \mathrm{Verify}(1^\lambda, c, m_1, r_1) = 1$$
except with negligible probability.

The graph-coloring protocol uses computational hiding and perfect binding.

13.7 The GMW Protocol for Graph 3-Coloring

The protocol uses the commitment primitive of Definition 13.12 to realize the zero-knowledge condition of Definition 13.10.

Protocol (GMW graph 3-coloring)

On input $(1^\lambda, U, E)$ and a valid coloring σ:

1. the prover samples a random permutation φ on $[3]$;
2. for every $w \in U$, it samples $k_w \leftarrow U_n$, sets
$$c_w = \varphi(\sigma(w)), \qquad C_w = \mathrm{Commit}(k_w, c_w),$$
and sends $(C_w)_{w \in U}$;
3. the verifier samples a random edge $(u, v) \leftarrow E$;
4. the prover opens the commitments at u, v by sending $(k_u, c_u), (k_v, c_v)$;
5. the verifier accepts if and only if
$$c_u \neq c_v$$
and both openings are valid.

Theorem 13.13 [GMW 3-coloring protocol]. The protocol is sound with error

$$1 - |E|^{-1}$$

and is zero knowledge. Sequential repetition reduces the soundness error while preserving zero knowledge.

Proof

Perfect completeness follows from the valid coloring. If the graph is not 3-colorable, perfect binding fixes one color for every vertex, and at least one edge is monochromatic. A uniform edge catches such an edge with probability at least $1/|E|$.
For zero knowledge, the simulator guesses the verifier's challenge edge $e = (u, v)$, chooses distinct random colors for u, v, fixes arbitrary colors for all other vertices, and commits to them. It runs the verifier and outputs the view if the challenge equals e; otherwise it rewinds and tries again. Computational hiding makes unopened commitments indistinguishable from real commitments. After $n|E|$ trials, failure has negligible probability. □

13.8 Succinctness and Zero Knowledge by Composition

Let (P, V) be a public-coin interactive proof.

Instead of sending plaintext prover messages a_i, send commitments

$$c_i = \mathrm{Com}(k_i, a_i).$$

The commitments cannot simply be opened, since this would reveal the transcript. Instead, prove in zero knowledge that valid accepting openings exist.

Define the NP language L_V by

$$(r_1, \ldots, r_n, c_1, \ldots, c_n) \in L_V \iff \exists k_1, \ldots, k_n, a_1, \ldots, a_n$$

such that, for every i,

$$c_i = \mathrm{Com}(k_i, a_i),$$

and

$$V(r_1, \ldots, r_n, a_1, \ldots, a_n) = 1.$$

The prover and verifier then run a zero-knowledge proof that the public random coins and commitments belong to L_V.

13.8.1 Soundness and Proofs of Knowledge

To prove soundness, one would like to transform a cheating compiled prover P^*_{ZK} into a cheating prover for the original argument. The ordinary zero-knowledge proof guarantees only that accepting openings exist, not that they can be efficiently found.

The required stronger property is a proof of knowledge. In the graph 3-coloring protocol, if P^* convinces the verifier with probability greater than $1 - |E|^{-1}$, then rewinding the prover and asking all possible edge challenges extracts valid openings for an entire coloring.

13.9 Removing Interaction

Definition 13.14 [Succinct non-interactive argument]. A succinct non-interactive argument consists of a setup algorithm producing common randomness

$$\mathsf{CRS} \leftarrow \mathrm{Setup},$$

a prover that outputs one proof π from (x, w, CRS), and a verifier that checks (x, π, CRS).

The common randomness may be generated by the verifier or by a trusted party.

13.10 The Fiat–Shamir Heuristic

Let (P, V) be a constant-round public-coin interactive proof. The Fiat–Shamir transformation replaces verifier challenges by outputs of a hash function H.

Protocol (Fiat–Shamir transformation)

For a protocol with verifier messages r_1, r_2 and prover messages a_1, a_2, set

$$\mathsf{CRS} = (H, r_1).$$

The prover computes a_1, sets

$$r_2 = H(a_1),$$

computes a_2, and publishes

$$\pi = (a_1, a_2).$$

The verifier recomputes $r_2 = H(a_1)$ and checks the original transcript.

The Fiat–Shamir heuristic can be proven secure when H is modeled as a random oracle.